

SUMMARIZED BY NOAM KIMHI

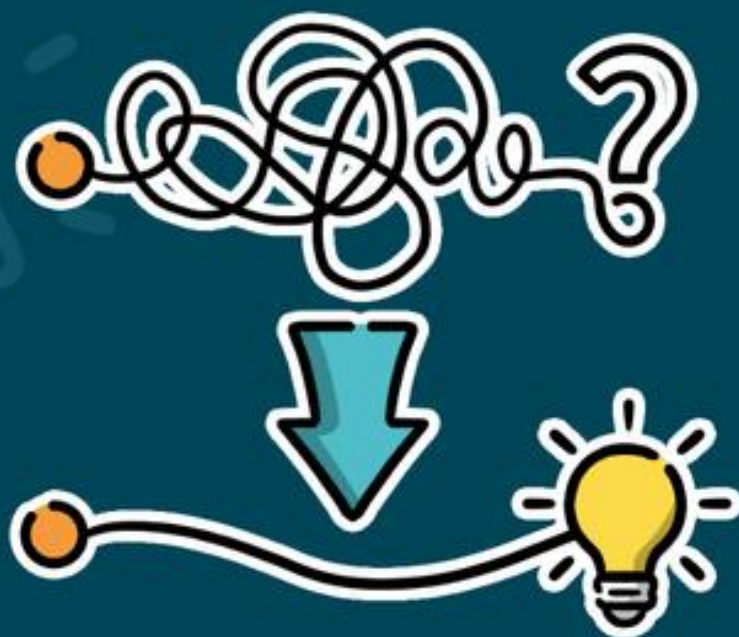
noam.kimhi@mail.huji.ac.il

מודלים חישוביים, חישוביות וסיבוכיות

YEAR 2

SEMESTER B

2025



סיכום קורס

COURSE NUMBER 67521

שבוע 1 – הרצאה	3
מבוא לחומר הקורס והגדרות מרכזיות	3
אוטומט סופי דטרמיניסטי – Deterministic Finite Automaton	4
שבוע 1 – תרגול	8
שבוע 2 – הרצאה	10
שפות רגולריות	10
אוטומט סופי לא דטרמיניסטי – Non Deterministic Finite Automaton	12
שבוע 2 – תרגול	14
שבוע 3 – הרצאה	17
מעברי אפסילון	17
שקילות Myhill-Nerode	20
שבוע 4 – הרצאה	22
מכונת טיורינג – (TM) Turing Machine	22
שבוע 5 – הרצאה	28
קריאה למ"ט כפרוצדורה/פונקציה	28
מכונת טיורינג בקלט למכונת טיורינג	30
בעיית העצירה	31
שבוע 6 – הרצאה	34
יחסים בין השפות R, RE	34
רדוקציה	35
שבוע 7 – הרצאה	39
סיבוכיות	39
מכונת טיורינג לא דטרמיניסטית	41
רדוקציה פולינומיאלית	43
שבוע 8 – הרצאה	45
קשיות ושלמות של מחלקות	46
דוגמאות לשפות NP-קשות	46
מוודא פולינומי	49
שבוע 9 – הרצאה	51
משפט קוק-ליין	52
שבוע 11 – הרצאה	57
משפט ההיררכיה	57

59	 משפט Savitch
62	 סיבוכיות מקום תת-לינארית
64	 שבוע 12 – הרצאה
67	 המחלקות NL, L
70	 שבוע 13 – הרצאה
70	 משפט Savitch – המשך
70	 Immerman-Szelepcsényi
76	 שבוע 14 – הרצאה
76	 מחלקות סיבוכיות עם אקראיות

מבוא לחומר הקורס והגדרות מרכזיות

משימות/בעיות חישוב

- (1) למיין מערך נתון, $O(n \log n)$
 - (2) בהינתן גרף G ושני צמתים s, t , למצוא מסלול קצר ביותר בין s ל- t (דוגמה לבעיה שבה כמה פלטים אפשריים), $\sim O(n^4)$
 - (3) בהינתן קליקה (קבוצת צמתים שמחוברים בקשת כולם אלו לאלו), מהו גודל הקליקה המקסימלית? $O(2^n \cdot n^2)$
 - (4) בהינתן תוכנית פייתון, יש להכריע האם היא עוצרת. אי אפשר לפתור את השאלה הזו, זה ממיר את השאלה מתחום הסיבוכיות לתחום החישוביות.
- המודל החישובי **משנה**. ראינו ב-DAST שמיון מבוסס השוואות מבוצע ב- $O(n \log n)$. באותה מידה, יכולנו להשתמש במודל המקרוני¹, שם אפשר לבצע את המיון ב- $O(n)$. האם זה מודל חישובי סביר? מה נחשב למודל חישובי סביר?

בעיה חישובית

הגדרה – א"ב: קבוצה סופית, לא ריקה Σ .

הגדרה – מילה: מילה מעל א"ב Σ היא מחרוזת של אותיות מתוך Σ .

מילה ללא אותיות כלל היא חוקית, ותסומן על ידי ϵ . האות ϵ לא תיכלל ב- Σ כדי למנוע בלבול.

הגדרה – בעיה חישובית: מורכבת מא"ב קלט Σ , א"ב פלט Γ ופונקציה שמתאימה לכל מילה בקלט קבוצה כלשהי של מילות פלט ("הפלטים החוקיים עבור הקלט").

בעיות הכרעה

הגדרה – בעיית הכרעה: בעיה חישובית שבה א"ב הפלט הוא מהצורה הבאה:

$$\Gamma = (\{T, F\}, \{acc, rej\}, \{Yes, No\})$$

ושבה לכל קלט הבעיה מתאימה קבוצת פלטים חוקיים המכילה מילה אחת בלבד, והמילה תהיה באורך אות אחת.

(בקצרה, לכל קלט התשובה צריכה להיות או כן – או לא).

¹ מודל המקרוני – מודל שבו אם עלינו למיין כמות מסוימת של מספרים, ניקח את אותה הכמות ונשבור מקרוני(ם) לפי הגודל שלהם. נארוז בכף היד וניישר, בכל פעם נוציא את המקרוני הארוך ביותר, וב- m פעולות נוכל למיין את המערך.

דוגמה לבעיית הכרעה:

בהינתן גרף G , צמתים s, t ומספר k , יש להחזיר "כן" אם יש מסלול באורך קטן/שווה k מ- s ל- t בגרף G . כמעט כל בעיית אופטימיזציה טבעית אפשר להמיר באמצעות מספר שינויים לבעיית הכרעה.

מה נוכיח בעתיד?

- אין תוכנית פייתון שפותרת את בעיית העצירה.
- אין אף מודל חישובי ריאליסטי (כזה שיש תקווה ליצור) שפותר את בעיית העצירה.
- יש בעיית הכרעה שאינה פתירה בעזרת תוכנית פייתון.

הגדרה – שפה: קבוצה של מילים.

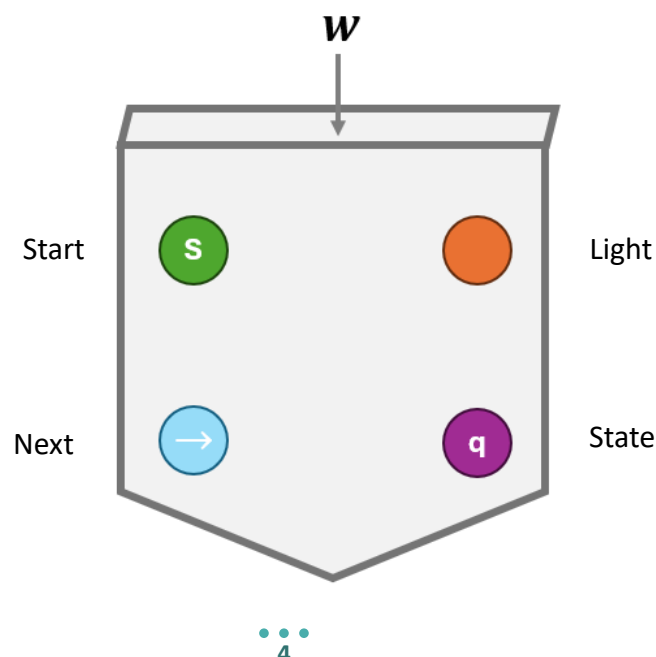
אבחנה: יש התאמה טבעית חח"ע ועל בין בעיות הכרעה מעל Σ לבין שפות מעל Σ . נתאים לבעיית הכרעה המוגדרת בעזרת פונקציה f את השפה L_f המכילה את המילים מעל Σ שעבורן f מחזירה "acc".

ספירה

- $\Sigma - \text{א"ב}$. מספר המילים באורך k מעל Σ הוא: $1 \leq |\Sigma|^k < \infty$.
- מספר המילים הכולל מעל Σ הוא $\aleph_0$.
- מספר השפות מעל Σ הוא: $\aleph_0 < 2^{\aleph_0} = \aleph$. יש $\aleph_0$ תוכניות פייתון. יש $\aleph$ בעיות הכרעה מעל $\Sigma = \{a, b\}$, ומתקיים $\aleph > \aleph_0$ ולכן קיימות בעיות הכרעה מעל Σ שלא ניתנות לפתרון בעזרת תוכניות פייתון. (למה יש $\aleph$ בעיות הכרעה? ראינו שיש $\aleph$ שפות והתאמה חח"ע ועל בין בעיות הכרעה לשפות).

אוטומט סופי דטרמיניסטי – Deterministic Finite Automaton

תרשים להמחשה



נניח $\Sigma = \{a, b\}$ ומילה aba . נקרא משמאל לימין:

$$S \Rightarrow q_0 \Rightarrow a \rightarrow q_1 \Rightarrow b \rightarrow q_2 \Rightarrow a \rightarrow q_3$$

לבסוף, או שנדלקת הנורה אם המכונה קיבלה את הקלט, או שהיא לא נדלקת אם הקלט לא התקבל.

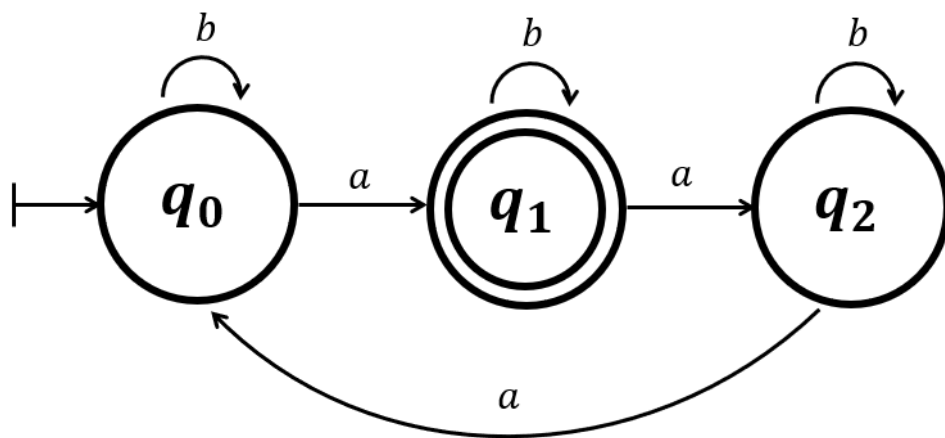
דוגמה:

$\Sigma = \{a, b\}$ ובעיית הכרעה: יש לקבל מילה w אם $\#_a(w) = 1 \pmod{3}$.

למשל נקבל את המילים: $ababab$, $abaaa$, $bbbbba$ ונדחה את המילה: $ababab$.

הגדרה לשפה המתאימה: $L = \{w \text{ over } \{a, b\} \mid \#_a(w) = 1 \pmod{3}\}$

אוטומט מתאים לבעיה:



כאשר עיגול כפול הוא סימון למצב מקבל, וחץ עם קו אנכי הוא מצב התחלתי.

נסמן את האוטומט ב- $\mathcal{A}$. נגדיר את אוסף המילים שהאוטומט מקבל ב- $L(\mathcal{A})$.

הגדרה – אוטומט מכריע שפה: נאמר שהאוטומט $\mathcal{A}$ מכריע את השפה L כאשר מתקיים $L(\mathcal{A}) = L$.

הוכחה שהאוטומט המצויר בתרשים מכריע את $L = \{w \text{ over } \{a, b\} \mid \#_a(w) = 1 \pmod{3}\}$:

נראה כי לכל מילה w שיש בה i מופעים של a מודולו 3, האוטומט יגיע בריצתו עליה למצב q_i (זו טענה

חזקה יותר ממה שנדרשנו להוכיח). נוכיח באינדוקציה על אורך המילה w שננסמן באות n .

בסיס: $n = 0$. עבור $w = \epsilon$ האוטומט מגיע למצב ההתחלתי q_0 ובזה מסיים בדיוק כפי שצריך.

צעד האינדוקציה: תהי w מילה באורך $n + 1$ ונכתוב את w בתור $w = w'\alpha$ כאשר w' מילה באורך n ו- α אות מבין $\{a, b\}$.

על w' האוטומט מגיע למצב q_i כאשר $i = \#_a(w') \pmod{3}$, מהנחת האינדוקציה.

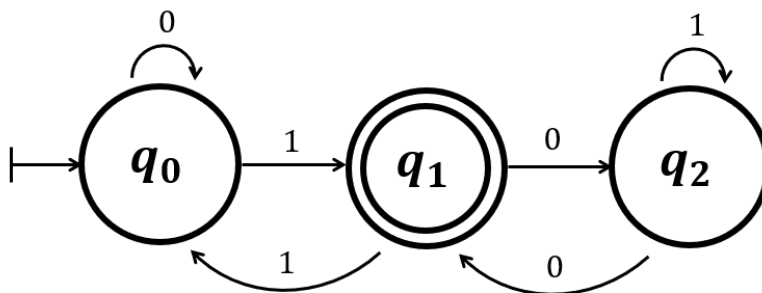
אם $\alpha = b$, האוטומט יישאר ב- q_i וזה תואם לטענה.

אחרת $\alpha = a$, האוטומט יעבור מ- q_i ל- q_j כאשר $j = i + 1 \pmod{3}$ ואכן $\#_a(w) = i + 1 \pmod{3}$.

דוגמה:

א"ב $\Sigma = \{0,1\}$. שפה $L = \{w \text{ over } \Sigma \mid (w)_2 = 1 \bmod 3\}$ (כאשר זהו הסימון לצורתו הבינארית של המספר w).

הרעיון: בריצתו על מילה w האוטומט יגיע למצב q_i כאשר $i = (w)_2 \bmod 3$ למשל 110 יתאים ל- q_0 .



אפשר לחשוב על הבעיה כקריאה מימין לשמאל, ובכל הוספה של ספרה 0 או 1 לחשוב מה יקרה למספר שהיה שייך ל- q_i לפני ההוספה.

שאלות על אוטומטים

- (1) האם ניתן להכריע את שפת הייצוגים הבינאריים של מספרים ראשוניים? לא.
- (2) האם יש אפיון כללי פשוט ומעניין לאוסף הבעיות שניתן להכריע בעזרת אוטומט? כן.
- (3) אם ניתן להכריע את השפה L ואת השפה L' בעזרת DFA – מה ניתן להגיד על $L \cup L'$? $L \cap L'$? $L \circ L'$?

הגדרה – אוטומט סופי דטרמיניסטי DFA: אוטומט A הוא החמישייה $A(\Sigma, Q, q_0, F, \delta)$ כאשר:

Σ – א"ב.

Q – קבוצה סופית לא ריקה של מצבים.

q_0 – מצב התחלתי.

$F \subseteq Q$ – קבוצת מצבים מקבלים.

$\delta: Q \times \Sigma \rightarrow Q$ פונקציית מעברים

הגדרה – ריצה של אוטומט: הריצה של A על מילה $w = \alpha_1 \alpha_2 \dots \alpha_n$ היא סדרת המצבים $q_0, \dots, q_n$

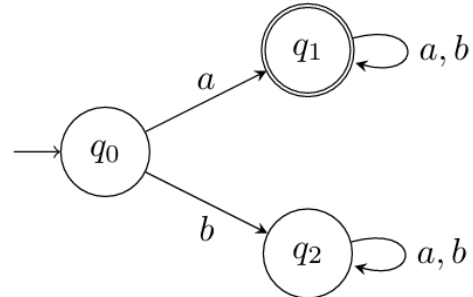
שמקיימת $q_i = \delta(q_{i-1}, \alpha_i)$, המצב ההתחלתי, q_0 .

הגדרה – פונקציית המעברים המורחבת של A : מוגדרת להיות $\delta^*(q_0, w) = q_n$ כאשר w היא מילה מעל הא"ב, ומחזירה את המצב שהאוטומט מגיע אליו לאחר מעבר על כל n האותיות במילה w .

שבוע 1 – תרגול

הוכחה שאוטומט מכריע שפה

הוצגה הוכחה שהאוטומט הבא:



מכריע את השפה $L = \{w \in \Sigma^* \mid w \text{ starts with 'a'}\}$

טענה: $L(A) = L$

נוכיח טענת עזר: לכל מצב המילים הבאות מסיימות את ריצתן במצב:

q_0 – המילה הריקה.

q_1 – מילים המתחילות באות a

q_2 – מילים לא ריקות או שאינן מתחילות ב- a

$w \in L \Leftrightarrow w$ מתחילה באות $a \Leftrightarrow A$ מסיימת ריצה על w במצב $q_1 \Leftrightarrow A$ מקבל את $w \Leftrightarrow w \in L(A)$.

נוכיח את טענת העזר באינדוקציה על אורך המילה w :

בסיס: עבור $|w| = 0$ זו המילה הריקה, ההרצה על w באוטומט נשארת במצב q_0 .

נניח עבור כל מילה באורך n ונוכיח עבור $w \in \Sigma^*$ כך ש- $|w| = n + 1$.

צעד: $q_0 - w' - q_0$ היא המילה הריקה ונקבל $\delta(q_0, w'\sigma) = \delta(q_0, \sigma)$.

אם $\sigma = a$ אז $\delta_0(q_0, a) = q_1$ ומאחר ש- $w = a$ בפרט מתחיל באות a , ואכן סיים ב- q_1 .

אם $\sigma = b$ אז $\delta_0(q_0, a) = q_2$ ומאחר ש- $w = b$ בפרט לא מתחילה באות a , ואכן סיים ב- q_2 .

$q_1 - w' - q_1$ התחילה באות a מהנחת האינדוקציה. $\delta(q_1, \sigma) = q_1$ בפרט גם w מתחילה ב- a ואכן A סיים את הריצה על w ב- q_1 .

את q_2 מראים באופן דומה.

$$\delta^*(q, w) \stackrel{\text{def}}{=} \begin{cases} q & w = \epsilon \\ \delta(\delta^*(q, w'), \sigma) & w = w'\sigma \end{cases}$$

סימון – שרשור מילים: יהיו $w_1, w_2 \in \Sigma^*$ מילים מעל א"ב Σ . שרשור שלהן מוגדר להיות $w_1 \cdot w_2$.

טענה: יהי אוטומט $A\langle Q, \Sigma, \delta, q_0, F \rangle$. לכל $q \in Q, w, w' \in \Sigma^*$ מתקיים:

$$\delta^*(q, ww') = \delta^*(\delta^*(q, w), w')$$

הוכחה: באינדוקציה על $|w'|$

בסיס: $|w'| = 0$ כלומר $w' = \epsilon$.

$$\delta^*(\delta^*(q, w), w') = \delta^*(\delta^*(q, w), \epsilon) = \delta^*(q, w) = \delta^*(q, w\epsilon) = \delta^*(q, ww')$$

הנחה: נניח נכונות עבור כל $|w'| = n$. נוכיח עבור $|w'| = n + 1$.

צעד: נגדיר $w' = \bar{w}\sigma$ כאשר $|\bar{w}| = n$ ו- $\sigma \in \Sigma$.

$$\begin{aligned} \delta^*(q, ww') &= \delta^*(q, w\bar{w}\sigma) = \delta(\delta^*(q, w\bar{w}), \sigma) = \delta(\delta^*(\delta^*(q, w), \bar{w}), \sigma) = \\ &= \delta^*(\delta^*(q, w), \bar{w}\sigma) = \delta^*(\delta^*(q, w), w') \end{aligned}$$

הגדרה – שפה רגולרית: שפה L תקרא רגולרית אם קיים אוטומט סופי דטרמיניסטי כך ש- $L(A) = L$.

שפות רגולריות

שאלות על REG

(1) האם $L_1, L_2 \in REG$ גורר שגם $L_1 \cap L_2 \in REG$? $L_1 \cup L_2 \in REG$? $\overline{L_1} \in REG$?

(2) האם אפשר לקבל אפיון מלא לאוסף השפות הרגולריות?

חזרה על δ ועל δ^*

נגדיר אוטומט $A = \langle Q, \Sigma, q_0, F, \delta \rangle$. $\delta(q, \alpha)$ הוא המצב אליו נעבור מ- q כאשר קוראים את α .

נגדיר $\alpha w = w'$ אז $\delta^*(q, w) = \delta(\delta^*(q, w'), \alpha)$ אם $w \neq \epsilon$ ואחרת $\delta^*(q, \epsilon) = q$.

$\delta^*(q, w)$ – המצב שאליו נגיע אם נתחיל ב- q ונעבור על כל האותיות ב- w .

למה δ^* שימושי? בין היתר להגדרות כמו - $L(A) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}$ (מקצר כתיבה).

איחוד וחיתוך של שפות רגולריות

טענה: בהינתן $L_1, L_2 \in REG$ אז גם $L_1 \cup L_2 \in REG$ (ונסיק במקביל $L_1 \cap L_2 \in REG$).

הוכחה: בה"כ ניתן להניח כי L_1 ו- L_2 מעל אותו א"ב Σ .

מדוע ניתן להניח זאת? אם L_1 מעל Σ_1 ו- L_2 מעל Σ_2 נוכל להגדיר $\Sigma = \Sigma_1 \cup \Sigma_2$. עכשיו צריך להסביר למה L_1 ו- L_2 עדיין שפות רגולריות גם מעל Σ . צריך להגדיר אוטומט חדש שבו יש מצב "בור" (sink) שלא מקבל, ואם הכנסנו אות שזרה לא"ב המקורי של השפה, נשלח למצב הבור.

נסמן $A = \langle Q, \Sigma, q_0, F, \delta \rangle$, $L(A) = L_1$, $B = \langle P, \Sigma, p_0, G, \eta \rangle$, $L(B) = L_2$.

הרעיון: נבנה אוטומט עם קבוצת מצבים $Q \times P$. על מילה w האוטומט יגיע למצב

$(\delta^*(q_0, w), \eta^*(p_0, w))$. האוטומט שנבנה הוא:

$$C = \langle Q \times P, \Sigma, (q_0, p_0), F \times P \cup Q \times G, \psi \rangle$$

מספיק להוכיח שהרעיון מתקיים כדי לסיים. זה יראה שכשאנחנו מריצים את האוטומט על מילה w זה ייתן את המצבים שלהם בסוף הריצה, ולפי האופן בו הגדרנו את קבוצת המצבים המקבילים. לכן רק נגדיר את ψ על ידי: $\psi((q, p), \alpha) = (\delta(q, \alpha), \eta(p, \alpha))$. נסיים בכך שנראה שלכל מילה w מתקיים:

$$\psi^*((q_0, p_0), w) = (\delta^*(q_0, w), \eta^*(p_0, w))$$

נוכיח באינדוקציה על $|w| = n$.

בסיס: $n = 0$ ולכן $w = \epsilon$.

$$\psi^*((q_0, p_0), \epsilon) = (q_0, p_0) \stackrel{\text{הגדרה}}{=} (\delta^*(q_0, \epsilon), \eta^*(p_0, \epsilon))$$

הנחה: נניח את נכונות הטענה עבור $|w| = n$.

צעד: נוכיח עבור $|w| = n + 1$.

$$\begin{aligned} \psi^*((q_0, p_0), w) &= \psi^*((q_0, p_0), w'\alpha) \stackrel{\text{def } \psi}{=} \psi(\psi^*((q_0, p_0), w'), \alpha) \stackrel{I.H}{=} \\ &= \psi((\delta^*(q_0, w'), \eta^*(p_0, w')), \alpha) \stackrel{\text{def } \psi}{=} \psi(\delta(\delta^*(q_0, w'), \alpha), \eta(\eta^*(p_0, w'), \alpha)) = \\ &= \psi(\delta^*(q_0, w'\alpha), \eta^*(p_0, w'\alpha)) \stackrel{\text{def } w}{=} \psi(\delta^*(q_0, w), \eta^*(p_0, w)) \end{aligned}$$

זה נקרא **אוטומט מכפלה** בגלל העובדה שיצרנו את קבוצת המצבים המקבילים שלו עם מכפלה קרטזית.

שרשור שפות

הגדרה – שרשור שפות: L_1, L_2 שפות מעל Σ (זה אינו תנאי מחייב). נגדיר את השרשור של השפות להיות

$$L_1 \circ L_2 = \{wz \mid w \in L_1, z \in L_2\}$$

מעט דוגמאות וסימונים:

$$\{\epsilon\} \circ L_1 = L_1 \circ \{\epsilon\} = L_1$$

$$L_1^0 = \{\epsilon\}$$

$$L_1^k = \overbrace{L_1 \circ \dots \circ L_1}^{k \text{ times}}$$

$$L^m \circ L^k = L^{m+k}$$

$$L \circ \emptyset = \emptyset$$

הגדרה – סגור קליני של שפה: סגור קליני של שפה L יסומן L^* ומוגדר להיות $L^* = \bigcup_{k=0}^{\infty} L^k$

תרגיל:

(1) $L = \{a^n \mid n \text{ is even}\}$ אז $L^* = L$. כל מילה מ- L שייכת גם ל- L^* . אם נשרשור כמה מילים של L ,

מאחר שהאורך של כולן זוגי אז גם השרשור שלהן באורך זוגי.

(2) $L = \{a\}$ אז $L^* = \{a^n \mid n \geq 0\}$.

(3) $L = \{a, b\}$ אז $L^* = \{a, b\}^*$.

יצירת שפות רגולריות

טענה – פעולות רגולריות:

$$\emptyset, \{\epsilon\}, \{a\} \in REG \quad (1)$$

(2) REG סגור לאיחוד, שרשור וסגור קליני.

(3) כל שפה ב- REG ניתן לקבל משפות מסוג (1) בעזרת פעולות מסוג (2).

דוגמה:

$$L = \{w \mid \text{the sequence } aaa \text{ is in } w\}$$

ניצור ביטוי רגולרי:

$$L = \underbrace{(\{a\} \cup \{b\})^* \circ \{a\} \circ \{a\} \circ \{a\} \circ (\{a\} \cup \{b\})^*}_{\substack{\text{words in } \{a,b\} \quad \text{the language with} \\ \text{aaa} \\ \text{words in } \{a,b\} \text{ that end with } aaa \\ \text{words in } \{a,b\} \text{ with the sequence } aaa \text{ in them}}}$$

מבט קדימה

כעת נגדיר "מודל חישובי" חדש NFA , נגדיר את $NREG$ (כל השפות שמזוהות על ידי NFA), נראה ש- $NREG$ סגור לשרשור ולסגור קליני ואז נסיים בכך שנראה $REG=NREG$.

אוטומט סופי לא דטרמיניסטי – Non Deterministic Finite Automaton

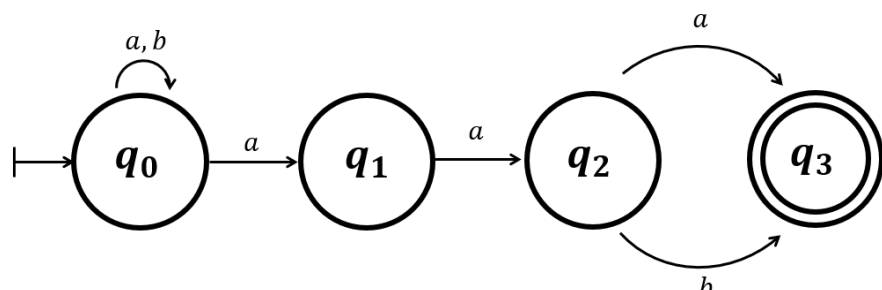
NFA

בהינתן מילה, יש לאוטומט כזה קבוצה של ריצות חוקיות (תתכן ריצה אחת חוקית, מספר ריצות, או קבוצה ריקה). מגדירים שה- NFA שנסמן ב- A מקבל אות w אם יש לו ריצה על A שמסתיימת במצב מקבל.

דוגמה:

$$L_1 = \{w \in \{a, b\}^* \mid w \text{ ends with } aaa \text{ or with } aab\}$$

הרעיון: נצא מהמצב ההתחלתי רק על האות השלישית לפני הסוף.



נשים לב שב- q_1 וב- q_2 לא מוגדר מה קורה אם ניתקל באות b , ולכן זו תהיה ריצה לא חוקית אם נגיע למצב כזה.

דוגמת הרצה:

	a	b	a	a	b
q_0	q_0	q_0	q_0	q_0	q_0
q_0	q_1	X	X	X	X
q_0	q_0	q_0	q_1	q_2	q_3

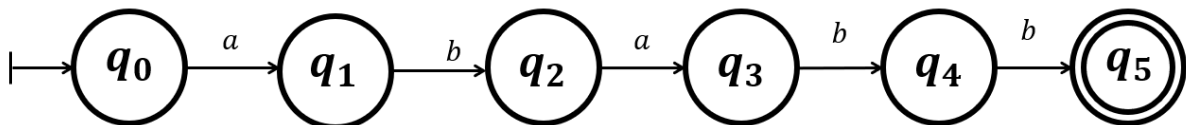
הגדרה – אוטומט מזהה שפה: אומרים על NFA שהוא מזהה את השפה $L(A)$.

איחוד שפות NREG

יהיו $L, L' \in NREG$. האם $L \cup L' \in NREG$? כן. נבנה אוטומט שמזהה את השפה $L \cup L'$ באמצעות חיבור ה- NFA של השפות המקוריות.

דוגמה:

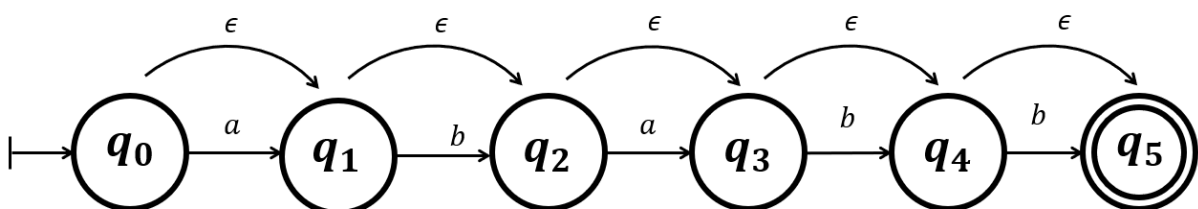
נתונה שפה $L_2 = \{ababb\}$ ונתון אוטומט לא דטרמיניסטי שמזהה אותה:



אם ננסה להריץ למשל את המילה $ababbb$ זו לא מילה שתקבל ריצה חוקית – וזה בסדר כי היא לא בשפה.

נגדיר: $L_3 = \{\text{all the words received by erasing letters from 'ababb'}\}$

איך ניצור אוטומט ל- L_3 ? נוסיף מעברי ϵ לאוטומט של L_2 . מעברים אלו מאפשרים לנו להתחיל לא רק מ- q_0 .



למשל עבור $bab \in L_3$ נדלג על q_0 ישיר ל- q_1 , משם נוכל או להגיע ל- q_2 עם ניצול האות b , או עם מעבר ϵ בלי ניצול של האות b .

שבוע 2 – תרגול

הגדרה פורמלית של NFA

הגדרה – NFA: $A = \langle Q, \Sigma, \delta, Q_0, F \rangle$ הוא חמישייה כדלל ש-

Q - קבוצת מצבים.

Σ - א"ב.

$\delta: Q \times \Sigma \rightarrow 2^Q$ פונקציית מעברים.

$Q_0 \subseteq Q$ קבוצת מצבים התחלתיים.

$F \subseteq Q$ - קבוצת מצבים מקבלים.

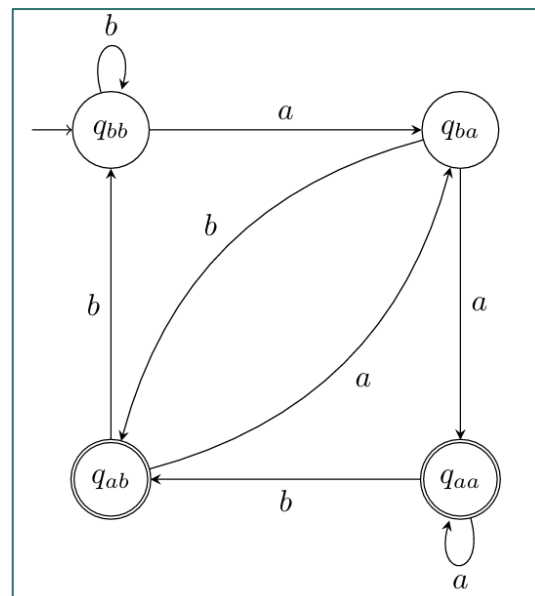
ריצה של NFA

ריצה של NFA שנסמן ב- A על מילה $w = w_1 \dots w_n$ היא $r_0, r_1, \dots, r_n \in Q$ כך ש- $r_0 \in Q_0$ וגם מתקיים $r_{i+1} \in \delta(r_i, w_{i+1})$ לכל $0 \leq i < n$. ריצה מקבלת את $r_n \in F$. מזהה את w אם קיימת ריצה מקבלת שלו עליה.

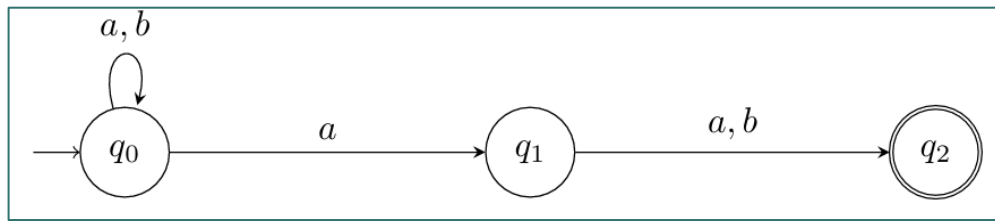
דוגמה:

$L_k = \{w \in \Sigma^* \mid \text{the } k^{\text{th}} \text{ letter of } w \text{ from the end is } a\}$ נגדיר $k \in \mathbb{N}$ ולכל $\Sigma = \{a, b\}$

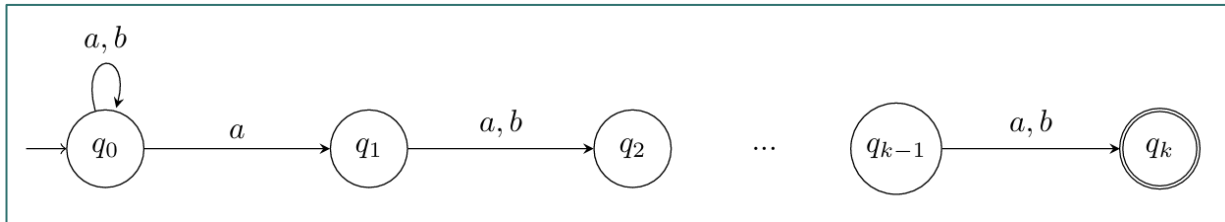
(1) DFA שמכריע את L_2 :



(2) NFA שמזהה את L_2 :



(3) עבור k כללי, הראו שיש NFA עם $k + 1$ מצבים שמזהה את L_k :



נרצה לאפיין לכל תת-קבוצה של מצבים אילו מילים w יכולות לסיים את ריצתן ב- Q' . כל מילה יכולה לסיים ב- q_0 , למילה w נסמן $i_1, \dots, i_l$ את המיקומים של a מהסוף רק ב- k האותיות האחרונות, ונטען שהמצבים אליהם תגיע הם $\{q_0, q_{i_1}, \dots, q_{i_l}\}$.

(4) עבור k כללי, הראו שכל DFA שמכריע את L_k הוא בעל לפחות 2^k מצבים.

נניח בשלילה ש- A עם פחות מ- 2^k מצבים כך ש- $L(A) = L_k$. נשים לב שיש 2^k מילים באורך k , לכן מעקרון שובר היונים קיימות 2 מילים $u \neq v$ שמסיימות את ריצותיהן באותו מצב. לכל $w \in \Sigma^*$ מתקיים כי uw ו- vw מסיימות באותו מצב ולכן הן מקבלות או נדחות ביחד. אם הן שונות באינדקס i כלשהו (וחייב להיות כזה, אחרת זו אותה מילה והנחנו שהן שונות) אז נסיף מילה באורך $i - 1$, ואז אם $u[i] = a$ ו- $v[i] = b$ אז $uw \in L_k$ בעוד ש- $vw \notin L_k$ בסתירה.

סגירות NREG לאיחוד

$L_1, L_2 \in NREG \iff L_1 \cup L_2 \in NREG$ מתקיים מעל Σ .

הוכחה:

נתון $L_1, L_2 \in NREG$ ולכן קיים $A = \langle Q, \Sigma, \delta, Q_0, F \rangle$ עבורו $L(A) = L_1$ וכן $B = \langle P, \Sigma, \eta, P_0, G \rangle$ עבורו $L(B) = L_2$.

נגדיר $C = \langle Q \cup P, \Sigma, \alpha, Q_0 \cup P_0, F \cup G \rangle$, כאשר $\alpha(Q, \sigma) = \begin{cases} \delta(q, \sigma) & q \in Q \\ \eta(q, \sigma) & q \in P \end{cases}$

תהי $w = w_1 \dots w_n$ ו- $r_0, \dots, r_n$ הריצה של C על w . נראה שכל הריצה מובלת או ב- Q או ב- P .

עבור $r_0 \in Q_0$ אפשר להראות (באינדוקציה) $\alpha(r_i, w_{i+1}) = \delta(r_i, w_{i+1}) \in Q$ ובאותו אופן אם $r_0 \in P_0$.

נוכיח ש- $L(C) = L_1 \cup L_2$ בהכלה דו כיוונית.

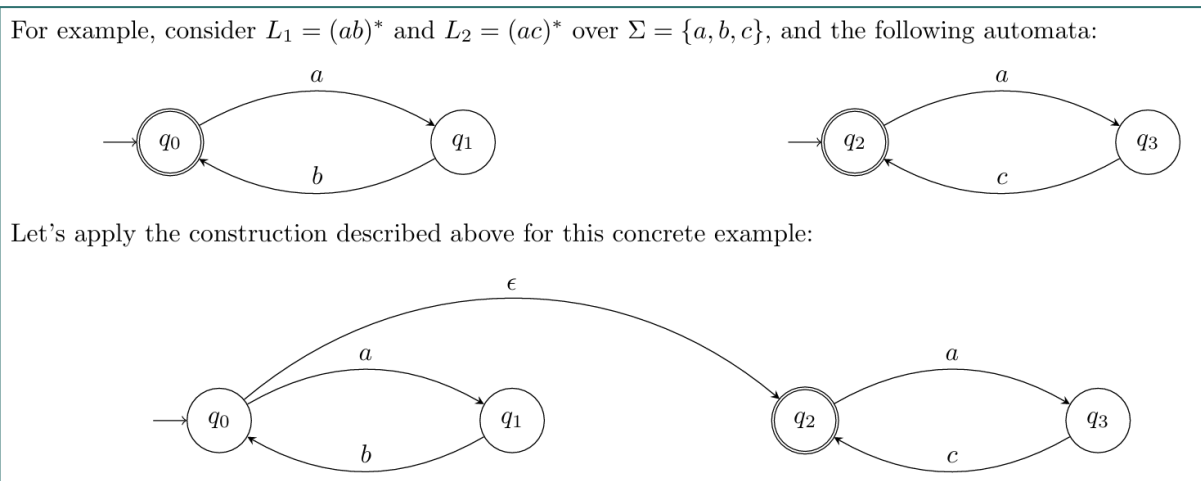
(1) תהי $w \in L_1 \cup L_2$, בה"כ $w \in L_1$. קיימת ריצה מקבלת של A על w ונסמנה r ריצה מקבלת של C על w .

(2) תהי $w \in L(C)$. קיימת ריצה מקבלת של C על w . בה"כ $r_0 \in Q_0$, לכן הריצה r מוכלת כולה ב- Q וזו ריצה מקבלת גם שם, לכן $L(C) = L_1 \cup L_2$.

סגירות NREG לשרשור

שפה NREG מקיימת סגירות לשרשור. באותם סימונים כמו קודם, נגדיר: $\langle Q \cup P, \Sigma, \alpha, Q_0, G \rangle$ עם α פונקציית מעברים שהיא כל המעברים של A , כל המעברים של B ומעבר אפסילון בין כל מצב מקבל ב- A לכל מצב התחלתי ב- B .

דוגמה מהתרגול:



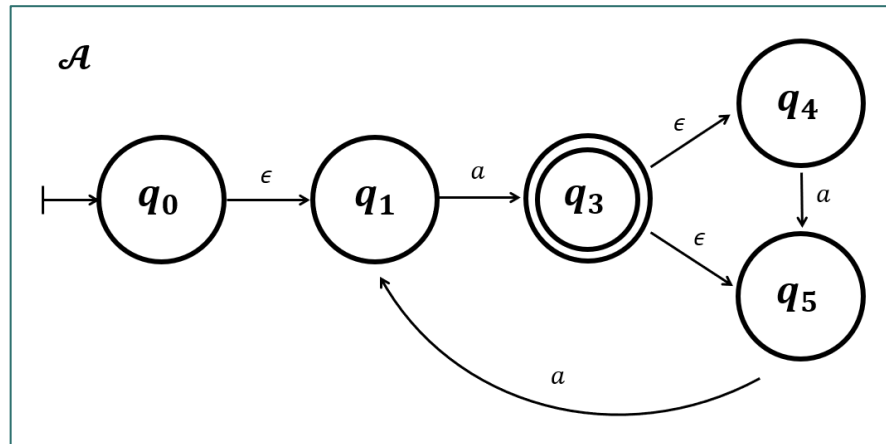
נרצה לתאר אוטומט ללא שימוש במעברי אפסילון, כיצד נשיג זאת? השינוי מאוטומט עם מעבר אפסילון לבלי הוא הוספת חץ התחלה למצב היעד של מעבר ה- ϵ , ובדוגמה מאחר שכל מעבר ל- q_0 יכול היה להוביל גם ל- q_2 הוספנו מעבר מ- q_1 ל- q_2 באמצעות b .

הכללה של הרעיון: לכל מצב q נגדיר $E(q) = \{q' \in Q \mid q' \text{ is reachable from } q \text{ using } \epsilon\}$ ואז $Q_0 = \bigcup_{q \in P_0} E(q)$. נשנה גם את קבוצת המצבים ההתחלתיים $\delta(q, \sigma) = \bigcup_{q' \in \eta(q, \sigma)} E(q')$.

הגדרה – פונקציית מעברים מורחבת δ^* ב-NFA: פונקציית מעברים מורחבת מוגדרת באופן הבא:

$$\delta^*(S, w) = \begin{cases} S & w = \epsilon \\ \bigcup_{q \in \delta^*(S, w')} \delta(q, \sigma) & w = w' \cdot \sigma \end{cases}$$

דוגמה:



$$\mathcal{A} = \langle \Sigma, Q, Q_0, F, \delta \rangle$$

$Q_0 = \{q_0, q_1\}$ – בגלל מעבר ה- ϵ מ- q_0 ל- q_1 , אפשר להתחיל גם ב- q_1 .

$\delta: Q \times \Sigma \rightarrow 2^Q$ – מספר דוגמאות בה: $\delta(q_5, a) = \{q_1\}$, $\delta(q_0, a) = \{q_4, q_3, q_1, q_5\}$

ריצה חוקית על $w \in \Sigma^*$ ממצב $q \in Q$: עבור $w = w_1 w_2 \dots w_k$, ריצה חוקית כ"ל היא סדרת מצבים $q^0, q^1, \dots, q^k$ אשר מקיימת $q^0 = q$ ו- $q^i \in \delta(q^{i-1}, w_i)$.

ריצה על w : ריצה חוקית על w שמתחילה באיזשהו $q^0 \in Q$.

פונקציית המעברים המורחבת δ^* : פונקציה $\delta^*: 2^Q \times \Sigma^* \rightarrow 2^Q$, כאשר $\delta^*(S, w)$ הוא:

(1) אוסף המצבים שניתן להגיע אליהם בריצה חוקית על w שמתחילה במצב כלשהו מ- S .

(2) מוגדר להיות S אם $w = \epsilon$, ואחרת אם $w = w'\alpha$ אז: $\bigcup_{q \in \delta^*(S, w')} \delta(q, \alpha)$.

טענה: הגדרות (1) ו-(2) שקולות לכל $w \in \Sigma^*$.

הוכחה: באינדוקציה על אורך המילה $|w|$, ראינו בעבר הוכחות דומות, בסיס באמצעות $w = \epsilon$, צעד באמצעות $w = w'\alpha$.

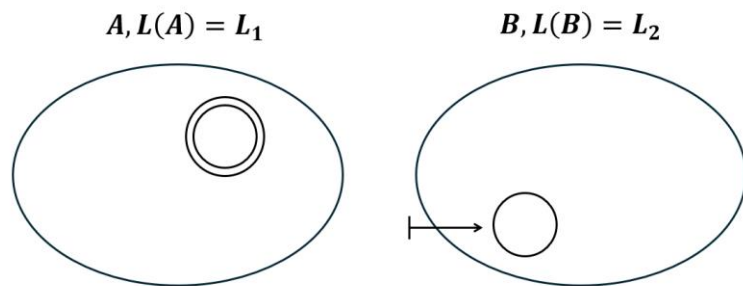
דוגמה:

בחזרה לדוגמה עם האוטומט $\mathcal{A}$ לעיל, נקבל:

$$\delta^*({q_1, q_3}, aa) = \{q_5, q_1, q_3, q_4\}$$

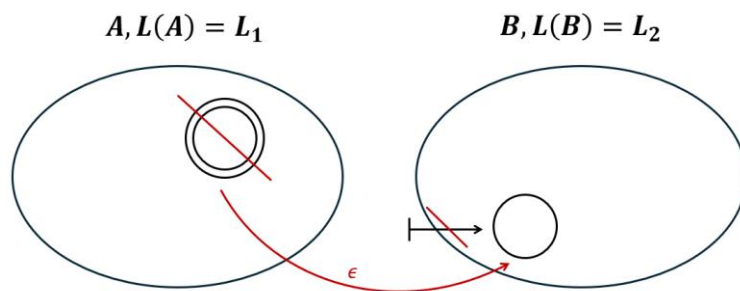
כאשר אל q_5, q_1 הגענו דרך q_1 , ואל q_1, q_3, q_4 דרך q_3 .

נתונים ה- $NFAs$ (חלקיים) הבאים המגדירים את השפות:

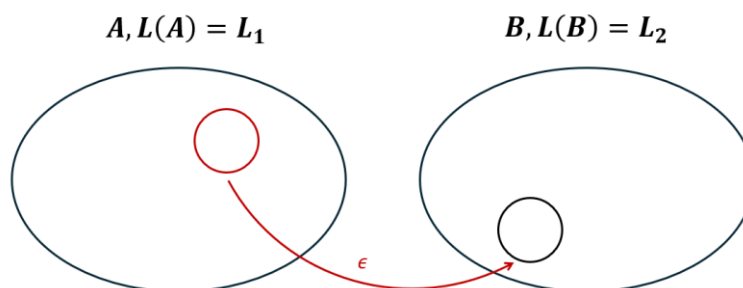


כיצד ניצור אוטומט לשרשור שלהם?

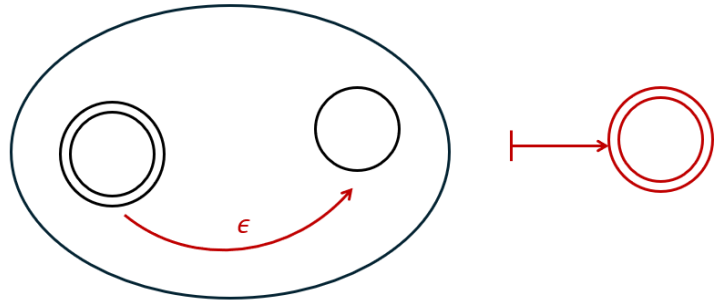
נוסיף מעברי ϵ מכל מצב מקבל ב- A למצב התחלתי ב- B . נבטל את המצבים ההתחלתיים ב- B להיות מצבים רגילים. נבטל את המצבים המקבלים ב- A להיות מצבים רגילים:



לבסוף נקבל:



נוסיף מעבר ϵ מכל מצב מקבל למצב התחלתי. נוסיף מצב התחלתי נוסף עבור מצבים של המילה הריקה.



טענה מרכזית: $NREG = REG$

הוכחה: נוכיח באמצעות הכלה דו-כיוונית.

$REG \subseteq NREG$ – הוכח במסגרת תרגיל הבית.

$REG \supseteq NREG$ – תהא $L \in NREG$ ויהי $A = \langle \Sigma, Q, Q_0, F, \delta \rangle$ אוטומט לא דטרמיניסטי המכריע אותה.

נבנה אוטומט דטרמיניסטי A_d המכריע את L על ידי $A_d = \langle \Sigma, 2^Q, Q_0, F_d, \delta_d \rangle$.

הרעיון: ב- A_d יהיה מצב עבור כל תת-קבוצה $S \subseteq Q$. בריצתו על מילה w , A_d יגיע למצב $\delta^*(Q_0, w)$ (זו קבוצת מצבים). באופן פורמלי:

$F_d = \{S \subseteq Q \mid S \cap F \neq \emptyset\}$ – קבוצת מצבים מקבלים על ידי:

$\delta_d(S, \alpha) = \delta^*(S, \alpha)$ – פונקציית מעברים המוגדרת על ידי:

טענה: $L(A_d) = L(A)$

מספיק להראות $\delta_d^*(Q_0, w) = \delta^*(Q_0, w)$ כי אז נקבל:

$$w \in L(A_d) \Leftrightarrow \delta_d^*(Q_0, w) \in F_d \Leftrightarrow \delta^*(Q_0, w) \in F_d \Leftrightarrow \delta^*(Q_0, w) \cap F \neq \emptyset \Leftrightarrow w \in L(A)$$

נוכיח באינדוקציה על אורך המילה $|w|$.

בסיס: $\delta_d^*(Q_0, \epsilon) = Q_0 = \delta^*(Q_0, \epsilon)$. $|w| = \epsilon$.

צעד: נסמן $w = w'\alpha$, נקבל:

$$\begin{aligned} \delta_d^*(Q_0, w'\alpha) &= \delta_d(\delta_d^*(Q_0, w'), \alpha) \stackrel{I.H}{=} \delta_d(\delta^*(Q_0, w'), \alpha) \stackrel{\text{def } \delta_d}{=} \delta^*(\delta^*(Q_0, w'), \alpha) \\ &= \delta^*(Q_0, w'\alpha) = \delta^*(Q_0, w) \end{aligned}$$

מסקנה: $NREG = REG$.

שאלה: האם הניפוח האקספוננציאלי של מספר המצבים הכרחי? כן.

דוגמה:

$\Sigma = \{0,1\}$ עבור $k \in \mathbb{N}$, נגדיר $L_k := \Sigma^* \circ \{1\} \circ \Sigma^{k-1}$ (כלומר, האות ה- k מהסוף היא 1).

שקילות Myhill-Nerode

הוצגה ההגדרה הבאה:

הגדרה – שקילות MN: L שפה מעל Σ , ו- $x, y \in \Sigma^*$ (לא בהכרח בשפה/מחוצה לה). נגדיר ש- x ו- y שקולות MN ביחס לשפה L אם לא קיימת סיומת מפרידה $z \in \Sigma^*$, כך ש- $xz \in L$ ו- $yz \notin L$, או $xz \notin L$ ו- $yz \in L$. במקרה זה נסמן כי: $x \sim_L y$. אם קיימת סיומת מפרידה z , אז $x \not\sim_L y$.

אבחנות:

- (1) נניח ש- $x \sim_L y$, ו- $x \in L$, אז $y \in L$. תקף גם עם $\notin$.
- (2) נניח ש- $x \not\sim_L y$ וש- $L = L(A)$, אז בריצתו על x ועל y , A מגיע למצבים שונים.
- (3) אם יש n מילים ב- Σ^* שאינן שקולות MN, אז ל-DFA שמכריע את L יש לפחות n מצבים. בניסוח אחר: אם ביחס $\sim_L$ יש k מחלקות שקילות, אז ב-DFA שמכריע את L יש לפחות k מצבים.

הערה: אם יודעים $x \sim_L y$ אז מחלקות השקילות של מילים אלו שוות: $[x]_L = [y]_L$. למשל, $L = \{a, aa\}$, המילה $a = z$ סיומת מפרידה, כי $aa \in L$, $aaa \notin L$. מזה אפשר להסיק בעזרת (3) שיש לפחות 2 מצבים ב-DFA שמזהה את השפה L .

מסקנה:

אם יש ∞ מחלקות שקילות $\sim_L$, אז $L \notin REG$. אם ב- $\sim_L$ יש לפחות k מחלקות, אז ב-DFA של L יש לפחות k מצבים, לכן אם יש אוטומט A עם n מצבים שמכריע את L אז מספר המחלקות ב- $\sim_L$ לכל היותר n .

חזרה לדוגמה של L_k :

עבור כל שתי מילים שונות $x, y \in \{0,1\}^k$, $x \not\sim_{L_k} y$ עבור מילים כאלו קיים $1 \leq i \leq k$ עבורו $x_i \neq y_i$ (אחרת היו זהות), נקבע את z להיות $\{0\}^{k-i-1}$, ואז האות ה- k מהסוף ב- xz היא x_i והאות ה- k לפני הסוף ב- yz היא y_i . $x_i \neq y_i$ ולכן רק אחד מבין xz ו- yz יהיו בשפה L_k .

מכאן נסיק – אוטומט שמכריע את L_k הוא בעל לכל הפחות 2^k מצבים.

דוגמה: $\Sigma = \{a,b\}$, $L = \{a^n b^n \mid n \geq 0\}$, עבור כל $m \neq n$, $a^n \not\sim_L a^m$. למה? נבחין כי $a^n b^n \in L$, לעומת $a^m b^n \notin L$. יש מספר אינסופי של מחלקות שקילות $L \notin REG$.

טענה: אם L שפה מעל Σ^* וב- $\sim_L$ יש $k < \infty$ מחלקות שקילות, אז $L \in REG$, ויש DFA עבור L עם k מצבים.

רעיון: נסמן את קבוצת מחלקות השקילות ביחס $\sim_L$ ב- Q . כמו כן, $\delta^*(q_0, w) = [w]_L$.

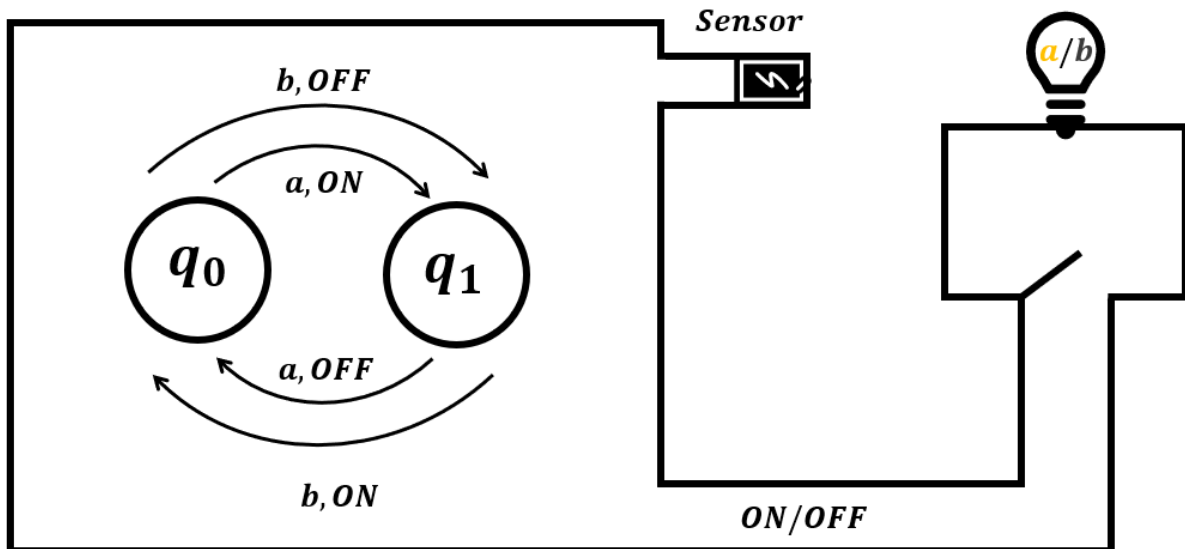
בניית A : $A = \langle \Sigma, Q, [\epsilon]_L, F, \delta \rangle$, כאשר:

$$F = \{[w]_L \mid w \in L\}$$

$$\delta([w]_L, \alpha) = [w\alpha]_L$$

יש לוודא גם שאם $[w]_L = [y]_L$ אז גם $[w\alpha]_L = [y\alpha]_L$. נראה זאת:

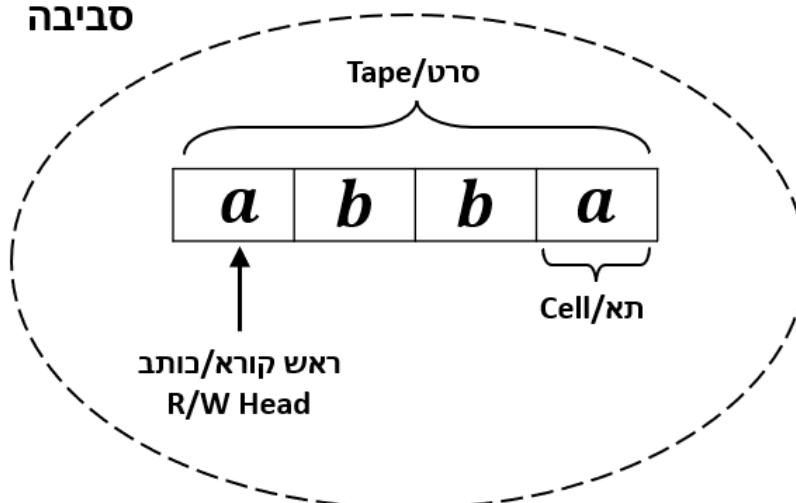
$$[w]_L = [y]_L \Rightarrow w \sim_L y \Rightarrow \forall z \quad wz \in L \iff yz \in L \xrightarrow{z=\alpha z'} \forall z' \quad w\alpha z' \in L \iff y\alpha z' \in L \Rightarrow w\alpha \sim_L y\alpha \Rightarrow [w\alpha]_L = [y\alpha]_L$$



ב- q_0 אם נראה שהנורה דולקת נשאיר אותה דולקת. אם היא כבויה תישאר כבויה.
בפועל, הנורה תדלק ותכבה לסירוגין.

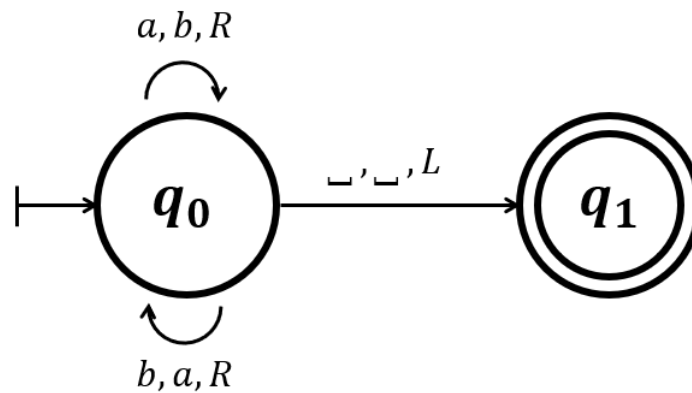
[היברות – מכונת טיורינג](#)

סביבה

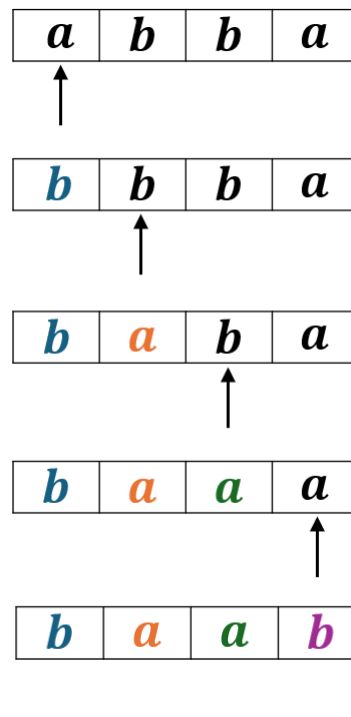


ראש קורא/כותב – מתחיל באות השמאלית ביותר של הקלט. שולח לאוטומט את האות שנמצאת מולו. על כל חץ האוטומט מוציא 2 הוראות: אות וכיוון.

דוגמה:

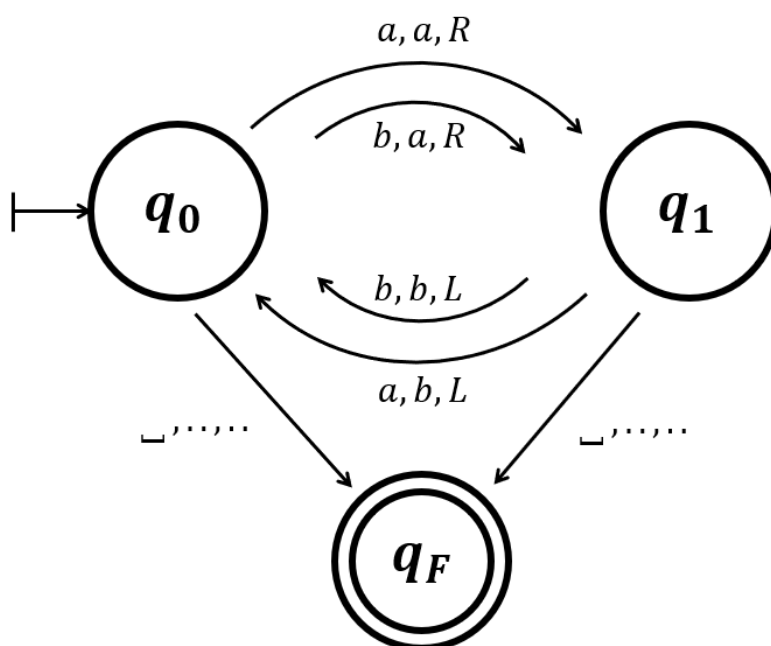


עבור הקלט $abba$ נקבל את הריצה הבאה:



בשמכונת טיורינג מגיעה אל מחוץ לסרט, הראש הקורא/כותב ממשיך אל מחוץ לה וכותב $_$ (רווח). משם נעבור ל- q_1 מצב סופי. אם למשל את החץ שיוצא ל- q_1 נשנה לחץ פנימי חזרה ל- q_0 , המכונה תחליף כל a ל- b וכל b ל- a , אבל הסרט לא יפסיק לגדול והמכונה תמשיך לכתוב רווחים ימינה.

דוגמה ל-TM שלא כותבת סרט שהולך וגדל, אבל גם לא עוצרת:



הגדרות

קונפיגורציה מה שכתוב על הסרט, מצב האוטומט, ומיקום הראש הכותב/קורא. לקונפיגורציה שבה הראש מכון על האות הראשונה, המצב הוא המצב ההתחלתי ובסרט יש את הקלט נקרא קונפיגורציה התחלתית, ונסמנה C_0 . הקונפיגורציה שתגיע אחרי C_0 תיקרא הקונפיגורציה העוקבת ל- C_0 . נסמנה ב- C_1 . לקונפיגורציה שבה המצב הוא מצב סופי נקרא קונפיגורציה סופית, ואין לה קונפיגורציה עוקבת. נסמן קונפיגורציה על מילה $\alpha_1 \alpha_2 \dots \alpha_i \dots \alpha_k$ כאשר הראש מכון על α_i והמ"ט במצב q כך: $\alpha_1 \dots \alpha_{i-1} q \alpha_i \dots \alpha_k$.

ריצה חלקית של מ"ט T על קלט w היא סדרה (סופית או אינסופית) של קונפיגורציות $C_0, C_1, C_2, \dots$ כאשר C_0 היא הקונפיגורציה ההתחלתית בריצת T על w , ולכל $i > 0$ נקבל כי C_i היא הקונפיגורציה העוקבת של C_{i-1} .

ריצה מלאה היא כאשר הריצה סופית ומסתיימת בקונפיגורציה סופית, או אם היא אינסופית.

זמן ריצה (של מ"ט T על קלט w) הוא אורך הריצה המלאה שלה על הקלט w , פחות 1. עבור ריצה אינסופית – אינסוף.

קלט – הסרט ההתחלתי.

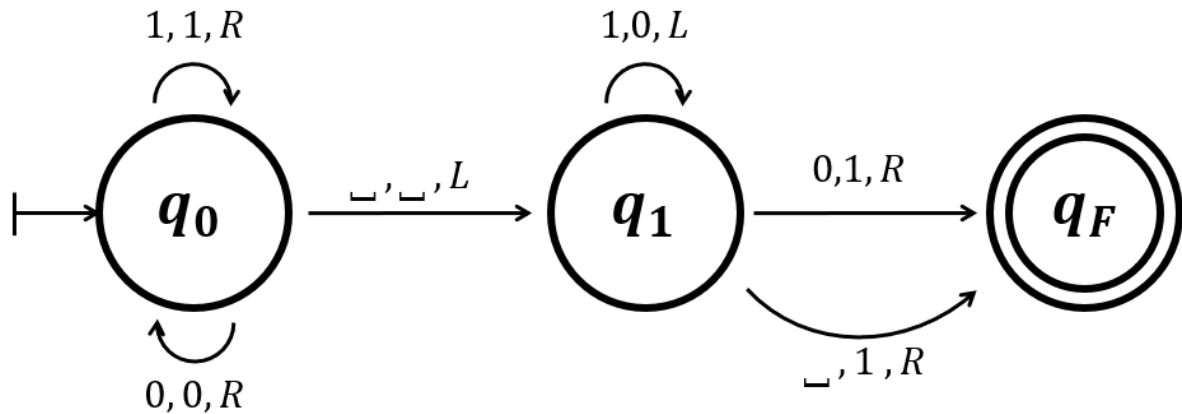
פלט – תוכן הסרט בסוף הריצה המלאה (אם היא סופית). עבור ריצה אינסופית אין פלט.

בעיה לדוגמה – מספר עוקב

קלט: $x \in \{0,1\}^*$. פלט: $x + 1$.

אלגוריתם:

- (1) נחפש את הקצה הימני של הקלט.
 - (2) נהפוך כל 1 שנתקל בו ל-0, עד שנמצא 0, נהפוך אותו ל-1 ונסיים.
- זמן ריצה: $2n + O(1)$, במקרה הגרוע ביותר בו לא נתקלנו ב-0 בכל הדרך.



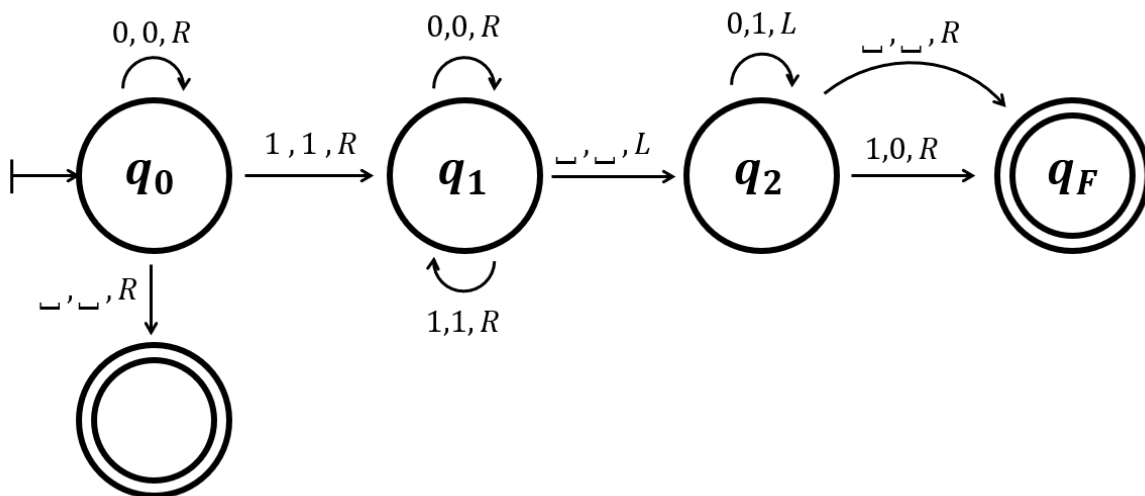
בעיה לדוגמה – מספר קודם

קלט: $x \in \{0,1\}^*$. פלט: $x - 1$ אם $x > 0$, אחרת 0.

אלגוריתם:

בגדול, הרעיון הוא לעבור על המספר מימין לשמאל, אפסים לשנות ל-1, ואת ה-1 הראשון שניתקל בו נהפוך ל-0. מזכיר את האוטומט הקודם פרט למקרה של $x \leq 0$.

- (1) נחפש את הקצה הימני של הקלט, אם הקלט הוא 0 נעצור.
- (2) כמו קודם החל מהמצב q_2 בתרשים הבא:



הגדרה פורמלית מכונת טיורינג – מ"ט היא שביעייה $\langle \Sigma, \Gamma, \sqsubset, Q, q_0, F, \delta \rangle$ כאשר:

Σ – א"ב הקלט, קבוצה סופית לא ריקה.

Γ – א"ב העבודה, יקיים $\Sigma \subseteq \Gamma$ (סופי).

$\sqsubset$ – תו מיוחד, $\sqsubset \in \Gamma \setminus \Sigma$.

Q – קבוצת המצבים (סופית).

$q_0 \in Q$ – המצב ההתחלתי.

$F \subseteq Q$ – קבוצת המצבים הסופיים. לרוב ניתקל ב- $F = \{q_F\}$ או ב- $F = \{q_{acc}, q_{rej}\}$.

δ – פונקציית המעברים: $\delta: \Gamma \times (Q \setminus F) \rightarrow Q \times \Gamma \times \{R, L\}$.

חיבור מספרים

קלט: $x, y \in \{0,1\}^*$, $x \# y$. פלט: $x + y$.

מימוש:

- (1) נלך ימינה עד תא אחד לפני התו $\#$ (נגיע לתחילת y).
- (2) נחסר אחד מ- y , אם $y = 0$ נעבור על האותיות של y מימין לשמאל ונהפוך אותן לרווחים. נעצור כאשר נגיע אל התו $\#$.
- (3) נלך שמאלה עד תא אחד שמאלה מ- $\#$. כרגע המצב הוא: $x \# (y - 1)$
- (4) נוסיף 1 ל- x .
- (5) נחזור לשלב 1.

זמן ריצה: אם n הוא האורך של x, y אז שלבים 1,2,3 לוקחים $O(n)$. חוזרים על כך y פעמים, ובמקרה הגרוע ביותר x הוא תו אחד, ולכן $|y| = n - 2$ ונקבל: $O(n) \cdot 2^{n-2}$.

העתקת מחרוזת

קלט: $x \in \{a, b\}^*$ פלט: $x \# x$.

איך נדע אילו אותיות כבר העתקנו? באמצעות קונספט חדש שנקרא **סימון**.

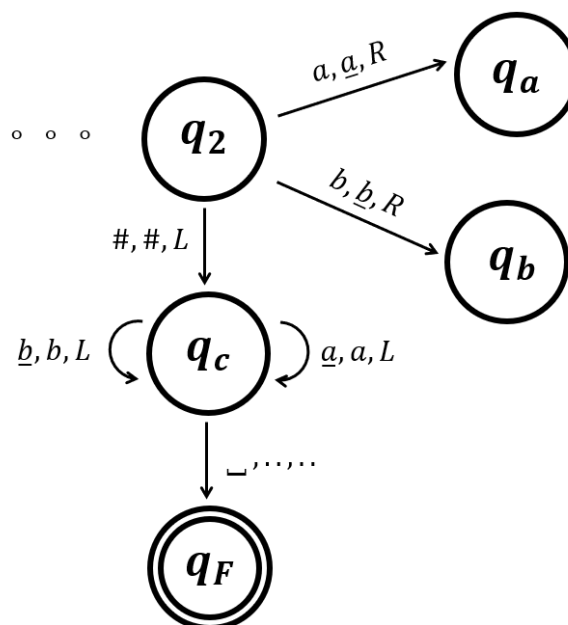
סימון – הגדרת א"ב עבודה חדש על ידי: $\dots \cup \{\sqsubset\} \cup \{\sqsupset, \bar{\sqsubset}\} \cup \Sigma$. $\Gamma = \Sigma \cup (\Sigma \times \{\sqsubset, \bar{\sqsubset}\}) \cup \{\sqsupset\}$.

פורמלית מדובר על הזוג הסדור $(a, \sqsubset)$ אבל אנחנו נסמן $\underline{a}$.

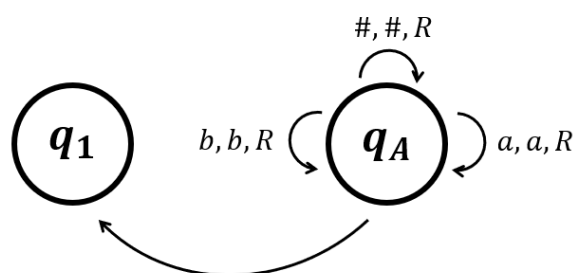
בדוגמה שלנו: $\Sigma = \{a, b\}, \Gamma = \Sigma \cup (\Sigma \times \{\sqsubset\}) \cup \{\sqsupset\}$.

האלגוריתם:

- (1) נסרוק ימינה עד למציאת רווח, ונחליף אותו ב-#.
- (2) נסרוק שמאלה עד רווח או עד אות מסומנת, ונצעד צעד נוסף ימינה.
- (3) מתואר בתרשים הבא:



- (4) עבור למשל המצב q_a , סריקה ימינה עד רווח ונחליף אותו ב- a . נחזור לסריקה שמאלה:



זמן ריצה: $O(n^2)$. בכל פעם שמעתיקים אות הולכים עד צד שמאל ואז עד ימין אז עבור כל אות $O(n)$.

כפל מספרים בינאריים

קלט: $x \# y$ פלט: $x \cdot y$ (כאשר $\cdot$ סימן הכפל בין מספרים בינאריים).

אלגוריתם:

- (1) הכפלת x פעם אחת, נקבל: $x \# x \# y$.
- (2) הכפלת x פעם נוספת, נקבל: $x \# x \# x \# y$ (לאורך הפתרון, נשאיר את x ההתחלתי כדי שנזכור אותו).
- (3) סכימת האיברים באמצע: $x \# x + x \# y$.
- (4) נוריד מ- y אחד, נקבל $x \# 2x \# y - 1$.
- (5) נחזור לשלב (2).

קריאה למ"ט כפּרוּצדורה/פונקציה

דוגמה להמחשה – ספירת צעדי חישוב

משימה: בהינתן מ"ט M רוצים לבנות מ"ט M' עם התכונה הבאה: אם M עוצרת על w , M' תעצור על w ויתקיים $M'(w) = M(w) \# t$ כאשר t הוא הייצוג הבינארי של מספר צעדי החישוב ש- M מבצעת על w . M עושה סימולציה של M אבל גם סופרת כמה צעדים). נניח ש- $\#$ לא חלק מא"ב העבודה של M .

הרעיון: נעשה צעד של M , נוסיף 1 למונה, נחזור ל- M ושוב למונה וכן הלאה.

נתחיל עם מכונת טיורינג S :

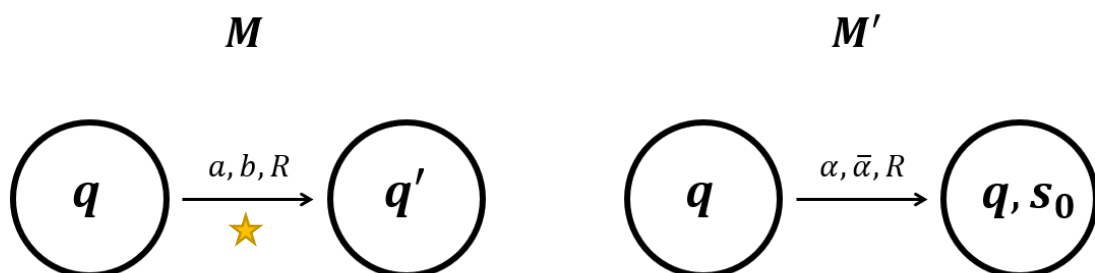
- (1) סורקת ימינה עד שמוצאת $\#$.
- (2) אם אין ספרה מימין ל- $\#$, S תכתוב 0.
- (3) נוסיף 1 למונה.
- (4) נסיים במצב S_f .

מכונת טיורינג M' :

- (1) תוסיף $\#$ מימין לקלט, ותחזור לתחילתו.
- (2) נעבור למצב ההתחלתי של M .
- (3) נסמלץ צעד חישוב של M .
- (4) "נקרא" ל- S כדי להוסיף 1 למונה.
- (5) נחזור חזרה למקום המקורי (בו היה נמצא הראש לפני שקראנו ל- S), ולמצב המקורי.
- (6) נחזור לשלב (3).

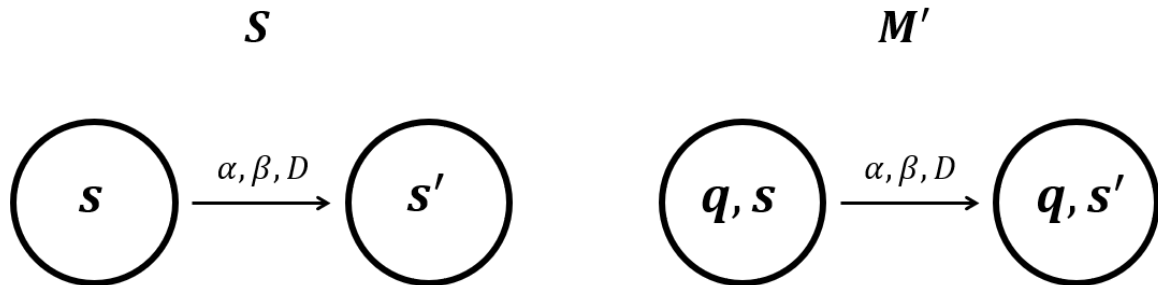
הערה: את העלאת המונה אפשר לעשות לפני M או אחרי M , לבחירתנו.

"קריאה" ל- S : לצורך ההסבר, נניח שאנחנו אחרי שלב 3 ואנחנו במצב q של המכונה M , ורוצים לעבור לשלב 4. אפשר לסמן באיזה מצב היינו בעזרת α^q , מרחיב מאוד את הא"ב. כאן נשתמש ב- $\bar{\alpha}$.



איך נזכור את המצב ממנו באנו? לא נוסף מ- q חץ ל- s_0 , אלא למצב שנגדיר (q, s_0) . נוסף מצב (q, s) לכל $q \in Q_M$ ו- $s \in Q_S$. ב- M' יהיו את כל המצבים ב- M אבל גם את כל הזוגות שהגענו, וכך מתבצעת ה"קריאה" ל- S .

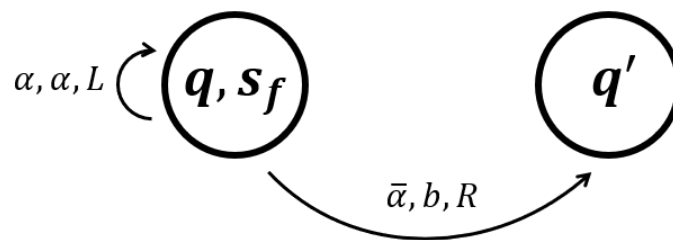
הביצוע של S :



בצורה זו S ממשיך לרוץ (ילך ימינה, יעלה את המונה באיזשהו שלב יסיים כאשר נגיע ל- (q, s_f)).

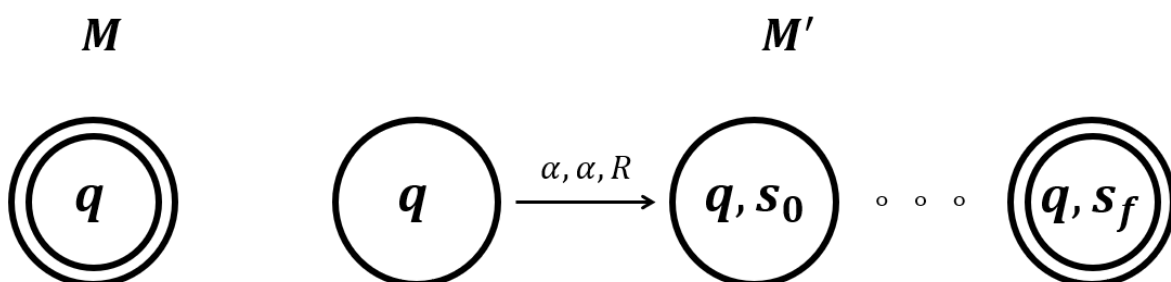
סיום S :

לכל α לא מסומנת:



בשנגיע לאות מסומנת, לא נחזור למצב המקורי (מסרבל). במקום זאת, נסמלץ את צעד החישוב הבא (מסומן בעמוד הקודם ב-★).

לכל חץ $q \xrightarrow{a,b,D} q'$ יהיה חץ מתאים $(q, s_f) \xrightarrow{\bar{a},b,D} q'$. אילו חצים יוצאים מ- (q, s_f) ? כל החיצים שיצאו מ- q (עבור אותיות מסומנות), ולולאות (עבור אותיות לא מסומנות).



עבור מצב q סופי, לא נסמן את α ובשנגיע ל- s_f , q נסיים.

נקודה למחשבה: כשמסמלים את M אנחנו יכולים להפריע לסימולציה שלו, כי הוא למשל יצפה לפגוש $\sqsubset$ ויתקל ב- $\#$ ששמנו כחלק מיצירת המונה, ועלולות להיות הפרעות נוספות. כדי לטפל בהן, נקרא לפרוצדורה מתאימה שתטפל בהפרעה.

מכונת טיורינג בקלט למכונת טיורינג

קידוד מכונת טיורינג בקלט

נרצה למצוא דרך להכניס מכונת טיורינג בקלט למכונת טיורינג.

בהינתן מ"ט $M = \langle \Sigma, \Gamma, \sqsubset, Q, q_0, F, \delta \rangle$ לא ניקח את M עצמה אלא מחרוזת שמתארת אותה. נתאר את M כמחרוזת מעל $\{0, 1, \#, |\}$. בה"כ נניח $\Sigma \subseteq \Gamma \subseteq \mathbb{N}$. כל אות ב- M ניתנת לייצוג כמספר טבעי. כמו כן, בה"כ נניח $Q \subseteq \mathbb{N}$ וגם $\Gamma \cap Q = \emptyset$ (מטעמי נוחות).

נבטא כל אספקט של M בצורה הבאה (תת אינדקס $(x)_b$ משמעותו הייצוג הבינארי של x):

$$\Sigma = \{\alpha_1, \dots, \alpha_k\} \rightarrow \langle \Sigma \rangle = (\alpha_1)_b \# \dots \# (\alpha_k)_b \#$$

$$\Gamma = \{\alpha_1, \dots, \alpha_k, \alpha_{k+1}, \dots, \alpha_n\} \rightarrow \langle \Gamma \rangle = (\alpha_1)_b \# \dots \# (\alpha_k)_b \# (\alpha_{k+1})_b \# \dots \# (\alpha_n)_b$$

$$\sqsubset = \alpha_i \rightarrow (\alpha_i)_b$$

$$Q = \{q_1, \dots, q_m\} \rightarrow \langle Q \rangle = (q_1)_b \# \dots \# (q_m)_b$$

ואת q_0, F באופן דומה.

עבור פונקציית המעברים אם מתקיים $\delta(q, \alpha) = (q', \beta, D)$ אז נגיד ש- $\delta(q, \alpha) \xrightarrow{\delta} (q', \beta, D)$ הוא כלל מעבר (ייצוג גרפי של המעבר). נייצג את δ ע"י שרשור כלל הייצוגים של כללי המעבר שהיא מגדירה. כל מעבר ייוצג:

$$\#\#(q)_b \# (\alpha)_b \# (q')_b \# (\beta)_b \# (D)_b \#\#$$

לבסוף, על מנת לייצג את כל $\langle M \rangle$: $\langle M \rangle = \langle \Sigma \rangle | \langle \Gamma \rangle | \langle \sqsubset \rangle | \dots | \langle \delta \rangle$

קידוד הקלט למכונת טיורינג

$$\langle w \rangle = (w_1)_b \# (w_2)_b \# \dots$$

קידוד קונפיגורציה

$$\langle C \rangle = (\alpha_1)_b \# (\alpha_2)_b \# \dots \# (\alpha_{i-1})_b \# \dots \# (\alpha_m)_b$$

לבסוף, כדי להעביר את המכונה M והמילה w כקלט למכונת טיורינג, נבצע:

$$\langle M, w \rangle = \langle M \rangle \langle w \rangle$$

הגדרה – מכונת טיורינג אוניברסלית: מ"ט U תיקרא אוניברסלית אם בהינתן $\langle M, w \rangle$:

- (1) אם M לא עוצרת בריצתה על w אז U גם לא תעצור.
- (2) אם M עוצרת בריצתה על w אז U תעצור, ויתקיים: $U(\langle M, w \rangle) = \langle M(w) \rangle$.
- (3) אם המצבים הסופיים של M הם $q_{rej} = 0, q_{acc} = 1$ אז U יסיים באותו מצב כמו M .

טענה

קיימת מ"ט אוניברסלית U .

רעיון הוכחה: U תפעל באופן הבא:

$$(0) \text{ קלט } \langle M, w \rangle$$

(1) תכתוב על הסרט

$$(2) \langle M \rangle \langle C \rangle \rightarrow \langle M \rangle \langle C^+ \rangle \text{ בלולאה עד קונפיגורציה סופית, אם קיימת.}$$

(3) אם נגיע לקונפיגורציה סופית:

(א) נמחק את $\langle M \rangle$

(ב) נמחק את $(q)_b$

(ג) אם $q \in \{0,1\}$ נסיים במצב q (נובע מתכונה 3 בהגדרה). אחרת נסיים ב- q_f כלשהו.

הערה: יש כאן הרבה בחירות והחלטות, כמעט אין שום דרך לעשות את זה בצורה שמגדילה מאוד את זמן הריצה. אם M רץ ב- $O(t)$ פעולות אז נגיע לזמן שהוא פולינומיאלי ב- t ולא אקספוננציאלי. ניתן להגיע אפילו ל- $O(t \log t)$ עם הרבה אופטימיזציות.

בעיית העצירה

האם M עוצרת על המילה w ?

הגדרה – $HALT$: שפה $HALT$ (בספרות לפעמים $HALT_{TM}$) תוגדר להיות:

$$HALT = \{ \langle M, w \rangle \mid M \text{ stops on } w \}$$

מכילה את כל המחרוזות שהן תיאור של מכונה עם w , אבל רק כאלו ש- M עוצרת על w .

הגדרה – מכונת החלטה: מ"ט M נקראת מכונת החלטה אם קבוצת המצבים הסופיים שלה מורכבת מהמצבים $\{q_{acc}, q_{rej}\}$.

השפה של מכונה כזו $L(M) = \{w \mid M \text{ stops at } q_{acc} \text{ when running on } w\}$.

הגדרה – M מקבלת את המילה w : אם M עוצרת במצב q_{acc} בריצתה על w .

הגדרה – M דוחה את המילה w : אם M עוצרת במצב q_{rej} בריצתה על w .

הגדרה – M מזהה את השפה L : נאמר שמ"ט M מזהה את השפה L כאשר $L(M) = L$.

הגדרה – M מכריעה את השפה L : נאמר שמ"ט M מכריעה את השפה L כאשר $L(M) = L$ וגם עוצרת על כל קלט.

הגדרה – $RE = \{L \mid \text{Exists a TM that recognizes } L\}$:

טענה

קיימת מ"ט שמזהה את $HALT$, כלומר $HALT \in RE$.

הוכחה: בעזרת קיום מ"ט אוניברסלית. נהפוך כל מצב סופי של המכונה האוניברסלית למצב סופי מקבל. (מכונה אוניברסלית מקבלת $\langle M, w \rangle$, אם M לא עוצרת על w האוניברסלית לא תעצור, ואם היא כן עוצרת אז גם האוניברסלית תעצור).

הגדרה – $R = \{L \mid \text{Exists a TM that decides } L\}$:

טענה

אין מ"ט שמכריעה את $HALT$, כלומר $HALT \notin R$.

הוכחה: נניח בשלילה שקיימת מ"ט D שמכריעה את $HALT$.

נבנה ממנה מ"ט E שעבור קלט $\langle M \rangle$ פועלת כדלהלן:

(1) E תכפיל את הקלט ונקבל: $\langle M, \langle M \rangle \rangle$.

(2) E תפעיל את D כפרוצדורה.

(3) אם D מחזירה "כן", E תכנס ללולאה אינסופית. אם D מחזירה "לא" אז E תקבל (בפרט, תעצור).

בעת, נריץ את E על הקלט $\langle E \rangle$. האם E עוצרת על קלט זה?

נניח **שכן**. אז D תקבע ש- E תעצור, ואז מהאופן בו הגדרנו את E היא תכנס ללולאה אינסופית – בסתירה.

נניח **שלא**. באופן דומה, D תקבע ש- E לא עוצרת, ואז E דווקא תעצור. שוב, סתירה.

שני המקרים הובילו לסתירה. מצב זה נובע מהנחת השלילה שקיימת מ"ט D שמכריעה את $HALT$.

הגדרה - $HALT_\epsilon$: שפה $HALT_\epsilon$ תוגדר להיות: $HALT_\epsilon = \{\langle M \rangle \mid M \text{ stops on } \epsilon\}$

טענה: $HALT_\epsilon \notin R$

הוכחה: ניתן לבנות מ"ט Red (מלשון רדוקציה) שפועלת באופן הבא על קלט $\langle M, w \rangle$:

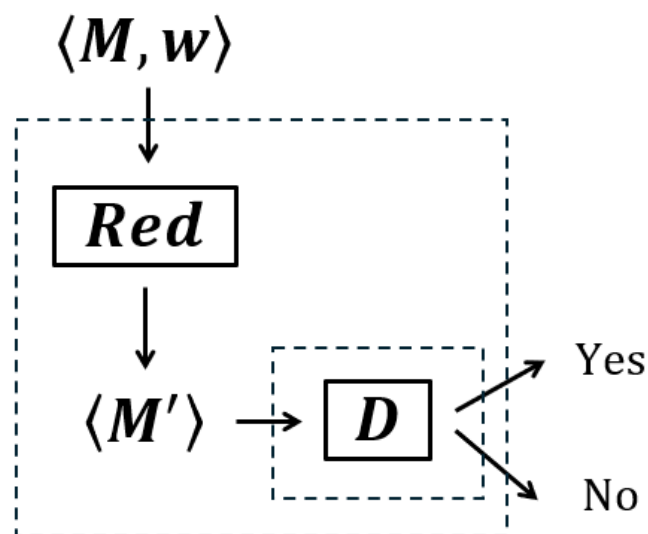
Red תייצר תיאור של מ"ט M' אשר בהינתן קלט ריק מדפיסה את w ואז רצה עליו כמו המכונה M :

$$M\langle M, w \rangle \xrightarrow{Red} \langle M' \rangle$$

נניח בשלילה ש- D מ"ט המכריעה את $HALT_\epsilon$. מכונה זו גם תכריע את השפה $HALT$ כי בעזרתה נוכל עבור

כל מ"ט M ומילה w לדעת האם המכונה תעצור. זה כמובן לא מתאפשר (הוכחנו לעיל $HALT \notin R$) ולכן

הגענו לסתירה.



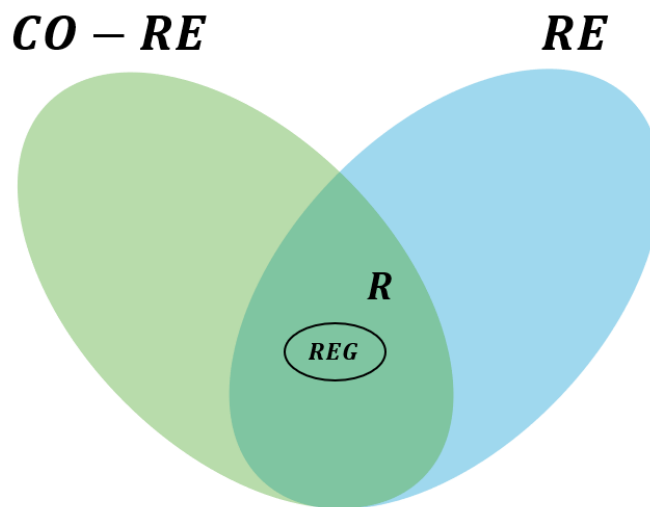
יחסים בין השפות RE, R

בהרצאה הקודמת הגדרנו את R ואת RE. היום לאורך השיעור נפתח את היחסים בין השפות ונעמיק את היחס.

הגדרה - $CO - RE$: נגדיר: $CO - RE = \{L \mid \bar{L} \in RE\}$ ואפשר להגדיר גם באופן שלא תלוי ב- $\bar{L}$ כך:

$$CO - RE = \{L \mid \text{exists TM } M \text{ that rejects input } x \Leftrightarrow x \notin L\}$$

במהלך ההרצאה נרחיב את הדיאגרמה הבאה:



טענה - $R \subseteq CO - RE$.

(1) האם קיים משהו ב- $RE \cap CO - RE$ חוץ מ- R ? לא. נוכיח כי $RE \cap CO - RE = R$.

צד אחד ברור, $R \subseteq RE \cap CO - RE$.

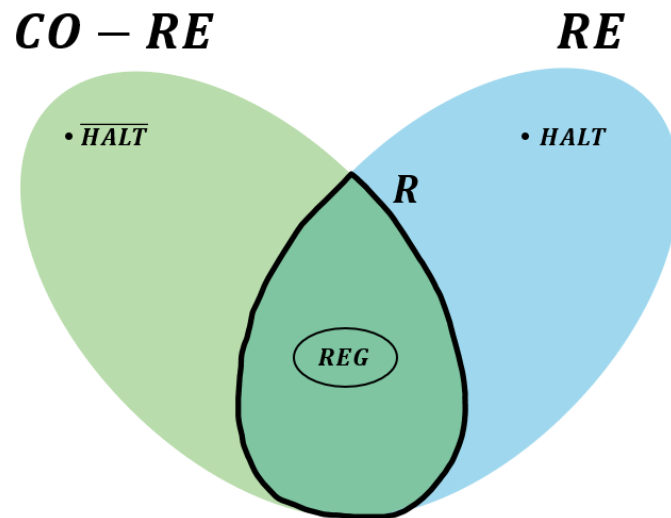
נראה צד שני: $R \supseteq RE \cap CO - RE$.

תהי $L \in RE \cap CO - RE$, אז יש מ"ט M_L ו- $M_{\bar{L}}$ שמזהות את L ואת $\bar{L}$ בהתאמה. ניתן לבנות מ"ט M_D שמכריעה את L ע"י שבהינתן קלט x המכונה M_D תפעיל לכל $i \in \mathbb{N}$ את M_L ואת $M_{\bar{L}}$ על x למשך i צעדים כל אחת. אם M_L תקבל/תדחה, אז M_D תקבל/תדחה בהתאמה. אם $M_{\bar{L}}$ תקבל/תדחה, אז M_D תדחה/תקבל בהתאמה. מובטח שלפחות אחד מהדברים יקרה לבסוף, ולכן ל- M_D תהיה תשובה.

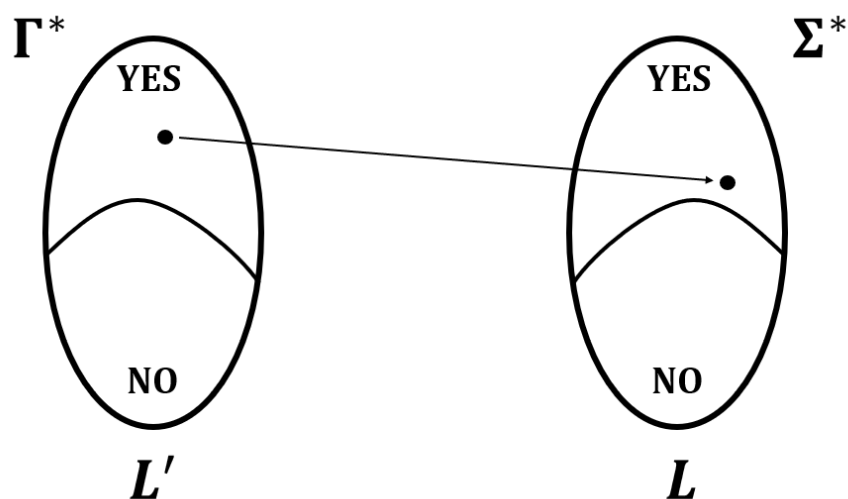
(2) האם יש שפה ב- $CO - RE$ אבל לא ב- R ? כן, $\overline{HALT}$. מהגדרתה, אם הייתה ב- RE אז הייתה גם ב- R .

ולכן היה אפשר להכריע אותה, ואנחנו יודעים שאי אפשר, לכן $\overline{HALT} \in CO - RE \setminus RE$.

נעדכן את הדיאגרמה לאור הטענה:



רדוקציה



נתעניין במ"ט Red , שלא פותרת לא את L ולא את L' , אלא מתרגמת שאלה מעל L' לשאלה מעל L , כשכל מה שידוע לה זה שלשתיהן יש אותה תשובה לשאלה.

הגדרה – מכונת רדוקציה: $L' \subseteq \Gamma^*, L \subseteq \Sigma^*$. מכונת רדוקציה מ- L ל- L' היא מ"ט Red שעוצרת על כל קלט

$$x \in \Gamma^*. \text{ ומחזירה מילה מעל } \Sigma \text{ כך שמתקיים } Red(x) \in L' \Leftrightarrow x \in L$$

אם קיימת מ"ט כנ"ל, נגיד שקיימת רדוקציה מ- L' ל- L . לפונקציה $Red(x) \mapsto x$ נקרא פונקציית רדוקציה מ- L' ל- L , ונסמן $L' \leq_m L$.

למה מסמנים $L \leq L'$ היא לפחות קשה מבחינת פתרון כמו L' , לכן אם היינו פותרים את L , נדע לפתור גם את L' .

משפט הרדוקציה - אם $L' \leq_m L$ אז:

(1) אם $L \in R$ אז $L' \in R$.

(2) אם $L \in RE$ אז $L' \in RE$.

(3) אם $L \in CO - RE$ אז $L' \in CO - RE$.

טענה - $HALT \leq_m A_{TM}$ כאשר $A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$.

רעיון הוכחה: את M' נשנה כך שרק כאשר M המקורית דחתה, M' תקבל.

הוכחה: הרדוקציה תחזיר בהינתן קלט $\langle M, w \rangle$ קידוד של $\langle M', w \rangle$ כאשר M' היא מ"ט שפועלת כמו M למעט ש- M' מקבלת גם כש- M דוחה.

נכונות: הרדוקציה המתוארת אכן ניתנת למימוש על ידי מ"ט שעוצרת תמיד.

$$\langle M', w \rangle \in A_{TM} \Leftrightarrow M' \text{ accepts } w \Leftrightarrow M \text{ stops } w \Leftrightarrow \langle M, w \rangle \in HALT$$

טענה - $A_{TM} \leq_m HALT$.

רעיון: המקרה הבעייתי הוא אם M דוחה את w . $\langle M, w \rangle \notin A_{TM}$, אבל $\langle M, w \rangle \in HALT$ וזה המקרה שבו צריך לטפל.

הוכחה: בהינתן $\langle M, w \rangle$ הרדוקציה תחזיר $\langle M', w \rangle$ כאשר M' היא מ"ט שפועלת כמו M למעט המקרה בו M דוחה, ואז M' תכנס ללולאה אינסופית.

נכונות: הרדוקציה היא חשיבה, ומתקיים:

$$\langle M, w \rangle \in A_{TM} \Leftrightarrow M \text{ accepts } w \Leftrightarrow M' \text{ stops (and accepts) } w \Leftrightarrow \langle M', w \rangle \in HALT$$

הגדרות - C-קשה, C-שלמה: אם C מחלקת שפות ו- $L \leq_m L'$ לכל $L' \in C$ אז נאמר ש- L היא C-קשה. אם בנוסף מתקיים ש- $L \in C$, נאמר ש- L היא C-שלמה.

טענה - A_{TM} היא RE-שלמה.

הוכחה: $A_{TM} \in RE$. נשאר להראות שאם $L \in RE$ אז $L \leq_m A_{TM}$. עבור L כזו, יש מ"ט M_L שמזהה אותה. בהינתן קלט x עבור L , הרדוקציה תחזיר $\langle M_L, x \rangle$ וקיבלנו $\langle M_L, x \rangle \in A_{TM} \Leftrightarrow x \in L$.

טענה - השפה $\overline{HALT}$ היא $CO - RE$ -שלמה.

הוכחה: תהא $L \in CO - RE$. אזי $\bar{L} \in RE$, לכן $\bar{L} \leq_m HALT$, ועם אותה רדוקציה נוכל לקבל גם $L \leq_m \overline{HALT}$.

הגדרה - $Non - HALT$: נגדיר $E \cup \{ \langle M, w \rangle \mid M \text{ doesn't stop on } w \} = \overline{HALT}$ כאשר E קבוצת קלטים לא חוקיים "Error". את החלק השני, נגדיר ב- $Non - HALT$.

טענה - $Non - HALT$ היא שלמה ב- $CO - RE$.

הוכחה: נראה כי:

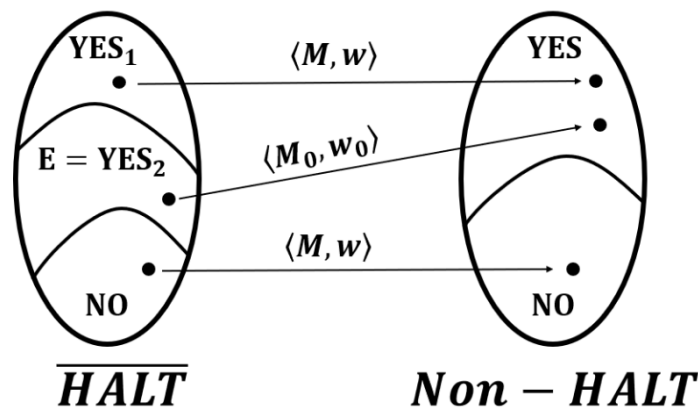
(1) $Non - HALT \in CO - RE$. נוכיח זאת באמצעות כך שנראה $\overline{Non - HALT} \in RE$

$$\overline{Non - HALT} = HALT \cup E$$

הראינו $HALT \in RE$, ומתקיים $E \in R$ (כי קלטים לא חוקיים ניתנים להכרעה האם מתקבלים או לא), ולכן גם $E \in RE$.

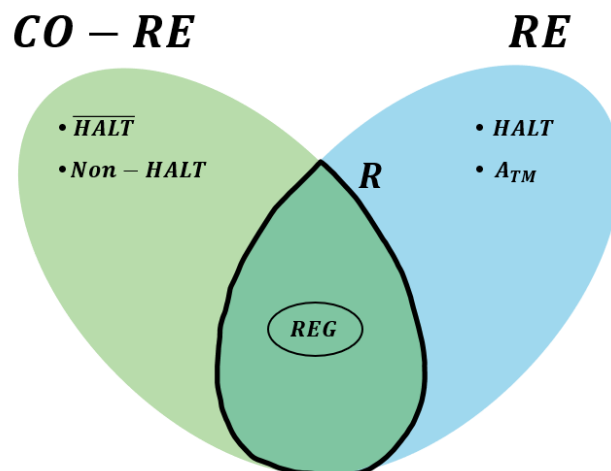
מסגירות RE לאיחוד נקבל $\overline{Non - HALT} \in RE$ ולכן $Non - HALT \in CO - RE$.

(2) נסיים בדקוניה מ- $\overline{HALT}$ ל- $Non - HALT$.



במיפוי מ- YES ל- YES או מ- NO ל- NO , נשמור על הערך כפי שהוא. ב- YES נחזיר $\langle M, w \rangle$ כך ש- M לא עוצר על w , וב- NO נחזיר $\langle M, w \rangle$ כך ש- M עוצר על w . הבעיה נוצרת במעבר מקלט x כלשהו שמקודד לערך לא מוגדר. מאחר ש- x בן ב- $\overline{HALT}$ אנחנו צריכים להחזיר משהו שישמור על ערך האמת, כלומר משהו שכן ב- $Non - HALT$. במקרה זה נחזיר $\langle M_0, w_0 \rangle$ שזו קידוד למכונה וקלט ידועים מראש כך שהמכונה תמיד לא עוצרת על הקלט שבחרנו.

נעדין את הדיאגרמה:



מה לגבי שפות "קשות יותר"? מכל שפה ב-RE וב-RE-CO? (כלומר שיש רדוקציה אליהן מכל שפה ב-RE וב-RE-CO). דוגמאות:

$$Y\text{-}HALT = \{ \langle M, w \rangle, Y \mid M \text{ halts on } w \}$$

$$N\text{-}HALT = \{ \langle M, w \rangle, N \mid M \text{ doesn't halt on } w \}$$

$$NY\text{-}HALT = Y\text{-}HALT \cup N\text{-}HALT$$

נשים לב שכדי לדעת אם מילה נמצאת ב-NY-HALT אנחנו נקבל $\langle M, w \rangle$ ונצטרך לדעת אם להוסיף אחרי הקידוד את N או את Y – ולשם כך צריך לדעת אם M עוצרת על w .

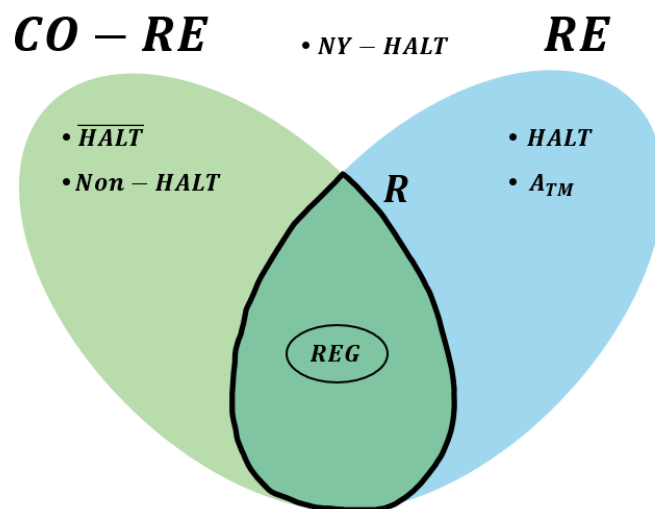
טענה - NY-HALT היא גם RE וגם CO-RE קשה.

הוכחה (RE): נראה רדוקציה מ-HALT. בהינתן $\langle M, w \rangle$ הרדוקציה תחזיר Y , $\langle M, w \rangle$.

הוכחה (CO-RE): רדוקציה מ-Non-HALT. בהינתן $\langle M, w \rangle$ הרדוקציה תחזיר N , $\langle M, w \rangle$.

טענה - NY-HALT $\notin$ RE.

הוכחה: אחרת, ממשפט הרדוקציה אנחנו נקבל $Non\text{-}HALT \in RE$. נתגלגל משם עד שנגיע לסתירה לדברים שראינו.



התזה של טיורינג וצ'רץ' - כל מכונת חישוב (ביקום שלנו) ניתנת לסימולציה על ידי מ"ט.

הגדרה - DECIDABLE: נגדיר $DECIDABLE = \{ \text{claims we can prove or contradict} \}$.

טיורינג טען $DECIDABLE \notin R$.

הוכחה: נניח בשלילה כי T מ"ט שמכריעה את $DECIDABLE$. נראה שזה יגרור $HALT \in R$:

נבנה מ"ט S שבהינתן $\langle M, w \rangle$, S תכתוב את הטענה " M לא עוצרת על w ", ואז תפעיל את T על הטענה:

(1) אם T ענתה "לא" (כלומר אי אפשר להוכיח או להפריך את הטענה) אז S תחזיר "לא".

(2) אם T ענתה "כן" (כלומר אפשר להוכיח או להפריך את הטענה) אז S תרוץ על כל ההוכחות האפשריות.

ללא תלות בתשובה של T נוכל להכריע האם M תעצור על S , כלומר מצאנו דרך להכריע את $HALT$ ולכן $HALT \in R$, בסתירה.

שבוע 7 – הרצאה

סיבוכיות

מה שניתן לחשב במשאבי חישוב מוגבלים. נדון רק בבעיות חישוביות כריעות, ונתעניין בעיקר כמה זמן/מקום על הסרט נדרש כדי להכריע.

בהרצאה זו מופיעות הגדרות רבות על זמן/מקום ריצה. בכל מקום שבו ניתן הגדרה עבור סיבוכיות הזמן, אפשר גם לתת הגדרה דומה עבור סיבוכיות מקום.

הגדרה – זמן ריצה: M מ"ט, w קלט, זמן הריצה של M על w הוא אורך הריצה המלאה של M על w , פחות 1 (עבור הקונפיגורציה ההתחלתית).

הגדרה – מקום ריצה: מספר התאים שהראש מגיע אליהם בזמן הריצה.

הגדרה – זמן הריצה של M : נתייחס למקרה הגרוע ביותר. אם M מכונת טיורינג, זמן הריצה שלה הוא פונקציה $T: \mathbb{N} \rightarrow \mathbb{N}$ המוגדרת על ידי:

$$T(n) = \max_{|w| \leq n} \{\text{running time of } M \text{ on } w\}$$

הגדרה – מקום הריצה של M : נתייחס למקרה הגרוע ביותר. אם M מכונת טיורינג, מקום הריצה שלה הוא פונקציה $S: \mathbb{N} \rightarrow \mathbb{N}$ המוגדרת על ידי:

$$S(n) = \max_{|w| \leq n} \{\text{running space of } M \text{ on } w\}$$

הגדרה – חסימת זמן ומקום הריצה: נגיד שמ"ט M רצה בזמן $O(f(n))$ אם $T(n) = O(f(n))$. אפשר להגדיר גם עבור $S(n)$ וכן עבור החסמים $\Theta, \Omega, \omega, o$. נניח ש- f היא מונוטונית תמיד.

הגדרה – $TIME(O(f(n)))$: תהא $f: \mathbb{N} \rightarrow \mathbb{N}$ מונוטונית עולה. נגדיר:

$$TIME(O(f(n))) = \{L \mid \exists \text{ TM that decides } L, \text{ and runs in } O(f(n)) \text{ time}\}$$

הגדרה – $SPACE(O(f(n)))$: תהא $f: \mathbb{N} \rightarrow \mathbb{N}$ מונוטונית עולה. נגדיר:

$$SPACE(O(f(n))) = \{L \mid \exists \text{ TM that decides } L, \text{ and runs in } O(f(n)) \text{ space}\}$$

נשים לב שהגדרות אלו מוגדרות רק עבור בעיות הכרעה, כי מניחים שקיימת TM מכריעה.

דוגמה 1 - INPUT-LENGTH

מתבצע ב- $O(n \log n)$ זמן. ראינו איך לעשות זאת בתרגילי הבית – אפשר למקם מונה משמאל וכל פעם להעלות אותו ב-1 (בזמן $\log n$) ואז להעתיק אותו ימינה ב-1 (בזמן $\log n$). יש n איטרציות כאלו ולכן הזמן הכולל הוא $O(n \log n)$. אי אפשר לבצע בפחות זמן, אך לא קל להראות את זה.

דוגמה 2 - $L \in \text{REG}$

זמן $O(n)$ וגם מקום $O(n)$.

דוגמה 3 - PALINDROM

שפת המילים שהן והיפוכן זהות. לזהות ייקח $\Theta(n^2)$ זמן, כי צריך n טיולים אחורה וקדימה בקלט, במקרה הגרוע טיול אורך n . $O(n)$ זמן. במ"ט דו-ראשית נוכל להוריד לזמן $\Theta(n)$.

הגדרה - POLYNOMIAL TIME: נגדיר P (לפעמים גם PTIME) להיות:

$$P = \bigcup_{k \in \mathbb{N}} \text{TIME}(O(n^k))$$

הגדרה - POLYNOMIAL SPACE: נגדיר PSPACE להיות:

$$\text{PSPACE} = \bigcup_{k \in \mathbb{N}} \text{SPACE}(O(n^k))$$

הערה: ההגדרה של P כמעט ואינה תלויה מודל. לפי תזת צ'רף-טיורינג המורחבת כל מחשב שניתן לבנות ביקום הפיזי שלנו, ורץ בזמן $O(f(n))$ ניתן לסמלץ במ"ט רגילה בזמן $O(f(n)^k)$ עבור k קבוע כלשהו. התזה גוררת ש- P אינו תלוי במודל החישובי.

הגדרה - E: נגדיר E להיות:

$$E = \bigcup_{k \in \mathbb{N}} \text{TIME}(O((2^k)^n))$$

הגדרה - EXP: נגדיר EXP להיות:

$$\text{EXP} = \bigcup_{k \in \mathbb{N}} \text{TIME}(O(2^{n^k}))$$

דוגמה 1 - CONNECTED

נגדיר: $\text{CONNECTED} = \{\langle G \rangle \mid G \text{ is a connected graph}\}$. בהינתן גרף, האם ניתן בזמן פולינומי להכריע האם הוא קשיר? כן. ראינו בקורסים קודמים באמצעות BFS/DFS . לכן, $\text{CONNECTED} \in P$.

דוגמה 2 - MIN-SPANNING-TREE

האם בעיה זו ב- P ? ללא שום שינוי, לא מדובר בבעיית הכרעה ולכן לא נוכל לדבר על P . נגדיר:

$$\text{MIN-SPANNING-TREE} = \{\langle G \rangle, \langle w \rangle, \langle k \rangle_b \mid \text{There is a spanning tree in } G \text{ in weight } \leq k\}$$

בעיה זו כן נמצאת ב- P .

נגדיר $SIMP-PATH = \{ \langle G \rangle, k \mid \exists \text{ simple path of length } k \text{ in graph } G \}$. נטען כי בעיה זו היא ב-E. הסיבה לכך היא שנעבור על כל $\binom{m}{k}$ יויות של קשתות, ונבדוק האם הן מסלול פשוט.

זמן הריצה: $\binom{m}{k} \leq \binom{n}{k}^{\text{binom}} \leq 2^n$

מכונת טיורינג לא דטרמיניסטית

תזכורת: במכונת טיורינג דטרמיניסטית ראינו לגבי פונקציית המעברים: $\delta(q, a) \in Q \times \Gamma \times \{R, L\}$. במכונת טיורינג לא דטרמיניסטית, δ תחזיר תת-קבוצה של $Q \times \Gamma \times \{R, L\}$ עבור כל הריצות האפשריות. הגדרות נוספות עבור מכונת טיורינג לא דטרמיניסטית (מ"ט ל"ד):

הגדרה – ריצה חלקית: סדרה של קונפיגורציות שבה הראשונה היא קונפיגורציה התחלתית (ניזכר שיתכנו כמה מצבים התחלתיים). כל קונפיגורציה שתגיע אחרי היא עוקבת של הקודמת (אחת מבין החוקיות). הגדרה – ריצה מלאה: ריצה חלקית אינסופית, או שמסתיימת בקונפיגורציה סופית.

הערה: אם הגענו למצב ממנו אין חצים יוצאים אבל זה לא מצב סופי – הריצה תקועה, ואינה ריצה מלאה.

הגדרה – מ"ט ל"ד מקבלת מילה w: אם קיימת ריצה מקבלת של המכונה על w.

הגדרה – מ"ט ל"ד מזהה שפה L: מ"ט ל"ד M מזהה את השפה L כאשר: $M \text{ accepts } w \Leftrightarrow w \in L$.

הגדרה - NP: נגדיר את NP להיות:

$$NP = \bigcup_{k \in \mathbb{N}} \text{NTIME}(O(n^k))$$

באופן דומה להגדרות שראינו לעיל, נוכל להגדיר:

$$\text{NPSPACE} = \bigcup_{k \in \mathbb{N}} \text{NSPACE}(O(n^k))$$

$$\text{NE} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(O((2^k)^n))$$

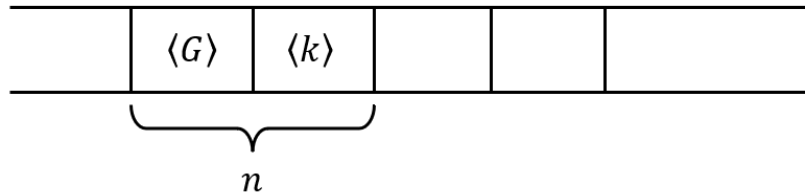
$$\text{NEXP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(O(2^{n^k}))$$

מה אנחנו יודעים להגיד על היחס בין P ו-NP? $NP \supseteq P$, כי כל מכונה דטרמיניסטית אפשר להסתכל עליה גם כלא דטרמיניסטית.

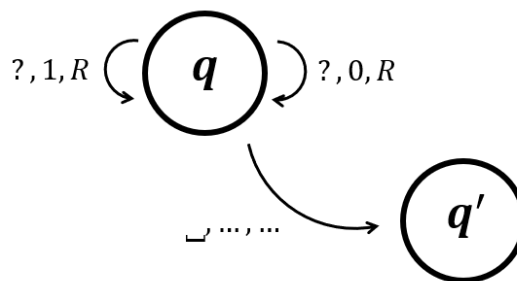
דוגמה – בעיית HAMILTON-PATH

מוגדרת להיות: $\text{HAMILTON-PATH} = \{\langle G \rangle \mid \exists \text{ a path of length } |V(G)| - 1\}$

בבעיית SIMP-PATH נניח שהקלט על הסרט נראה כך:



בהנחה שהקלט הוא בגודל n , אז לא ייתכן שכדי לייצג רק את הקשתות נצטרך יותר מ- n מקום, לכן נניח n . אם נרצה לתאר קשת אחת נצטרך $\log(n)$. כדי לתאר k קשתות צריך $k \log n$. המכונה תספור את אורך הקלט n , תחשב את $\log n$ ואז תכתוב $k \log n$ סימני שאלה, תחזור ל- "?" הראשון ותעבור למצב מנחש:



יש $2^{k \log n}$ ריצות אפשריות. נבדוק האם המחרוזת שכתבנו היא במקרה תיאור של מסלול העונה על הדרישה. זמן ריצה: בכל ריצה נכתוב $k \log n$ פעמים ? ונחשב את n ואת $k \log n$ ולבדוק האם זה מסלול בגרף. הכל מתבצע בזמן פולינומי. אם קיים מסלול כזה, זה יהיה באחת מהריצות ונחזיר "כן", אחרת נחזיר "לא". לכן אנחנו מסיקים $\text{SIMP-PATH} \in \text{NP}$.

בעיית CNF-SAT (Conjunctive Normal Form)

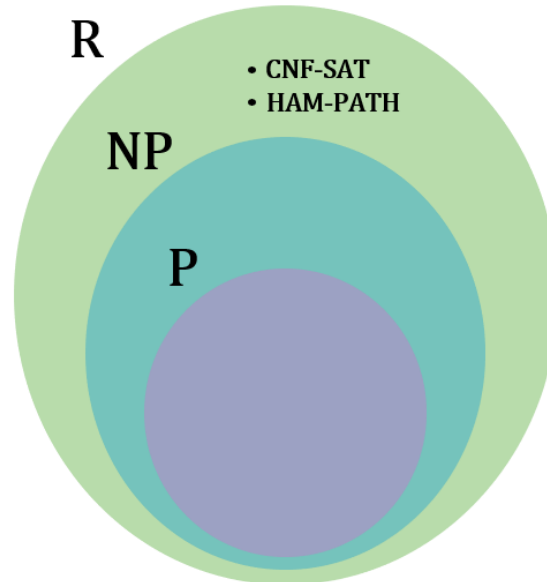
קלט: נוסחה בוליאנית מעל משתנים בוליאנים $x_1, \dots, x_n$ מהצורה: $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m$. כל C_i הוא מהצורה: $C_i = x_1 \vee x_2 \vee \dots \vee x_k$ כלומר כל C_i הוא מהצורה:

$$C_i(x_1 \vee x_2 \vee \overline{x_4} \vee x_7)$$

משתנה או שלילתו נקראים ליטרל, ולכן C_i הוא "או" על ליטרלים. קלט כזה הוא בשפה אם יש השמה למשתנים שמספקת את φ .

טענה: $\text{CNF-SAT} \in \text{E}$. לעבור על כל השמה של המשתנים הבוליאנים ולבדוק אם יש כזו שמספקת את φ . יש 2^n השמות – לכל אחת נעבור על הפסוק. כמה זמן לוקח לבדוק האם השמה מספקת? לא מאוד מעניין, זמן פולינומי.

האם $CNF-SAT \in NP$? כן, כמו בדוגמת SIMP-PATH נכתוב n סימני שאלה ואז נמלא 0 או 1 בכל תא, ונבדוק האם רצף זה הוא השמה מספקת.



רדוקציה פולינומיאלית

הגדרה – רדוקציה בזמן פולינומיאלי: נגיד על מכונת רדוקציה מ- L' ל- L שהיא רדוקציה בזמן פולינומיאלי מ- L' ל- L כאשר היא רצה בזמן שחסום על ידי פולינום באורך הקלט שלה. אם קיימת כזו, נסמן $L' \leq_p L$.

משפט – רדוקציה פולינומיאלית:

אם $L \in P$ ו- $L' \leq_p L$, אז $L' \in P$.

לא נוכיח, אבל ניתן אינטואיציה:

$$x \in L' \xRightarrow[n^{k_1}]{Red} \underbrace{Red(x) \in L}_{n^{k_1} \text{ באורך לכל היותר}} \Rightarrow \underbrace{M \circ Red(x)}_{n^{k_1 k_2}}$$

כי הרדוקציה רצה בזמן n^k אז לא ייתכן שכתבה משהו ארוך יותר

עדיין בזמן פולינומיאלי, ולכן אם אפשר לפתור את L בזמן פולינומיאלי אפשר לפתור גם את L' בזמן פולינומיאלי.

הגדרה – שפה C -קשה: תהי C מחלקת שפות. L תיקרא C -קשה ביחס לרדוקציה פולינומית אם $L' \leq_p L$.
לכל $L' \in C$ היא תיקרא C -שלמה אם היא C -קשה וגם $L \in C$.

אם נצליח למצוא לפתור בעיה NP-שלמה בזמן פולינומי, זה יכריע את **שאלת מיליון הדולר**, ונוכיח $P = NP$.

תזכורת



טענה

אם $A \leq_p B$ וגם $B \leq_p C$ אז גם $A \leq_p C$.

הוכחה:

לפי $A \leq_p B$ קיימת פונקציה f ומ"ט דטרמיניסטית פול' M_f שמחשבת את f ומתקיים $w \in A \Leftrightarrow f(w) \in B$.
 באופן דומה לפי $B \leq_p C$ קיימת פונקציה g ומ"ט דטרמיניסטית פול' M_g שמחשבת את g ולפיה מתקיים $w \in B \Leftrightarrow f(w) \in C$.

בנייה – נבנה מכונה M_h שמחשבת את h רדוקציה מ- A ל- C , כך ש- M_h מ"ט דטרמיניסטית פול'. עבור קלט x המ"ט M_h תחשב את $y = f(x)$ כמו M_f ואז תחשב את $z = g(y)$ כמו M_g ותחזיר את z . צריך להראות נכונות וזמן ריצה פולינומיאלי.

$$x \in A \xLeftrightarrow{M_f} y = f(x) \in B \xLeftrightarrow{M_g} z = g(y) \in C$$

זמן ריצה –

(א) הפעלת M_f – ידוע ש- M_f מ"ט פול' ולכן רצה בזמן $O(|x|^{k_1})$ עבור קבוע k_1 כלשהו.

(ב) הפעלת M_g – ידוע ש- M_g מ"ט פול' ולכן רצה בזמן $O(|y|^{k_2})$ עבור קבוע k_2 כלשהו.

הבעיה – זמן הריצה של שלב ב' מתואר כפונקציה של y שהוא משתנה פנימי, בעוד שהגדרנו זמן ריצה על גודל הקלט, שהוא x . צריך חסם כפונקציה של אורך הקלט x .

הפתרון – נשים לב ש- M_f יכולה ליצור פלט y מאורך לכל היותר כמספר צעדי הריצה שלה, לכן מתקיים כי $|y| = O(|x|^{k_1})$. כלומר, זמן ריצת שלב (ב) הוא $O(|x|^{k_1+k_2})$, אז זמן ריצת (א)+(ב) הוא $O(|x|^{k_1+k_2})$ עם k_1, k_2 קבועים.

קשיות ושלמות של מחלקות

$L \in \text{NP-Complete}$ אם $L \in \text{NP}$ וגם $L \in \text{NP-hard}$.

האם $P = \text{NP}$? אם הייתה לכך הוכחה/ הפרכה, מאיזה כיוון היא תגיע?

כיוון הוכחה: אם $L \in \text{NP-Complete}$ וגם $L \in P$ אז $P = \text{NP}$.

הסבר: אם $L \in \text{NPC}$ אז L היא NP-קשה, כלומר לכל $L' \in \text{NP}$ מתקיים $L' \leq_p L$ ואם בנוסף $L \in P$ לפי משפט הרדוקציה הפול" $L' \in P$ כלומר $\text{NP} \subseteq P$, ואת הכיוון השני כבר ראינו לכן $\text{NP} = P$.

כיוון הפרכה: אם $L \in \text{NP-Complete}$ וגם $L \notin P$ אז $P \neq \text{NP}$.

דוגמאות לשפות NP-קשות

3-SAT

נגדיר: $3\text{-SAT} = \{\varphi \mid \varphi \text{ נוסחת } 3\text{-CNF} \text{ וגם } \varphi \text{ ספיקה}\}$

תזכורת: נוסחה בוליאנית תיקרא CNF אם היא מהצורה הבאה – מורכבת מפסוקיות (הסגרים). בתוך כל פסוקית משתנים ושליטת משתנים ביניהם סימן $\vee$. בין הפסוקיות סימן $\wedge$.

נוסחה תיקרא 3-CNF אם בכל פסוקית יש בדיוק 3 ליטרלים (3 משתנים או שליטת משתנים). נוסחה φ תיקרא ספיקה אם יש הצבה מספקת למשתנים, כלומר הצבה של ערכי True/False למשתנים כך שערך האמת של φ הוא True.

טענה: $3\text{-SAT} \in \text{NP-Complete}$. צריך להראות (א) $3\text{-SAT} \in \text{NP}$ (ב) $3\text{-SAT} \in \text{NP-hard}$.

(א) הוכחה:

נראה ש- $3\text{-SAT} \in \text{NP}$. כלומר, נראה שקיימת מ"ט פול" ל"ד שמכריעה את 3-SAT. המכונה M :

- תוודא שהנוסחה מהצורה 3-CNF.
- תנחש הצבה ותוודא שההצבה מספקת את φ .

זמן ריצה פול" – ברור. נכונות – אם יש הצבה מספקת אז יש ריצה מקבלת, אם לא אז אין כזו.

(ב) שבוע הבא.

CLIQUE

נגדיר: $\text{CLIQUE} = \{\langle G, k \rangle \mid G \text{ is an undirected graph with a clique of size } k\}$

(כאשר קליקה היא קבוצת קודקודים שמחוברים בקשת כולם אלו לאלו).

טענה: $\text{CLIQUE} \in \text{NP-Complete}$. נראה: (א) $\text{CLIQUE} \in \text{NP}$ (ב) $\text{CLIQUE} \in \text{NP-hard}$.

(א) הוכחה:

נראה מ"ט ל"ד שמכריעה את CLIQUE . המכונה עבור קלט $\langle G, k \rangle$ תנחש באופן ל"ד תת-קבוצה S של צמתים ואז תוודא באופן דטרמיניסטי את הבאים:

(1) תספור כמה צמתים ב- S , אם לא שווה k תעצור ותחזיר q_{rej} .

(2) לכל זוג צמתים ב- S תבדוק האם יש ביניהם קשת.

אם נמצא זוג ב- S ללא קשת – תעצור ותחזיר q_{rej} . אם יש קשת בין כל זוג צמתים ב- S תחזיר q_{acc} .

נכונות:

אם $\langle G, k \rangle \in \text{CLIQUE}$ אז קיימת תת-קבוצה S של צמתי G כך ש- $|S| = k$ וגם בין כל זוג צמתים ב- S יש קשת. ולכן אם המכונה M תנחש את S אז תסיים במצב q_{acc} .

אם $\langle G, k \rangle \notin \text{CLIQUE}$ אז לכל ניחוש S , או שאינו בגודל k או שיש בו צמתים ללא קשת ולכן בכל מקרה M תחזיר q_{rej} .

זמן ריצה:

ניחוש S : לינארי באורך הקלט.

בדיקת קשתות: $O(k^2)$ זוגות ולכל זוג $O(n)$. סה"כ $O(n^3)$ ולכן זמן ריצה פולי.

(ב) הוכחה:

נרצה להראות ש- $\text{CLIQUE} \in \text{NP-hard}$. כלומר לכל $L' \in \text{NP}$ יתקיים $L' \leq_p \text{CLIQUE}$. נראה "רק" רדוקציה $\text{CLIQUE} \leq_p 3\text{-SAT}$, ואז כיוון שהוכחנו כבר $3\text{-SAT} \in \text{NP-hard}$ מתקיים לכל $L' \in \text{NP}$ ש- $L' \leq_p 3\text{-SAT}$ הראינו שרדוקציות פולי סגורות להרכבה. לכן, נסיק שלכל $L' \in \text{NP}$ מתקיים $L' \leq_p \text{CLIQUE}$ כלומר $\text{CLIQUE} \in \text{NP-hard}$.

צ"ל: $\text{CLIQUE} \leq_p 3\text{-SAT}$, כלומר שקיימת פונקציה f וקיימת מ"ט M_f כך ש- M_f מחשבת את f בזמן פולי וגם f פונקציית רדוקציה. כלומר:

$$\varphi \in 3\text{-SAT} \Leftrightarrow f(\varphi) = \langle G, k \rangle \in \text{CLIQUE}$$

ב- G יש קליקה בגודל $k \Leftrightarrow \varphi$ היא 3-CNF ספיקה

בנייה:

בהינתן נוסחה φ המכונה M_f תפעל באופן הבא:

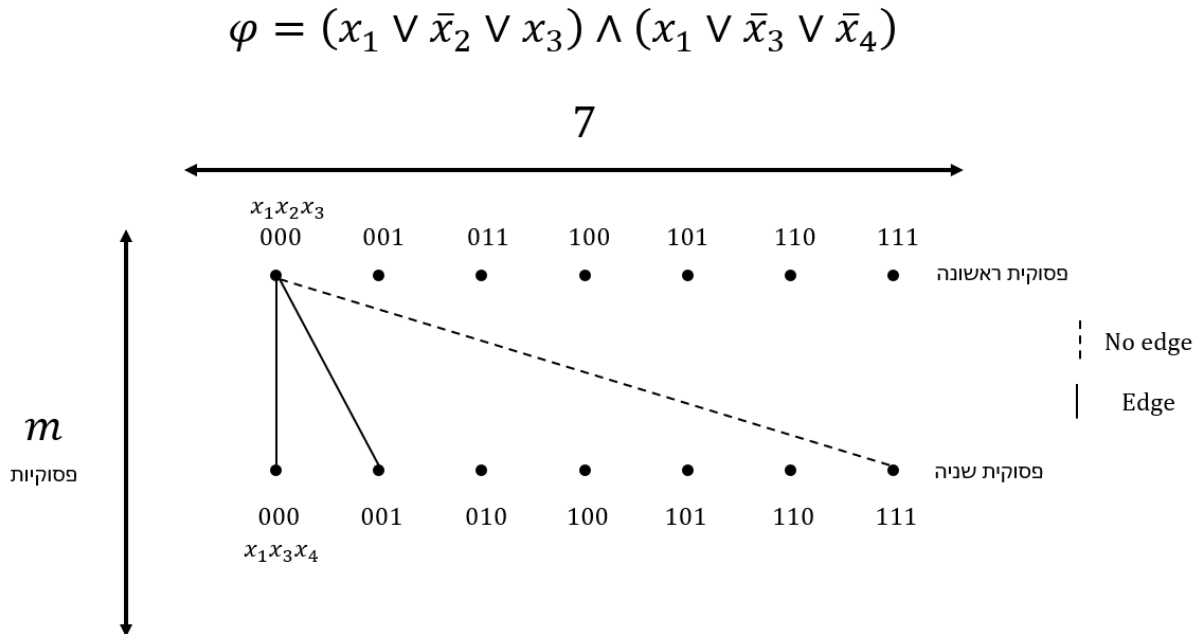
(1) תבדוק האם φ מצורת 3-CNF (קל), אם לא, תחזיר $\langle G, k \rangle \notin \text{CLIQUE}$ (כמו למשל k שגדול ממספר הצמתים בגרף ולכן כמובן אין קליקה בגודל k).

(2) בה"כ φ מצורת 3-CNF. M_f תפעל באופן הבא:

- לכל פסוקית ב- φ תגדיר 7 צמתים, אחד לכל הצבה מספקת של הפסוקית.
- בין כל זוג צמתים M_f תחבר קשת אלא אם הם מתייחסים לאותו משתנה ולא מסכימים בהצבה עליו.
- k יהיה מספר הפסוקיות ב- φ .

הערה: בתוך שכבה לעולם לא תהיה קשת, כי מדובר באופציות שונות לאותה ההשמה.

דוגמה:



בתוך שכבה לעולם לא תהיה קשת, כי מדובר באופציות שונות לאותה ההשמה.

זמן ריצה:

יצירת הצמתים: זמן פולינומיאלי ב- φ (יש $7m$ צמתים, $m < |\varphi|$)

חיבור קשתות: צריך לעבור על כל זוגות הצמתים $O((7m)^2)$ כלומר $O(|\varphi|^2)$ $O(m^2)$.

לכל זוג לבדוק: אם לא מתייחסים למשתנה משותף מחברים בקשת, אם כן מתייחסים למשתנה משותף ונותנים לו את אותו הערך – נחבר בקשת, אחרת לא. זה לוקח $O(|\varphi|)$ לכל זוג לכן סה"כ קיבלנו $O(|\varphi|^3)$.

נכונות:

בכיוון $\Leftarrow$ צ"ל שאם $\varphi \in 3\text{-SAT}$ אז $\langle G, k \rangle \in \text{CLIQUE}$. כלומר אם φ נוסחת 3-CNF ספיקה אז ב- G יש קליקה בגודל k . נסתכל על הצבה מספקת A עבור φ . לכל פסוקית, נבחר צומת (מבין השביעייה) שמתאים להצבה שמסכימה עם A . נשים לב ש- A הצבה מספקת, בפרט מספקת כל אחת מהפסוקיות ולכן מתאימה לבדיוק צומת אחת מכל שביעייה.

טענה: הקבוצה S של צמתים שבחרנו כך היא קליקה ב- G בגודל k .

הוכחה: גודל k – כי ב- S יש בדיוק צומת אחת בכל שכבה/ פסוקית. קליקה – נבהיר למה כל זוג צמתים ב- S מחוברים בקשת: נסתכל על זוג צמתים ב- S . אם הם לא מתייחסים למשתנה משותף לפי הבנייה בהכרח יש ביניהם קשת ואם הם כן מתייחסים למשתנים משותפים/ים, כיוון שההצבה לכל פסוקית נגזרה מהצבה כללית A יש הסכמה על ההצבה למשתנים משותפים, כלומר לפי הבנייה יש קשת בין זוג הצמתים.

הסבר נוסף לאינטואיציה: בכל שורה נעבור ונבחר את הקודקוד שמייצג את ההצבה הגלובלית שמספקת. לכן סה"כ נקבל k צמתים. למה מחוברות בקשת? אם אין משתנה משותף יש קשת, אבל אם יש משותף תהיה קשת כי כל הפסוקיות סופקו על ידי אותה ההצבה, אז למשל עבור x_7 בשניהם יש True.

בכיוון $\Rightarrow$ צ"ל שאם $\langle G, k \rangle \in \text{CLIQUE}$ אז $\varphi \in 3\text{-SAT}$. נסתכל על תת קבוצה S של הצמתים שהיא קליקה ב- G בגודל k .

נשים לב ש- S בהכרח מכילה בדיוק צומת אחד מכל שכבה. הסבר: לא יותר מאחד כי בתוך שכבה אין קשתות, ו- S היא קליקה. לא פחות מאחד כי גודל הקליקה כמספר השכבות.

נגדיר הצבה A בעזרת S . לכל פסוקית – הצומת ב- S מהשכבה המתאימה מגדיר הצבה למשתני הפסוקית. נשים לב שזוג צמתים שמתייחסים למשתנה משותף מסכימים על ההצבה עליו כי S קליקה, כלומר הם מחוברים בקשת לכן קיבלנו הצבה תקנית לכל המשתנים. בנוסף, כל צומת מתאים להצבה של אותה ההשמה, ובכיוון שב- S יש צומת מכל שכבה A מספקת את כל הפסוקיות כלומר מספק את φ כלומר $\varphi \in 3\text{-SAT}$.

מוודא פולינומי

נציג הגדרה חלופית ל-NP.

הגדרה – מוודא פולינומי: מ"ט דטרמיניסטית M היא מוודא פולינומי עבור שפה L אם M מקבלת זוג קלטים (w, c) רצה בזמן פולינומי ב- $|w|$, ומתקיים:

לכל $w \in L$ קיים c עבורו $M(w, c) = q_{acc}$

ולכל $w \notin L$ ולכל c מתקיים $M(w, c) = q_{rej}$.

הגדרה (חלופית) – NP' : נגדיר NP' להיות מחלקת כל השפות L שיש עבורן מוודא פולינומי.

טענה

$$NP = NP'$$

דוגמה:

נראה שיש מוודא פולינומי עבור השפה CLIQUE.

w יהיה זוג $\langle G, k \rangle$ המועמד לשייכות ל-CLIQUE.

c יהיה העד לשייכות, במקרה זה אמור לייצג קליקה ב- G בגודל k .

המ"ט הדטרמיניסטית M תבדוק האם c הוא קבוצה של k צמתים, וכן האם בין כל שניים מהם יש קשת. אם

כן – M תחזיר q_{acc} , ואחרת q_{rej} .

נכונות: אם $\langle G, k \rangle \in \text{CLIQUE}$ ההצבה של c להיות קליקה בגודל k תגרום ל- M להחזיר q_{acc} ואם $\langle G, k \rangle \notin \text{CLIQUE}$

אז כל הצבה של c , לא משנה מה תהיה, תסתיים ב- q_{rej} .

זמן ריצה: קל לראות שפולי- $|G, k|$.

שבוע 9 – הרצאה

הוכחת הטענה $NP = NP'$

הערה: בהרצאה הוכחה זו הוצגה בסוף השיעור, אך על מנת לשמור על הרצף עם סוף השיעור הקודם הקדמתי אותה לכאן.

נוכיח בעזרת הכלה דו כיוונית:

כיוון ראשון $NP \supseteq NP'$

נתונה שפה L ויש עבורה מוודא פולינומי דטרמיניסטי, כלומר M' בהתאם להגדרות שראינו לעיל. נרצה בעזרת M' לבנות מכונה M שהיא פולינומית לא דטרמיניסטית ומזהה את L . בה"כ האורך של העד c חסום על ידי פולינום ב- $|w|$. הסבר: M' רצה בזמן פול' ב- $|w|$ ולכן לא יכולה להספיק לקרוא יותר ממספר פולינומי של אותיות של c .

בנייה: M תפעל באופן הבא – תייצר מחרוזת c באורך פולינומי ב- $|w|$ באופן לא דטרמיניסטי, ואז תרוץ כמו M' על w, c .

נכונות:

- אם $w \in L$ אז יש c עבורו $M'(w, c) = q_{acc}$ ולכן יש ניחוש של M עבורו ערכי האותיות של c יגרום ל- M להחזיר q_{acc} .

- אם $w \notin L$ אז לכל c מתקיים $M'(w, c) = q_{rej}$ ולכן לכל c ש- M תייצר הריצה תסתיים במצב q_{rej} .

זמן ריצה: לייצר את c לוקח אורך של c , כלומר בה"כ חסום מלמעלה על ידי פולינום ב- $|w|$. בנוסף, השלב של הרצת $M'(w, c)$ ידוע שלוקח זמן פולינומי ב- $|w|$, סה"כ זמן ריצה פול' ב- $|w|$.

כיוון שני $NP \subseteq NP'$

נתונה שפה L ויש עבורה מ"ט פול' ל"ד M . נרצה בעזרתה לבנות מ"ט M' שהיא מוודא פולינומי עבור השפה L . נתבונן בפונקציית המעברים δ , מה הטווח שלה?

$$\delta: Q \times \Gamma \rightarrow P[Q \times \Gamma \times \{R, L\}]$$

מספר האפשרויות k לבחירה לא דטרמיניסטית בכל צעד וצעד הוא לכל היותר $2 \cdot |\Gamma| \cdot |Q|$. נסתכל על ריצה של $M(w)$. בכל צעד ריצה יש בחירה ל"ד של δ . זה מגדיר מחרוזת שבה כל אות היא מספר בין 1 ל- k . אורך המחרוזת הוא כמספר צעדי הריצה, כלומר פול' ב- $|w|$.

בנייה: אפשר להגדיר מ"ט M' דטרמיניסטית שמקבלת קלט w ומחרוזת כזו c ומדמה את ריצת $M(w)$ כאשר החלופה לצעד ל"ד זה צעד שנבחר על פי אות מתוך c . כלומר M' פועלת כמו M בכל צעד מדומה של M היא קוראת אות נוספת מתוך c ובעזרתה בוחרת איזה צעד של δ (של M) לבצע. נשים לב ש- M' דטרמיניסטית.

נכונות:

- אם $w \in L$ קיימת ריצה מקבל של $M(w)$ כלומר קיימת סדרה של בחירות ל"ד של δ שגורמות ל- M להחזיר q_{acc} . לכן, אם c תהיה המחרוזת שמתארת את סדרת הבחירות האלה אז $M'(w, c)$ תחזיר q_{acc} .
 - אם $w \notin L$ אז כל ריצה של $M(w)$ מחזירה q_{rej} , לכן כל מחרוזת c תוביל ל- $M'(w, c) = q_{rej}$.
- זמן ריצה:** האורך של c הוא פולינומי ב- $|w|$. מספר צעדי הסימולציה של M' עושה הוא כמספר הצעדים של $M(w)$, כלומר פולינומי ב- $|w|$. סימולציה של כל צעד לוקחת לכל היותר כאורך הסרט, כלומר לכל היותר זמן פולינומי ב- $|w|$. סה"כ זמן ריצה פולינומי ב- $|w|$.

משפט קוק-ליין

ראינו בעבר $SAT \in NP$, $3-SAT \leq_p SAT$. לכן:

$$SAT \in NP\text{-Complete} \Rightarrow 3-SAT \in NP\text{-Complete}$$

צריך להוכיח שלכל $L \in NP$ אם $SAT \leq_p L$ אז $w \in L \Leftrightarrow \varphi \in SAT$. מה הבעיה? L לא ידועה. כל שידוע לנו זה שהיא נמצאת ב- NP , כלומר שקיימת מ"ט M פול" לא דטרמיניסטית שמזהה את L .

תזכורת – קונפיגורציה

קונפיגורציה היא תיאור של מצב כללי במהלך ריצה. פורמלית $C = uqv$ כאשר $q \in Q$, $u, v \in \Gamma^*$. הקונפיגורציה c מתארת מצב כללי בו המצב הפנימי של המכונה הוא q , תוכן הסרט הוא uv והראש על האות הראשונה של v .

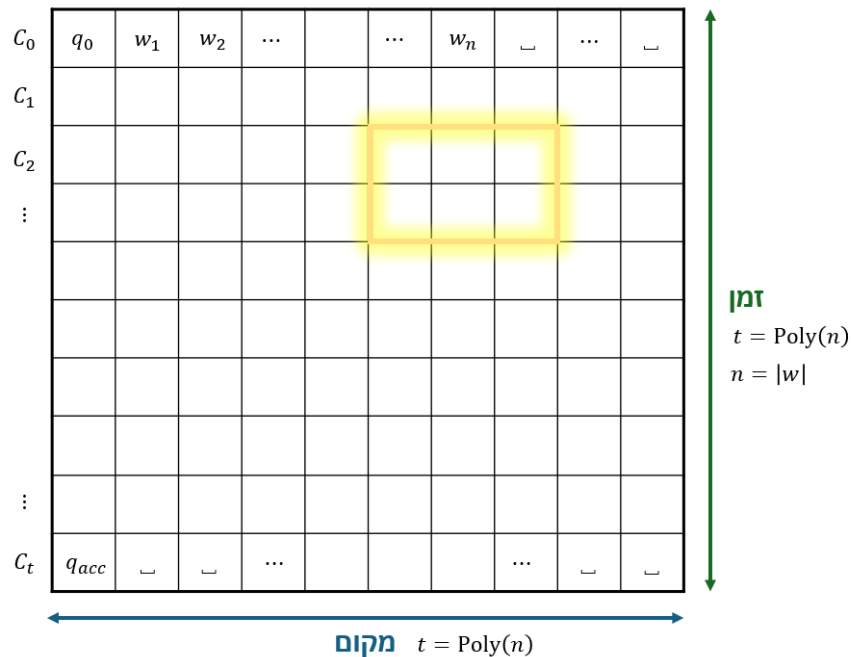
פירוט נוסף: הקונפיגורציה ההתחלתית בריצת $M(w)$ היא $C_0 = q_0w$ (לשים לב – היא יחידה). קונפיגורציה מקבלת היא קונפיגורציה שמכילה את המצב q_{acc} . קונפיגורציה דוחה היא קונפיגורציה שמכילה את המצב q_{rej} .

טענת עזר: בה"כ, למכונה M יש קונפיגורציה מקבלת יחידה.

הוכחה: נראה איך לשנות את M כך שתהיה לה קונפיגורציה מקבלת יחידה וזמן הריצה יישאר פולינומי. נשנה את מצב q_{acc} למצב q_{acc}^{almost} . במצב זה M מוחקת את כל הסרט, מחזירה את הראש לתחילת הסרט ועוברת למצב q_{acc} . ברור שעכשיו יש קונפיגורציה מקבלת יחידה, ונשים לב שזמן הריצה הוא בלכל היותר פולינום באורך הקלט כי על הסרט שהיה צריך למחוק יש לכל היותר מספר פולינומי בגודל הקלט של תווים.

באופן דומה, מוכיחים של- M בה"כ יש קונפיגורציה דוחה יחידה.

רעיון ההוכחה: נתאר ריצה של M על w על ידי טבלה. כל שורה בטבלה מתאימה לקונפיגורציה, החל משורה 0 בה יש את C_0 , (קונפ' התחלתית). ריצה תתאים למילוי חוקי של הטבלה, ואם השורה האחרונה היא הקונפיגורציה המקבלת אז המילוי מתאים לריצה מקבלת. נבנה נוסחה שבודקת האם המילוי בטבלה חוקי וכן מסתיים בקונפיגורציה מקבלת.



כל תא בטבלה מכיל אות מתוך $Q \cup \Gamma$. גובה הטבלה t לפי חסם על זמן הריצה. רוחב הטבלה לפי חסם על אורך הקונפיגורציה הארוכה ביותר, כלומר חסם על המקום ולכן גם חסום על ידי t . לחלק המסומן בצהוב בטבלה נקרא לאורך ההוכחה "שישייה".

מילוי הטבלה הוא חוקי אם:

- בשורה 0 מופיעה הקונפיגורציה ההתחלתית.
- בשורה t מופיעה קונפיגורציה מסיימת (מקבלת אם הריצה מקבלת).
- לכל $0 \leq i < t$ מתקיים C_i, C_{i+1} קונפיגורציות עוקבות, כלומר מתאימות לצעד ריצה אפשרי של M .
 C_i, C_{i+1} זהות כמעט בכל מקום למעט אולי במקום בו מסומן מיקום הראש של המכונה, ושם ההבדלים צריכים להתאים לפונקציה δ של M .

העקרון ברור, מה נותר להראות?

- איך ננסח את כל הבדיקות של השורה ה-0, השורה ה- t ושל כל השישיות?
- איך להמיר את כל ה"ל לנוסחאות CNF, בפרט מעל משתנים בוליאניים?
- איך לייצר את נוסחת ה-CNF ע"י מ"ט M' פול' דטרמיניסטית? (מכונת הרדוקציה).
- להוכיח נכונות וזמן ריצה.

(א) בניית בדיקות נכונות עבור מילוי של הטבלה

נקרא לתאים בטבלה $x_{i,j}$ כאשר $x_{0,0}$ הוא התא השמאלי עליון, $x_{t,t}$ התא הימני תחתון.

○ בדיקת שורה 0

$$\varphi_{init}(x_{0,0}, x_{0,1}, \dots, x_{0,t}) = (x_{0,0} = q_0) \wedge \bigwedge_{j=1}^n (x_{0,j} = w_j) \wedge \bigwedge_{j=n+1}^t (x_{0,j} = _)$$

נשים לב שאם בהמשך כל תנאי ייבדק על ידי נוסחת CNF אז φ_{init} כוללה תהיה נוסחת CNF.

○ בדיקת שורה t

$$\varphi_{acc}(x_{t,0}, \dots, x_{t,t}) = (x_{t,0} = q_{acc}) \wedge \bigwedge_{j=1}^t (x_{t,j} = _)$$

○ בדיקת שישיות

נרצה דרך לבדוק ששתי שורות עוקבות מתאימות לקונפיגורציות עוקבות.

אפשרויות למילוי חוקי של שישיה:

$$\forall a, b, c \in \Gamma$$

a	b	c
a	b	c

אילו מילויים חוקיים יש כאשר האזור עובר שינוי (הראש בסביבה)? עבור (q_2, b, R) $\exists (q_1, a)$

q_1	a	c	d
b	q_2	c	d

d	c	q_1	a
d	c	b	q_2

c	q_1	a
c	b	q_2

q_1	a	c
b	q_2	c

$$\forall c, d \in \Gamma$$

באופן דומה נסיף את כל הבדיקות עבור מילוי חוקי לשישייה כאשר פקודת התנועה היא שמאלה. נגדיר

בדיקה עבור שישיה:

$$\varphi_{legal}(x_{i,j}, x_{i,j+1}, x_{i,j+2}, x_{i+1,j}, x_{i+1,j+1}, x_{i+1,j+2})$$

בודקת האם המילוי חוקי לפי לפחות אחת מהאפשרויות שתיארנו קודם.

קיבלנו שהנוסחה היא:

$$\varphi = \varphi_{init} \wedge \varphi_{acc} \wedge \bigwedge_{i=0}^t \bigwedge_{j=0}^t \varphi_{legal}$$

(ב) המרה לנוסחאות CNF מעל משתנים בוליאניים

טסט φ_{legal} מחזיר T אם"ם ההצבה לששת המשתנים הרלוונטיים היא הצבה מותרת מתוך כלל ההצבות שהן $(\Gamma \cup Q)^6$.

נרצה להעביר הכל לעולם בוליאני, כלומר כל משתנה בטבלה $x_{i,j}$ יוחלף על ידי קבוצת משתנים בוליאניים.

כל בדיקה נרצה לנסח אותה: **(1)** כנוסחה על משתנים בוליאניים **(2)** בצורת CNF לכל משתנה מקורי $x_{i,j}$ נצטרך $\lceil \log_2(|\Gamma| + |Q|) \rceil$ משתנים בוליאניים. זה קבוע שתלוי במכונה – לא בשפה. נסמן ב- $\hat{\varphi}$ את הגרסה הבוליאנית של הנוסחאות מהשלב הקודם: $\varphi_{init}(x_{0,0}, \dots, x_{0,t})$. מאלצת ערך אחד ויחיד לכל משתנה שלה, ולכן גם $\hat{\varphi}_{init}$ עושה אותו דבר. $(x_{0,0}^1) \wedge (\overline{x_{0,0}^2}) \wedge \dots$ באופן דומה גם את $\hat{\varphi}_{acc}$ קל לתאר כנוסחת CNF.

נותר לטפל ב- φ_{legal} . תהיה עמודה לכל משתנה בוליאני שמקודד כל אחד מהשישייה המקורית. לכל הצבה לשישיית משתנים מקוריים יש הצבה מתאימה למשתנים הבוליאניים שמקודדים אותם. בכל שורה כזאת, נרצה שהנוסחה תחזיר T אם הערך לשישייה חוקי ו- F אחרת. כלומר, קיבלנו את טבלת האמת של הנוסחה $\hat{\varphi}_{legal}^{i,j}$ ונרצה למצוא ייצוג CNF שמסכים עם טבלת האמת הזו. אם לא הייתה דרישה לצורת CNF, פתרון אפשרי הוא נוסחת DNF, כלומר לכל שורה שמתאימה לערך (פלט) T , נשים פסוקית שבתוכה $\wedge$ בין משתנים, ומאלצת את ערכי האמת המתאימים לשורה (ההצבה) ובין הפסוקיות $\vee$.

קידוד בוליאני של כל אחד מבין $x_{i,j}$

	$x_{i,j}$	...	$x_{i+1,j+2}$	$x_{i,j}$					
$(\Gamma + Q)^6$				T	0	0	...	1	0
				F	1	0	...	1	1

כדי לייצר נוסחת CNF נייצר נוסחת DNF עבור כל השורות המסומנות ב- F , ואז שלילה של הנוסחה

המתקבלת (דה-מורגן) תיתן CNF עבור $\hat{\varphi}_{legal}^{i,j}$.

$$\hat{\varphi} = \hat{\varphi}_{init} \wedge \hat{\varphi}_{acc} \wedge \bigwedge_{i=0}^t \bigwedge_{j=0}^t \hat{\varphi}_{legal}^{i,j}$$

סיימנו להראות שקיימת נוסחת CNF שסימנו ב- $\hat{\varphi}$, והיא ספיקה אם"ם $w \in L$.

(ג) ייצור הנוסחה על ידי מ"ט M דטרמיניסטית פולינומיאלית

צריך להראות שקיימת רדוקציה $SAT \leq_p L$ כלומר שקיימת מ"ט M דטרמיניסטית פולינומיאלית שעבור קלט w מחזירה $\hat{\varphi}$ כך ש- $\varphi \in SAT \Leftrightarrow w \in L$, כלומר להראות איך מייצרים את $\hat{\varphi}$ מ- w ע"י מ"ט M דטרמיניסטית פולינומיאלית.

בנייה:

(1) M' תחשב את t (פולינום כלשהו על $n = |w|$), קל לבצע בזמן פולי.

(2) M' תייצר את φ'_{init} וגם את φ'_{acc} , קל לבצע בזמן פולי.

(3) M' תריץ לולאה כפולה (על i ועל j , מ-0 ועד t) ולכל i, j תדפיס עותק של $\hat{\varphi}_{legal}^{i,j}$. נשים לב שכל

הנוסחאות $\hat{\varphi}_{legal}^{i,j}$ הן אותו הדבר עד כדי שינוי שמות משתנים. כלומר, הנוסחה $\hat{\varphi}_{legal}$ יכולה להיות

חלק מקידוד המכונה M' , ו- M' רק צריכה להדפיס אותה שוב ושוב עם עדכון i, j ובין כל שני עותקים

סימן $\wedge$.

הערה: הפולינום שמשמש לצורך חישוב t מתוך $n = |w|$ קיים, ולכן יכול להיות מקודד בתוך M' .

(ד) הוכחת נכונות זמן ריצה

זמן ריצה: M' צריכה להדפיס את $\hat{\varphi}_{init}$, $\hat{\varphi}_{acc}$ כל אחת בזמן פולי. בנוסף, היא צריכה להדפיס את $\hat{\varphi}_{legal}^{i,j}$

לכל ערך i, j בין 0 ל- t פעמים, כלומר t^2 פעמים. סה"כ מספר פולי של פעמים, ולכן זמן הריצה כולו הוא פולינומיאלי.

נכונות:

- אם $w \in L$ אז קיימת ריצה מקבלת של $M(w)$ ולכן קיים מילוי חוקי לטבלה המקורית שבו שורה ראשונה היא C_0 (הקונפיגורציה ההתחלתית), שורה אחרונה היא קונפיגורציה מקבלת וכל 2 שורות עוקבות מתאימות לקונפיגורציה עוקבת בריצה – וכל הבדיקות יעברו. לכן, קיימת הצבה למשתנים בוליאנים שבה כל בדיקות ה-CNF עוברות, ולכן יש הצבה מספקת עבור φ .

- אם $w \notin L$ אז כל מילוי של הטבלה המקורית הוא או מתאים לריצה חוקית שמסתיימת בקונפיגורציה הדוחה, או אינו מתאים לריצה חוקית. בכל מקרה, לפחות בדיקה אחת מתוך $\varphi_{init}, \varphi_{acc}, \varphi_{legal}^{i,j}$ נכשלת, לכן לפחות נוסחת CNF אחת מבדיקות המשתנים הבוליאנים מחזירה F ולכן כל הצבה לא מספקת את $\hat{\varphi}$. מסקנה $\hat{\varphi} \notin SAT$.

משפט ההיררכיה

משפט ההיררכיה בזמן

לכל פונקציה f שחשיבה בזמן $O(f(n))$ מתקיים:

$$\text{Time}(o(f(n))) \subsetneq \text{Time}(f(n) \log n)$$

משפט ההיררכיה במקום

לכל פונקציה f שחשיבה במקום $O(f(n))$ מתקיים:

$$\text{Space}(o(f(n))) \subsetneq \text{Space}(f(n))$$

הרעיון: נחזור על הוכחת הלכסון שמראה $R \subsetneq RE$ ואז נעדין אותה עם מגבלה על זמן/מקום ריצה, ונקבל את משפט ההיררכיה.

תזכורת להוכחת הלכסון: $A_{TM} = \{\langle M, w \rangle \mid M(w) = q_{acc}\}$

נניח בשלילה שקיימת מ"ט דטרמיניסטית H שמכריעה את A_{TM} , כלומר:

$$H(\langle M, w \rangle) = q_{acc} \Leftrightarrow M(w) = q_{acc}$$

נגדיר:

$$D(\langle M \rangle) = H(\langle M \rangle, \langle M \rangle) \quad ; \quad \underbrace{\widehat{D}(\langle M \rangle) = \overline{D(\langle M \rangle)}}_{\text{swap } q_{acc} \text{ and } q_{rej}}$$

מה מחזירה $\widehat{D}(\langle \widehat{D} \rangle)$?

אם $\widehat{D}(\langle \widehat{D} \rangle) = q_{acc}$ אז $D(\langle \widehat{D} \rangle) = q_{rej}$ ולכן $H(\langle \widehat{D} \rangle, \langle \widehat{D} \rangle) = q_{rej}$ אז $\widehat{D}(\langle \widehat{D} \rangle) = q_{rej}$ סתירה.

אם $\widehat{D}(\langle \widehat{D} \rangle) = q_{rej}$ אז $D(\langle \widehat{D} \rangle) = q_{acc}$ ולכן $H(\langle \widehat{D} \rangle, \langle \widehat{D} \rangle) = q_{acc}$ אז $\widehat{D}(\langle \widehat{D} \rangle) = q_{acc}$ סתירה.

הוכחה – זמן:

נגדיר שפה:

$$A_f = \{\langle M, w \rangle \mid M(w) = q_{acc} \wedge M' \text{'s run on } w \text{ finishes with at most } f(|w|) \text{ steps}\}$$

ניתן להכריע את A_f בעזרת מ"ט אוניברסלית ומונה (בכמה זמן?) נראה בדומה להוכחה עבור A_{TM} שלא ניתן להכריע את A_f בזמן $o(f(n))$.

נניח בשלילה שקיימת מ"ט H_f שמכריעה את A_f תוך הסתפקות ב- $o(f(n))$ זמן ריצה.

נבנה בעזרת H_f את המ"ט D : $D(\langle M \rangle) = H_f(\langle M \rangle, \langle M \rangle)$

נבנה בעזרת D את המ"ט $\hat{D}$: $\hat{D}(\langle M \rangle) = \overline{D(\langle M \rangle)}$

מה מחזירה $\hat{D}(\langle \hat{D} \rangle)$?

אם $\hat{D}(\langle \hat{D} \rangle) = q_{acc}$ אז $D(\langle \hat{D} \rangle) = q_{rej}$ ולכן $H_f(\langle \hat{D} \rangle, \langle \hat{D} \rangle) = q_{rej}$ אז $\hat{D}(\langle \hat{D} \rangle) = q_{rej}$ סתירה. (זמן הריצה של H_f הוא $O(f(n))$ לכן גם של $D, \hat{D}$ – כלומר הדחייה אינה בעקבות זמן הריצה אלא בעקבות $(D(\langle \hat{D} \rangle) = q_{rej})$.

אם $\hat{D}(\langle \hat{D} \rangle) = q_{rej}$ אז $D(\langle \hat{D} \rangle) = q_{acc}$ ולכן $H_f(\langle \hat{D} \rangle, \langle \hat{D} \rangle) = q_{acc}$ אז $\hat{D}(\langle \hat{D} \rangle) = q_{acc}$ סתירה.

הראינו שקיימת שפה A_f שלא שייכת ל- $\text{Time}(o(f(n)))$. ברור ש- A_f כריעה, השאלה באיזו מגבלת זמן. דרך להכריע את A_f : לאתחל מונה ל- $f(|w|)$ ולהריץ אז המכונה האוניברסלית $\mathcal{U}$: $\mathcal{U}(\langle M, w \rangle)$ ואחרי סימולציה כל צעד ריצה של M להוריד את המונה ב-1. אם המונה הגיע ל-0, נעצור ונחזיר q_{rej} ואם לפני שהמונה הגיע ל-0 הסימולציה של M מגיעה ל- q_{acc} או ל- q_{rej} נענה בהתאם.

ברור שהמכונה הנ"ל מכריעה את A_f , אך בכמה זמן? יש מימוש של מכונת טיורינג אוניברסלית $\mathcal{U}$ שמדמה ריצה של $f(n)$ צעדים בעזרת $O(f(n) \log f(n))$ צעדים של $\mathcal{U}$. נדרוש שהפונקציה f תהיה חשיבה בזמן $O(f(n))$. כלומר בהינתן 1^n ניתן לחשב את $1^{f(n)}$ בזמן $O(f(n))$.

מסקנה: לכל פונקציה f שחשיבה בזמן $O(f(n))$ מתקיים:

$$\text{Time}(o(f(n))) \subsetneq \text{Time}(f(n) \log(f(n)))$$

דוגמה:

$$n^{2.1} \log n^{2.1} = O(n^3) \text{ וכן } n^2 = o(n^{2.1}) \text{ וגם } O(n^{2.1})$$

$$\text{Time}(n^2) \subsetneq \text{Time}(n^3)$$

$$\text{P} \subsetneq \text{EXP}$$

למשל, יש שפה $L \in \text{Time}(2^n)$ אבל $L \notin \text{Time}(n^k)$ לאף k קבוע.

הוכחה - מקום

ההוכחה כמעט זהה להוכחה עבור משפט ההיררכיה בזמן. נחליף את מגבלת הזמן בכל נקודה בהוכחה במגבלת מקום. המשפט חזק יותר לגרסת המקום (הפרדה צפופה יותר) כי קיימת מ"ט אוניברסלית $\mathcal{U}$ שמדמה ריצה של $M'(w)$ במקום $O(f(n))$ כאשר $f(n)$ מגבלת מקום ריצת M .

לכל פונקציה $f(n)$ שחשיבה במקום $O(f(n))$.

$$\text{NSpace}(f(n)) \subseteq \text{Space}(f^2(n))$$

כל מה שניתן להכריע על ידי מ"ט ל"ד ע"י הסתפקות ב- $f(n)$ מקום, אפשר להכריע עם מ"ט דטרמיניסטית ריבועי במקום.

מסקנה: $\text{NPSpace} = \text{PSpace}$

תזכורת:

$$\text{PSpace} = \bigcup_{k=1}^{\infty} \text{Space}(n^k) \quad ; \quad \text{NSpace} = \bigcup_{k=1}^{\infty} \text{NSpace}(n^k)$$

הוכחת המסקנה:

- כיוון $\text{PSpace} \subseteq \text{NPSpace}$ כי כל מ"ט דטרמיניסטית היא מקרה פרטי של מ"ט ל"ד עם אותה מגבלת מקום.

- נראה הכלה בכיוון השני. תהי $L \in \text{NPSpace}$, כלומר קיים k קבוע כלשהו כך ש- $L \in \text{NSpace}(n^k)$. לפי משפט Savitch מתקיים $L \in \text{Space}((n^k)^2)$. סך הכל קיבלנו:
 $L \in \text{Space}((n^k)^2) = \text{Space}(n^{2k}) \subseteq \text{PSpace}$

נשים לב שאילו היה משפט תואם עבור מחלקות זמן אז היינו מסיקים $P = NP$.

הוכחת משפט Savitch

עבור מ"ט M וקלט w נגדיר את גרף הקונפיגורציות $G_{M,w}$. צמתי הגרף הם כל הקונפיגורציות האפשריות. יש קשת מ- u ל- v אם הם הקונפיגורציה המתאימה ל- v היא קונפיגורציה עוקבת של הקונפיגורציה המתאימה ל- u .

עבור מ"ט דטרמיניסטית לכל קונפיגורציה יש קונפיגורציה עוקבת יחידה ולכן לכל צומת יש קשת יוצאת אחת ויחידה, אבל במ"ט לא דטרמיניסטית יתכנו מספר קונפיגורציות עוקבות לכל קונפיגורציה, ולכן לכל צומת תתכן יותר מקשת יוצאת אחת.

נגדיר שני צמתים מיוחדים בגרף – צומת q_0 שמתאים לקונפיגורציה ההתחלתית, וצומת q_{acc} שמתאים לקונפיגורציה המקבלת $C_{acc} = q_{acc}$.

תזכורת: הראינו במהלך משפט [קוק-ליין](#) שבה"כ יש קונפיגורציה מקבלת יחידה. נשים לב שהשינוי ל- M כדי שתהיה קונפיגורציה מקבלת יחידה לא הגדיל את סיבוכיות המקום.

נשים לב: $w \in L \Leftrightarrow$ קיימת ריצה מקבלת של $M(w) \Leftrightarrow$ בגרף $G_{M,w}$ יש מסלול מכוון מ- q_0 ל- q_{acc} .

כמה צמתים יש ב- $G_{M,w}$ (כמה קונפיגורציות אפשריות יש)?

$$\#conf = \underbrace{|Q|}_{\text{מצב פנימי}} \times \underbrace{O(f(n))}_{\text{מקום ראש}} \times \underbrace{|\Gamma|^{O(f(n))}}_{\text{תוכן סרט}}$$

ברור שאפשר להכריע את L על ידי מ"ט דטרמיניסטית באופן הבא:

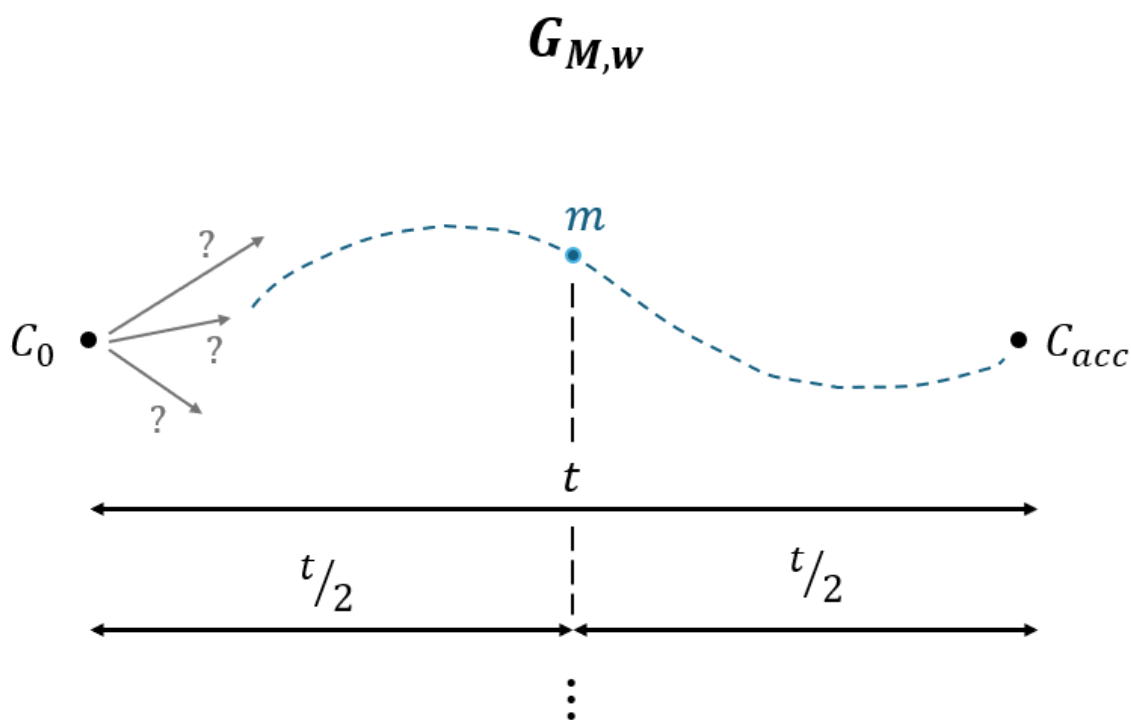
נניח את $G_{M,w}$, נניח BFS/DFS החל מהצומת C_0 ונענה q_{acc} אם"ם מגיעים ל- C_{acc} . הנכונות ברורה.

מקום הריצה $\geq$ מספר הצמתים ב- $G_{M,w}$, בפרט אקספוננציאלי ב- $f(n)$.

עבור שפה $L \in \text{NSpace}(f(n))$ קיימת מ"ט לא דטרמיניסטית M שמסתפקת במקום $O(f(n))$. נרצה

להראות שקיימת מ"ט M' דטרמיניסטית שמכריעה את L תוך הסתפקות במקום $O(f^2(n))$. רוצים לבדוק

אם יש מסלול בגרף הבא בלי לייצר את כולו (אחרת נחרוג מהמגבלות שהצבנו):



אבל, נבצע *reuse* במקום – נזכור רק מה התשובה של השלב הקודם ואז נשתמש באותו זיכרון. הפרוצדורה

$Reach(u, v, t)$ עונה על השאלה: האם יש מסלול ב- $G_{M,w}$ שמתחיל ב- u מסתיים ב- v ואורכו $\geq t$. נראה

מ"ט דטרמיניסטית שמחשבת את $Reach$.

Reach(u, v, t)

```

1  |  $u = C_0, v = C_{acc}, t = \#conf$ 
2  | if ( $u = v$ ) return T
3  | if ( $v$  is a successor of  $u$ ) return T
4  | if ( $t \leq 1$ ) return F
5  | for each  $m \in V(G_{M,w} = \langle V, E \rangle)$  do:
6  |   |  $q_1 = \text{Reach}(u, m, \lceil t/2 \rceil)$ 
7  |   |  $q_2 = \text{Reach}(m, v, \lceil t/2 \rceil)$ 
8  |   | if ( $q_1 \wedge q_2$ ) return T
9  | return F

```

נכונות ברורה. ננתח מקום ריצה:

לכל קריאה רקורסיבית נפתח שלשה חדשה אבל 2 קריאות באותו עומק רקורסיה משתמשות באותו המקום. נרצה לנתח את עלות המקום של M' . בדיקה האם $u = v$ לא דורשת מקום נוסף. בדיקה האם v עוקב של u גם לא דורשת מקום נוסף.

כמה מקום דורשת שלשה אחת?

○ $u - \log(\#conf)$ כלומר:

$$\log(|Q| \times O(f(n)) \times |\Gamma|^{O(f(n))}) = \log|Q| + O(\log(f(n))) + O(f(n)) \cdot \log|\Gamma| = O(f(n))$$

○ $v - \text{תיאור דורש } O(f(n))$

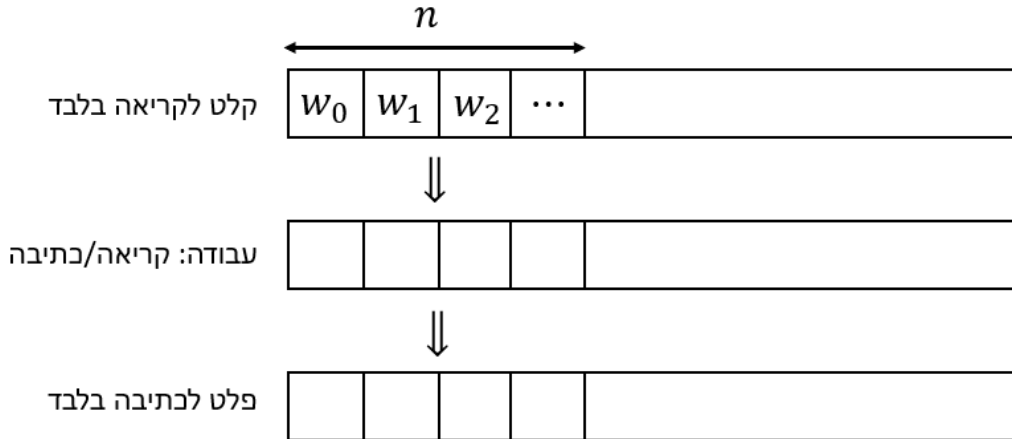
○ $t - \text{תיאור דורש } O(f(n))$

סה"כ לכל השלשה - $O(f(n))$. מספר השלשות המקסימלי הוא עומק הרקורסיה המקסימלי (כמה שלשות יש במקביל על הסרט). סה"כ $O(\log(\#conf))$ כלומר $O(f(n))$. סה"כ מקום $O(f^2(n))$.

דוגמאות: $NL = NSpace(\log(n))$, $L = Space(\log(n))$

רלוונטי במקרים בהם זיכרון המחשב $\gg$ גודל הקלט (למשל, אפליקציה ששואלת שאלות על האינטרנט כולו).

נגדיר מודל מ"ט שמתאים לדיון על מקום תת-לינארי. מכונה עם 3 סרטים:



ראש סרט הפלט יכול בכל צעד ריצה להדפיס אות (או לא), אם מדפיס אז צעד ימינה ולא חוזר:

$$\delta: \underbrace{Q}_{\text{ראש סרט}} \times \underbrace{\Sigma}_{\text{כתיבה}} \times \underbrace{\Gamma}_{\text{ראש סרט עבודה}} \rightarrow \underbrace{Q}_{\text{ראש סרט}} \times \underbrace{\Gamma}_{\text{בסרט עבודה}} \times \underbrace{\{\Sigma \cup \emptyset\}}_{\text{כתיבה בסרט פלט}} \times \underbrace{\{R, L\}}_{\text{פקודת תנועה לראש סרט קלט ועבודה}}$$

מקום הריצה נמדד על סרט העבודה בלבד, לכן גם דרשנו שראש סרט הפלט לא יוכל לחזור אחורה. במכונת הכרעה עם מקום תת-לינארי, אין צורך בסרט פלט. במכונת חישוב פונ' עם מקום תת-לינארי, בסוף הריצה המכונה עוברת ל- q_{acc} ועל סרט הפלט רשום $f(w)$ (הפונקציה שהמכונה מחשבת).

דוגמה - שפה $A \in Space(\log(n)) = L$ (מרחב שפות תת-לינאריות) אם קיימת מ"ט M דטרמיניסטית שמכריעה את A ומסתפקת במקום $O(\log(n))$, כלומר M היא מכונת 2 סרטים (קלט ועבודה) ומגבלת המקום על סרט העבודה היא $O(\log(n))$.

טענה

$L \subseteq P$, כלומר אם $A \in L$ אז $A \in P$.

אם יש מ"ט דטרמיניסטית שמסתפקת במקום לוגריתמי **אז** יש מ"ט דטרמיניסטית עבור אותה השפה שמסתפקת בזמן פולינומיאלי.

הוכחה

תהי M מ"ט דטרמיניסטית שמכריעה את A תוך הסתפקות במקום לוגריתמי. כמה קונפיגורציות יש למכונה?

$$|Q| \times n \times O(\log n) \times |\Gamma|^{O(\log(n))}$$

↑ ↑ ↑

אפשרויות למיקום ראש הקלט אפשרויות לתוכן סרט העבודה אפשרויות למיקום ראש סרט העבודה

נעריך את כמות האפשרויות לתוכן סרט העבודה:

$$|\Gamma|^{O(\log(n))} = 2^{\log_2 |\Gamma| \cdot O(\log(n))} = O(n^c)$$

עבור c קבוע כלשהו.

סה"כ מספר הקונפיגורציות אם כך יהיה $O(n^d)$ עבור d קבוע כלשהו, בעוד ש- M לא חוזרת על קונפיגורציה פעמיים כי אז הייתה בלולאה אינסופית.

מסקנה: זמן ריצת M על $w \geq$ מספר הקונפיגורציות האפשריות שחסום ב- $O(n^d)$. כלומר, זמן ריצה פולינומי, לכן $A \in P$.

טענה

אם M מחשבת פונקציה f , ו- M מ"ט דטרמיניסטית שמסתפקת במקום $O(\log(n))$ אז אורך הפלט $|f(w)|$ חסום מלמעלה על ידי פולינום.

הוכחה:

בדומה להוכחה הקודמת, מספר הקונפיגורציות חסום על ידי פולינום ולכן מספר צעדי הריצה וגם מספר ההדפסות חסום על ידי פולינום.

כדי לדון במחלקות מקום תת-לינארי הגדרנו מ"ט עם 3 סרטים: סרט קלט (קריאה בלבד), עבודה (קריאה/כתיבה, ועליו בלבד מודדים את המקום), פלט (כתיבה בלבד, בכל צעד אפשר להדפיס אות או לא. אם מדפיסים, הראש זז צעד ימינה ולא חוזר שמאלה). $|w| = n$, מודדים מקום כתיבה בלבד.

מחלקות: $L = \text{Space}(\log n)$, $NL = \text{NSpace}(\log n)$.

משפט Savitch:

$$\text{NSpace}(f(n)) \not\subseteq \text{Space}(f^2(n))$$

לכל $f(n)$ שחשיבה במקום $O(f(n))$. הראינו לפונקציה $f(n) \geq n$. המשפט נכון גם עבור $f(n) < n$.

תהי $f(x)$ פונקציה חשיבה במקום $O(\log n)$. כלומר, קיימת מ"ט M_f עם 3 סרטים (כמוגדר למעלה) כך שלכל קלט x על סרט הקלט M_f מסיימת במצב q_{acc} עם $f(x)$ על סרט הפלט, תוך שימוש במקום $O(\log n)$ בסרט העבודה כאשר n הוא אורך הקלט. תהי $g(x)$ פונקציה חשיבה גם במקום $O(\log n)$ ע"י המכונה M_g .

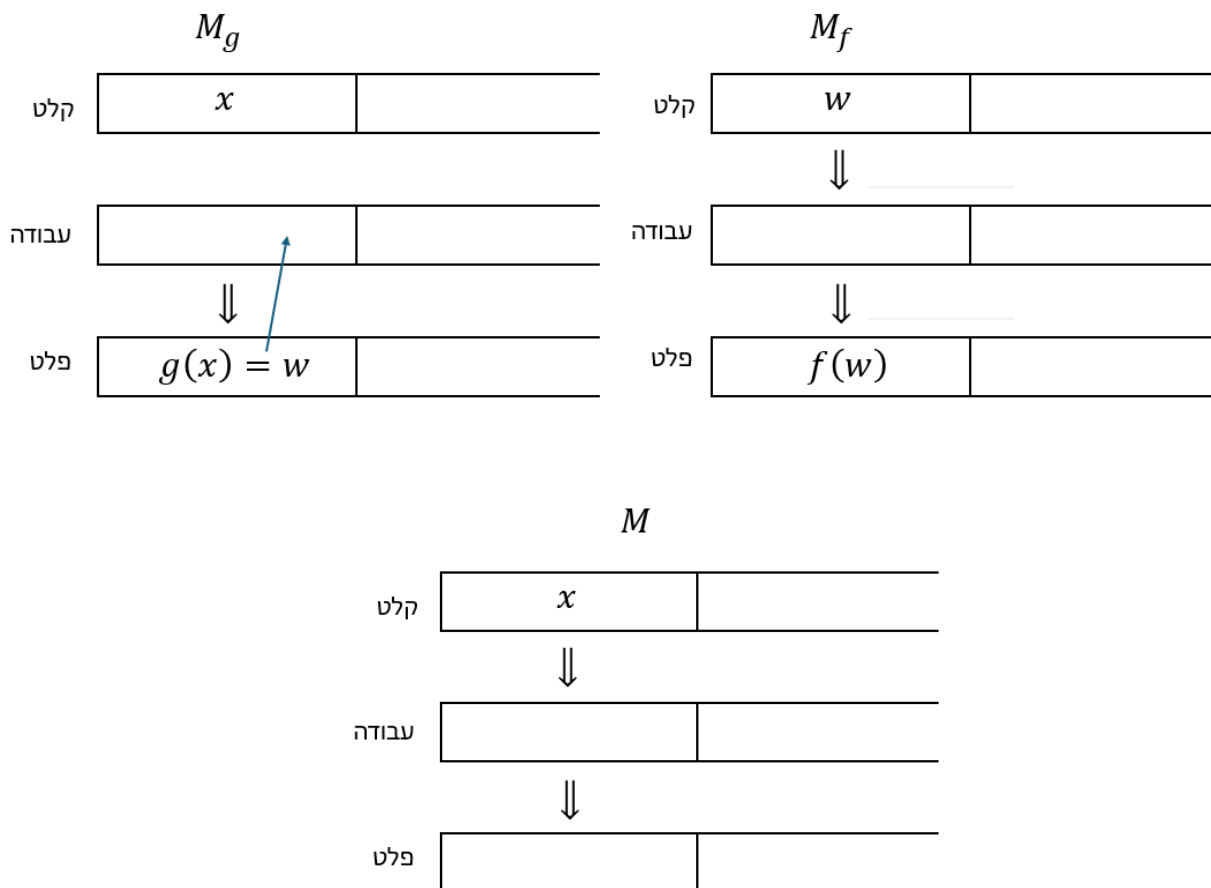
טענה

$f(g(x))$ חשיבה במקום $O(\log n)$.

תזכורת: הראינו שאם f, g חשיבות אז $f \circ g$ חשיבה, והראינו שאם f, g חשיבות בזמן פולינומי אז $f \circ g$ חשיבה בזמן פולינומי. איך? בנינו מכונה חדשה M שעבור קלט x מריצה את $M_g(x) = w$, שומרת את w על הסרט ואז מריצה את $M_f(w)$ וכך מקבלים את $f(g(x))$. ברור שאם f, g חשיבות הנ"ל הוכחה ש- $f \circ g$ חשיבה. בנוסף אם f, g חשיבות בזמן פולינומי אז ההרכבה הנ"ל רצה בזמן פולינומי ב- $|x|$, כלומר $f \circ g$ חשיבה בזמן פולינומי.

איך אפשר להרכיב 2 מכונות של 3 סרטים כל אחת למכונת 3 סרטים שמחשבת את $f(g(x))$?

פתרון 1 – נכון אבל בזבזני במקום: נשנה את M_g כך שבמקום להדפיס לסרט הפלט, תדפיס לסרט העבודה. נשנה את M_f כך שבמקום לקרוא מסרט הקלט תקרא מסרט העבודה. המכונה החדשה M תפעל באופן הבא: תריץ את M_g החדשה על x ואז תריץ את M_f החדשה (על מה שכתוב בסרט העבודה). נכונות ברורה. נותר להבין עלות מקום.



התוצאה של M היא אכן $f(g(x))$ הבעיה היא ש- M עלולה להשתמש במקום שחורג ממגבלה $O(\log n)$. הסבר: הראינו בשבוע שעבר שמ"ט דטרמיניסטית עם מגבלת מקום לוגריתמית, זמן הריצה שלכן ולכן גם אורך הפלט שלה חסומים על ידי פולינום. זה אומר ש- $|w| = |g(x)|$ עלול להיות פולינומי, ולכן המכונה M עלולה להשתמש במקום פולינומי ב- n במקום לוגריתמי ב- n .

פתרון 2 – הרצה סימולטנית של M_g ו- M_f : נראה פתרון שמאפשר הרכבה כל שסה"כ מקום הריצה הוא $O(\log n)$. הרעיון: נחשב אותיות של $g(x)$ באופן דינמי בכל פעם מחדש. כלומר, נריץ בו זמנית את M_f ואת M_g . הסבר מפורט יותר:

נחזיק 2 מונים, אחד שמחליף את ראש ההדפסה של M_g ואחד שמחליף את ראש הקלט של M_f . המכונה M תחשב את $f(g(x))$ באופן הבא: תרוץ כמו M_f , אבל בכל צעד ריצה כדי לדעת מה האות הנוכחית מתוך $M, g(x)$ תריץ את M_g מספר נדרש של הדפסות. כלומר: כל תנועה שמאלה של ראש קורא של f נדמה על ידי הורדת מונה ב-1 שמדמה ראש סרט קלט M_f וכל תנועה ימינה – הגדלה באחד של המונה. כל פקודת הדפסה של M_g נדמה על ידי הגדלת מונה ראש פלט של M_g ב-1. בכל צעד של M_f נריץ את M_g על x

מההתחלה (מאפסים את מונה ההדפסות). מריצים את M_g עד שמונה ההדפסות שווה בערכו למונה של ראש סרט קלט M_f ואז M יודעת מה האות הנוכחית ש- M_g מדפיסה ו- M_f קוראת.

נכונות המכונה M ברורה. ברור גם שזמן הריצה גדול. נרצה להשתכנע שמקום הריצה הוא $O(\log n)$. מה M צריכה להחזיק על סרט העבודה?

- מקום של סרט עבודה של M_f .
- מקום של סרט עבודה של M_g .
- מונה עבור ראש פלט M_g .
- מונה עבור ראש קלט M_f .

נעריך את גודלם:

א. M_f מסתפקת במקום $O(\log |g(x)|)$ (האורך של הקלט של M_f , כלומר $|g(x)|$). אם כן, M_f מסתפקת במקום $O(\log(n^c))$ ולכן M_f מסתפקת במקום $O(\log n)$.

תזכורת: מכונה דטרמיניסטית שמסתפקת במקום לוגריתמי, אורך הפלט שלה חסום על ידי פולינום, כלומר אורכו $n^c \geq$ עבור c קבוע כלשהו.

- M_g מסתפקת במקום $O(\log |x|)$ לחישוב $g(x)$ כלומר $O(\log n)$.
- צריך מונים שיכולים לספור עד $|g(x)|$ (האורך של פלט של M_g , כלומר האורך של הקלט של M_f). הראינו כבר ב-א' ש- $|g(x)| \leq n^c$, לכן עבור מונה לאורך זה נצטרך $O(\log(n^c))$ תאים, כלומר $O(\log n)$ תאים.
- כן"ל.

סה"כ ל-4 הסעיפים: $O(\log n)$ תאים. כלומר M מכונה שמחשבת את $f(g(x))$ תוך הסתפקות במקום $O(\log n)$. כלומר הרכבת שתי פונקציה חשיבות במקום לוגריתמי היא פונקציית חשיבה במקום לוגריתמי.

הערה: הראינו עבור מקום **לוגריתמי**, אפשר היה להראות גם עבור פונקציות תת לינאריות אחרות.

הגדרה – רדוקציה לוגריתמית (במקום): עבור שפות A, B , נגיד שיש רדוקציה לוגריתמית מ- A ל- B ונסמן $A \leq_L B$ אם קיימת רדוקציה f מ- A ל- B כך ש- f חשיבה במקום לוגריתמי.

הערה: אם $A \leq_L B$ אז $A \leq_P B$ המכונה שמחשבת את הרדוקציה במקום לוגריתמי מסתפקת גם בזמן פולינומי.

שאלות:

אם $L = NL$ אז שאלה פתוחה. האם $L = P$ אז שאלה פתוחה.

האם $NL = P$ אז שאלה פתוחה. ראינו $L \subseteq P$, נראה בתרגול $NL \subseteq P$.

טענה – סגירות הרכבת רדוקציות

אם $A \leq_L B$ וגם $B \leq_L C$ אז $A \leq_L C$.

הוכחה: בדומה להוכחת הרכבת רדוקציות פולינומיות. נשתמש פה בסגירת הרכבת פונקציות חשיבות במקום לוגריתמי.

כלומר אם f חשיבה במקום לוגריתמי היא הרדוקציה מ- A ל- B ,

וגם g חשיבה במקום לוגריתמי היא הרדוקציה מ- B ל- C ,

אז $g \circ f$ חשיבה במקום לוגריתמי היא הרדוקציה מ- A ל- C .

הגדרה - NL-קשה: נגיד ששפה B היא NL-קשה אם לכל $A \in NL$ מתקיים $A \leq_L B$.

טענה

אם B שפה NL-קשה וגם $B \leq_L C$ אז C היא NL-קשה.

הוכחה: $B \in \text{NL-Hard}$ אז לכל $A \in NL$ מתקיים $A \leq_L B$. בנוסף, $B \leq_L C$ (נתון). מתוך סגירות רדוקציות $\log \text{space}$ להרכבה נובע $A \leq_L C$ כלומר $C \in \text{NL-Hard}$.

הגדרה - NL-שלמה: נגיד ששפה C היא NL-שלמה אם לכל $C \in \text{NL-Hard}$ וגם $C \in NL$.

נרצה להראות שפה שלמה ב-NL, והיא תשמש אותנו באופן דומה לשימוש של משפט קוק-ליון עבור המחלקה NP. נגדיר את השפה:

$\text{PATH} := \{(G, s, t) \mid G \text{ is a directed graph, } s, t \in V, \text{ and there is a directed path from } s \text{ to } t \text{ in } G\}$

לפעמים נקראת גם S-T-CONN.

$PATH \in NL\text{-Complete}$.

עלינו להראות:

$PATH \in NL$ (1)

$PATH \in NL\text{-Hard}$ (2)

הוכחה:

(1) נראה מ"ט (לא דטרמיניסטית) שמסתפקת במקום $O(\log n)$ עבור השפה $PATH$. המכונה M תפעל באופן הבא: תחזיק משתנה עבור צומת נוכחי שיאותחל ל- s . בכל צעד, M תעבור באופן לא דטרמיניסטי לאחד השכנים של הצומת הנוכחי, ותבדוק האם הגענו ל- t . אם כן, M תעצור ותחזיר q_{acc} . אם לא, תמשיך לשכן הבא.

נכונות: אם $(G, s, t) \in PATH$ אז קיים מסלול מכון מ- s ל- t ב- G , לכן יש ריצה שבה M מבצעת טיול מ- s ל- t ואז מחזירה q_{acc} . אם $(G, s, t) \notin PATH$ אז בכל ריצה של M הטיול לא מגיע ל- t ולכן M לא מחזירה q_{acc} .

מקום: M מחזיקה מקום על סרט העבודה עבור צומת נוכחי $O(\log n)$, ומקום עבור הצומת הבא ועוד מונה או שניים כדי לחפש קשתות בייצוג של G בסרט הקלט. כל אחד ב- $O(\log n)$, סה"כ $O(\log n)$.

הערות:

- אפשר לשנות את M כך שתמיד תעצור. למשל, להוסיף מונה אורך המסלול. מגדילים אותו באחד בכל צעד על המסלול, ואם הוא חורג ממספר הצמתים ב- G עוצרים ומחזירים q_{rej} .
- המעבר מצומת לשכן שלו הוא בעזרת הקלט G . איך לעשות זאת תלוי בצורת הייצוג שדרשנו מ- G (רשימת קשתות, מטריצת שכנויות וכו'). בכל מקרה, ניתן לביצוע בעזרת מונה או שניים, כלומר מקום נוסף $O(\log n)$.

(2) נראה ש- $PATH \in NL\text{-Hard}$. כלומר, לכל $A \in NL$ מתקיים $A \leq_L PATH$. כלומר יש פונקציה f חשיבה במקום לוגריתמי, כך שלכל w מתקיים $(G, s, t) \in PATH \Leftrightarrow f(w) = (G, s, t) \in PATH$. הנתון הנוסף היחיד: קיימת מ"ט ל"ד M שמסתפקת במקום לוגריתמי עבור A .

תזכורת: גרף הקונפיגורציות $G_{M,w}$ מקיים שיש מסלול מצומת הקונפיגורציה ההתחלתית C_0 אל צומת הקונפיגורציה המקבלת C_{acc} בגרף $G_{M,w}$ אם ורק אם יש ריצה מקבלת של $M(w)$ אם $w \in A$ (בה"כ יש קונפ' מקבלת יחידה).

לכן: עבור קלט w הרדוקציה תחזיר את $(G, s, t) = (G_{M,w}, C_0, C_{acc})$.
נכונות ברורה, נותר להראות שהרדוקציה הזו חשיבה במקום לוגריתמי.

נרצה להראות שאפשר, בהינתן w , לייצר את $(G_{M,w}, C_0, C_{acc})$ על ידי מ"ט דטרמיניסטית שמסתפקת במקום לוגריתמי.

C_0 – הקונפיגורציה ההתחלתית, כלומר סרט עבודה ריק והראשים בהתחלה, מצב פנימי q_0 .

C_{acc} – הקונפיגורציה המקבלת, כלומר סרט עבודה ריק, הראשים בהתחלה, מצב פנימי q_{acc} .

קל לייצר את C_0, C_{acc} . נותר להבין איך מדפיסים את $G_{M,w}$ ע"י מכונת $\log$ space. נבחר את הייצוג של קשתות, כלומר זוגות צמתים.

מכונת הרדוקציה M_f תייצר את $G_{M,w}$ באופן הבא:

M_f תחזיק זוג משתנים על סרט העבודה. כל אחד מהם מייצג קונפיגורציה. המקום הנדרש לייצוג מספר הקונפיגורציות הוא:

$$|Q| \times n \times O(\log n) \times |\Gamma|^{O(\log n)}$$

$\uparrow$
מצב

$\uparrow$
מיקום ראש
סרט קלט

$\uparrow$
מיקום ראש
סרט עבודה

$\uparrow$
תוכן סרט
עבודה

אפשר לשמור קונפיגורציה במקום $O(\log n)$. לכל זוג הצבות לשני המשתנים המכונה תבדוק האם הם תואמים לזוג קונפיגורציות עוקבות. אם כן – תעתיק/תדפיס לסרט הפלט. בכל מקרה, גם אם לא, תקדם את המשתנים והמונים. כלומר, M תעבור על כל הערכים האפשריים לשני המשתנים.

בדיקה אם זוג ערכים מתאים לזוג קונפיגורציות עוקבות – כלומר האם 2 הקונפיגורציות כמעט זהות, למעט שינויים שתואמים לפונקציה δ של M , ניתן לעשות במגבלת המקום.

הערה: כדי לשמור על קונפיגורציה אחת צריך מקום לתוכן סרט העבודה $O(\log n)$, מיקום ראש סרט עבודה $O(\log(\log n))$, מיקום ראש סרט קלט $O(\log n)$ ומצב פנימי $O(\log n)$. סה"כ $O(\log n)$.

העשרה: מה חסם הריצה העליון שלנו? המקום לוגריתמי, אז הזמן פולינומי.

$$L \subseteq NL \subseteq P \subseteq NP$$

בכל אחד מהמעברים, אנחנו לא יודעים להכריע האם יש הכלה ממש או שוויון.

$$L \stackrel{?}{=} NL, L \stackrel{?}{=} P, NL \stackrel{?}{=} P, P \stackrel{?}{=} NP$$

משפט Savitch – המשך

לכל $f(n)$ שחשיבה במקום $O(f(n))$ מתקיים $\text{NSpace}(f(n)) \subseteq \text{Space}(f^2(n))$. הראינו את הנ"ל לכל $f(n)$ שהיא לפחות $n \leq f(n)$. נראה עכשיו שהמשפט תקף גם עבור $\log n \leq f(n) \leq n$.

תזכורת: תהי $A \in \text{NSpace}(f(n))$ ותהי M מ"ט לא דטרמיניסטית עבור A שמסתפקת במקום $O(f(n))$. עבור מילה w נסתכל על גרף הקונפיגורציות $G_{M,w}$ שבו צומת לכל קונפיגורציה, קשת לכל מעבר חוקי של δ של M , ושתי קונפיגורציות מיוחדות C_0 ו- C_{acc} . $w \in A \Leftrightarrow$ קיימת ריצה מקבלת $M(w) \Leftrightarrow \langle G_{M,w}, C_0, C_{acc} \rangle \in \text{Path}$. כלומר, קיים מסלול מכוון מ- C_0 ל- C_{acc} ב- $G_{M,w}$.
 בנינו מכונה דטרמיניסטית M' שמכריעה האם $\langle G_{M,w}, C_0, C_{acc} \rangle \in \text{Path}$ תוך הסתפקות במקום קטן יחסית: $O(f^2(n))$. [להוכחה המלאה](#)

מה מספר הקונפיגורציות עבור מכונת NL (כאשר $f(n) = O(\log n)$). מספר הקונפיגורציות:

$$|Q| \times (n + 1) \times O(\log n) \times |\Gamma|^{O(\log n)}$$

$\uparrow$
מצב

$\uparrow$
מיקום ראש
סרט קלט

$\uparrow$
מיקום ראש
סרט עבודה

$\uparrow$
תוכן סרט
עבודה

הערה 1: נשים לב שבספירת הקונפיגורציות לא ספרנו את האפשרויות של סרט הקלט (כביכול $|\Sigma|^n$). מדוע?
 כיוון ש- w הוא נתון. בהינתן קלט נתון, בין 2 קונפיגורציות שונות, סרט הקלט זהה ולכן לא משתנה בין צמתים שונים של אותו הגרף $G_{M,w}$.

הערה 2: למה ה-1+ במיקום ראש סרט קלט? בגלל ה- $_$ בסוף המילה.

$$\log(\# \text{conf}) = O(\log n)$$

משפט Immerman-Szelepcsényi

משפט: $\text{NL} = \text{coNL}$

כלומר, $\text{NSpace}(\log n) = \text{coNSpace}(\log n)$, כלומר $A \in \text{NL} \Leftrightarrow \bar{A} \in \text{NL}$.

ראינו כבר שמתקיים $\text{NPspace} = \text{Pspace}$.

תזכורת: $\text{Pspace} \subseteq \text{NPspace}$ – ברור. את $\text{Pspace} \supseteq \text{NPspace}$ מראים בעזרת משפט Savitch.

תהי $A \in \text{NPSpace}$, כלומר $A \in \bigcup_{k=1}^{\infty} \text{NSpace}(n^k)$. כלומר, קיים k כך שמתקיים: $A \in \text{NSpace}(n^k)$.
 ממשפט Savitch נסיק כי $A \in \text{Space}((n^k)^2)$ כלומר $A \in \text{Space}(n^{2k})$ ולכן $A \in \bigcup_{k=1}^{\infty} \text{Space}(n^k) = \text{PSPACE}$.

טענה:

$\text{NPSpace} = \text{coNPSpace}$. כלומר, $A \in \text{NPSpace} \Leftrightarrow \bar{A} \in \text{NPSpace}$.

הוכחה:

תהי $A \in \text{NPSpace}$. נראה כי $\bar{A} \in \text{NPSpace}$. הנחנו $A \in \text{NPSpace}$, אבל $\text{NPSpace} = \text{PSPACE}$ ולכן $A \in \text{PSPACE}$, כלומר יש מ"ט דטרמיניסטית שמכריעה את A תוך הסתפקות במקום פולינומי. לכן גם $\bar{A} \in \text{PSPACE}$. הסבר: אותה מכונה דטרמיניסטית מכריעה את A . בהחלפת q_{acc} ו- q_{rej} היא תהיה גם מכונת הכרעה עבור $\bar{A} \in \text{NPSpace}$ עם אותן דרישות מקום, ולכן $\bar{A} \in \text{NPSpace}$ כנדרש.

האם יכולנו להשתמש באותה השיטה כדי להראות $\text{NL} = \text{coNL}$?

לא. שימוש בממשפט Savitch כדי לעבור ממ"ט לא דטרמיניסטית לדטרמיניסטית מעלה בריבוע את סיבוכיות המקום. עבור מקום פולינומי זה לא הפריע לנו, אבל עבור NL זה נותן:

$$B \in \text{NL} \Rightarrow B \in \text{Space}(\log^2 n)$$

ולא ברור אם אפשר מפה לחזור ל- coNL או ל- NL . אפשר לקבל מ"ט שמכריעה את $\bar{B}$ אבל היא במקום $O(\log^2 n)$.

נתונה $A \in \text{coNL}$, כלומר $\bar{A} \in \text{NL}$. תהי M מכונת NL עבור $\bar{A}$. האם החלפת מצבי q_{acc}, q_{rej} נותנת מכונת NL עבור A ?

באופן כללי – לא. הסבר: מכונת NL עבור $\bar{A}$ מבטיחה שאם $w \in \bar{A}$ אז קיימת ריצה מקבלת, ואם $w \notin \bar{A}$ אז כל ריצה היא דוחה. היפוך המצבים q_{acc} ו- q_{rej} ייתן מכונה שבה אם $w \in \bar{A}$ קיימת ריצה דוחה ואם $w \notin \bar{A}$ אז כל ריצה היא מקבלת. כלומר, אם $w \notin A$ אז יש ריצה דוחה, ואם $w \in A$ אז כל ריצה מקבלת.

דוגמה קונקרטית:

$$\text{Path} = \{ \langle G, s, t \rangle \mid \text{There's a directed path from } s \text{ to } t \text{ in } G \}$$

ראינו $\text{Path} \in \text{NL}$, בפרט $\text{Path} \in \text{NL-Complete}$. ראינו מכונת NL עבור Path , שמתחילה מ- s ובוחרת מסלול באופן ולא דטרמיניסטית ומחזירה q_{acc} אם המסלול מסתיים ב- t . החלפת מצבי q_{acc}, q_{rej} נותנת מכונה שלא מבטיחה נכונות עבור $\overline{\text{Path}}$. בפרט, עבור $\langle G, s, t \rangle$ אם המכונה החזירה q_{acc} ייתכן שזה בגלל

שאין מסלול מ- s ל- t ב- G , כלומר $\langle G, s, t \rangle \notin \overline{\text{Path}}$, אבל ייתכן שמאחר שהמכונה ניחשה מסלול אחר, ודווקא יש מסלול מ- s ל- t ב- G , כלומר המכונה הייתה אמורה להחזיר q_{rej} .

טענה

$$\text{NL} = \text{coNL} \Leftrightarrow \overline{\text{Path}} \in \text{NL}$$

הוכחה:

$\Leftarrow$ אם $\text{NL} = \text{coNL}$ אז כיוון ש- $\text{Path} \in \text{NL}$ נסיק $\overline{\text{Path}} \in \text{NL}$.

$\Rightarrow$ נראה בהכלה דו-כיוונית.

תזכורת: $\text{Path} \in \text{NL-Hard}$. תהי $A \in \text{coNL}$, אז $\bar{A} \in \text{NL}$, לכן $\bar{A} \leq_L \text{Path}$. מכאן $A \leq_L \overline{\text{Path}}$ (באמצעות אותה הרדוקציה), ונתון לנו $\overline{\text{Path}} \in \text{NL}$ אז ממשפט הרדוקציה נקבל $A \in \text{NL}$. מסקנה $\text{NL} \supseteq \text{coNL}$.

תהי $B \in \text{NL}$, לכן $\bar{B} \in \text{coNL}$. לפי ההכלה נקבל $\bar{B} \in \text{NL}$, כלומר $\text{NL} \subseteq \text{coNL}$.

לכן $\text{NL} = \text{coNL}$.

הוכחנו גרירה בשני הכיוונים ולכן $\overline{\text{Path}} \in \text{NL}$ הוא תנאי הכרחי ומספיק לכך ש- $\text{NL} = \text{coNL}$. לכן, כדי להוכיח את משפט אימרמן נרצה להראות מכונת NL עבור:

$$\overline{\text{Path}} = \{ \langle G, s, t \rangle \mid \text{The directed graph } G \text{ has no path from } s \text{ to } t \}$$

דוגמאות לשימוש במשפט אימרמן:

דוגמה 1

$$A = \{ \langle G, u, v \rangle \mid G \text{ מכונן } u \text{ לא באותו רכיב קשירות חזקה בגרף המכוון } G \}$$

תזכורת: 2 צמתים u, v באותו רכיב קשירות חזקה אם יש מסלול מכוון מ- u ל- v וגם יש מסלול מכוון מ- v ל- u ב- G .

תזכורת: אם $C, D \in \text{NL}$ אז גם $C \cup D, C \cap D \in \text{NL}$.

צ"ל: $A \in \text{NL}$.

הוכחה: ממשפט אימרמן $\overline{\text{Path}} \in \text{NL}$. $\langle G, u, v \rangle \in A \Leftrightarrow$ אין מסלול מ- u ל- v או אין מסלול מ- v ל- $u \Leftrightarrow \langle G, v, u \rangle \in \overline{\text{Path}}$ או $\langle G, u, v \rangle \in \overline{\text{Path}}$.

כלומר, איחוד של 2 שפות ב- NL ולכן $A \in \text{NL}$.

$$2\text{-Con} = \{G \mid G \text{ גרף מכונן, ויש ב-} G \text{ לפחות 2 רכיבי קשירות חזקה}\}$$

רוצים להראות $2\text{-Con} \in \text{NL}$.

הוכחה: נראה מכונת NL עבור 2-Con . M תפעל באופן הבא: לכל זוג צמתים $u, v \in V(G)$ תריץ את מכונת $\overline{\text{Path}}$ על $\langle G, u, v \rangle$.

אם המכונה מחזירה q_{acc} אז M תענה כמוה ותעצור. אחרת M תמשיך לזוג u, v הבא. אם עברה על כל הזוגות אז M תעצור ותחזיר q_{rej} .

נכונות: אם $G \in 2\text{-Con}$ אז קיימים זוג צמתים u, v שלא באותו רכיב קשירות חזקה. בה"כ, אין מסלול מ- u ל- v , ואז בהרצת מכונת $\overline{\text{Path}}$ על $\langle G, u, v \rangle$ יש ריצה שתחזיר q_{acc} , לכן יש ריצה של M שתחזיר q_{acc} .

אם $G \notin 2\text{-Con}$ כלומר כל G הוא רכיב קשירות חזקה אחד, אז לכל זוג צמתים u, v כל ריצה של מכונת $\overline{\text{Path}}$ על $\langle G, u, v \rangle$ מחזירה q_{rej} , ולכן M תמיד מחזירה q_{rej} .

סיבוכיות מקום: משתנה עבור u ועבור v : $O(\log n)$. מקום עבור ריצת מכונת $\overline{\text{Path}}$ הוא $O(\log n)$. סה"כ $O(\log n)$.

הוכחת משפט אימרמן:

צ"ל $\overline{\text{Path}} \in \text{NL}$.

רעיון כללי: נסתכל על מכונות NL שמבצעות חישובים ומבטיחות שבתום הריצה:

א. הפונקציה שמחשבים $f(n)$ רשומה בפלט, והמצב הפנימי הוא q_{acc} .

או

ב. לא בטוח מה רשום בפלט, והמצב הפנימי הוא q_{rej} .

ולכל קלט יש

ג. לפחות ריצה אחת מסתיימת ב- q_{acc} .

איך זה עוזר?

דוגמה: נניח שיש מכונת NL כנ"ל שנקרא לה M' , שעבור קלט $\langle G, s \rangle$ מחשבת את C , מספר הצמתים שאפשר להגיע אליהם מ- s ב- G . בעזרת M' ניתן לבנות מכונת NL בעזרת $\overline{\text{Path}}$. נריץ את M' פעם אחת על $\langle G, s \rangle$ ונחשב את C , פעם שניה על $\langle G \setminus t, s \rangle$ ונחשב את C' . כלומר בפעם השנייה M' מחשבת לכמה צמתים אפשר להגיע מ- s כאשר מוחקים את t מ- G .

אם $C = C'$ אז $\langle G, s, t \rangle \in \overline{\text{Path}}$. אם $C \neq C'$ אז $\langle G, s, t \rangle \in \text{Path}$. M' תענה q_{acc} אם $C = C'$.

נסמן ב- C_k את מספר הצמתים שאפשר להגיע אליהם מ- s ב- G על ידי k צעדים או פחות.

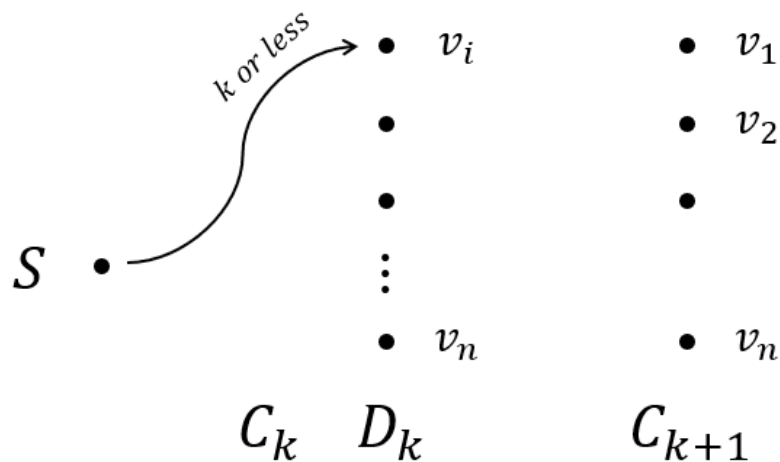
$C_0 = 1$, כי במסלול באורך $0 \geq$ אפשר להגיע ל- s בלבד.

$1 +$ מספר שכני s $C_1 = s$ במסלול באורך $1 \geq$ אפשר להגיע ל- s או לשכן שלו.

נראה דרך איך לחשב את C_{k+1} מתוך $\langle G, s, C_k \rangle$. המכונה M'' תפעל באופן הבא:

תתחיל מ- $C_0 = 1$ ותחשב בלולאה עד C_n כאשר n מספר צמתי G . באופן דומה נריץ את M'' כדי לחשב את C'_n עד C'_n עבור G' שהוא גרף כמו G אבל ללא הצומת t . המכונה M מחשבת את C_n ואת C'_n ועונה q_{acc} אם $C_n = C'_n$.

הערה: המכונה M'' עלולה בכל שלב של הריצה לעבור למצב q_{rej} אבל אם $\langle G, s, t \rangle \in \overline{\text{Path}}$ אז יש ריצה שבה M'' לא מחזירה q_{rej} אף פעם. כלומר מחשבת נכון את הערכים C_k .



נתרכז במשימת החישוב של C_{k+1} מתוך C_k . לכל צומת $v_1, \dots, v_n$ נרצה לדעת בוודאות האם ניתן להגיע אליו במסלול באורך $k + 1 \geq$. אם כן, נעלה את המונה ב- C_{k+1} , ואם לא אז לא. בסוף התהליך, נקבל ערך מדויק של C_{k+1} . איך נדע בוודאות האם אפשר להגיע ל- v_1 למשל, תוך $k + 1$ צעדים? אפשר, אם יש צומת v_i שמגיעים אליו ב- k או פחות צעדים וממנו קשת אחת אל v_1 . לכן לכל צומת $v_i \in V(G)$, מועמד להיות הלפני-אחרון במסלול מ- s ל- v_1 , ננחש מסלול מ- s ל- v_i באורך $k \geq$. אם הצלחנו להגיע ל- v_i נעלה מונה D_k ב- 1 , ונבדוק האם יש גם קשת (v_i, v_1) . אחרי שעברנו על כל ההצבות ל- v_i נבדוק האם $D_k = C_k$ (כאשר C_k ידוע לנו מהאיטרציה הקודמת). אם כן, סימן שעברנו על כל הצמתים במרחק $k \geq$, ואם לא סימן שפספסנו לפחות אחד. לכן, אם כן ולא הצלחנו להגיע ל- v_1 אז בוודאות לא ניתן להגיע אליו, אבל אם $D_k \neq C_k$ נעצור ונחזיר q_{rej} . באופן דומה ל- v_1 , נעשה לכל הצמתים $v_2, \dots, v_n$ כאשר את D_k מאפסים בכל פעם.

מקום: עבור D_k, C_k, C_{k+1} נדרש $O(\log n)$. עבור ניחוש מסלול מ- s נדרש $O(\log n)$. עבור ניחוש מסלול מ- s $O(\log n)$, כי מנחשים מסלולים צעד אחרי צעד וסופרים כמה צעדים היו. עבור C, C' נדרש $O(\log n)$. סה"כ $O(\log n)$.

נכונות: לכל k , אם לא החזרנו q_{rej} , מובטח ש- C_{k+1} חושב נכון, מתוך C_k . ולכן, גם C וגם C' מחושבים נכון (בריצה שבה לא מחזירים q_{rej}) כלומר M עונה נכון על $\overline{\text{Path}}$.

מחלקות סיבוכיות עם אקראיות

נרצה להרחיב את מודל מכונת טיורינג כך שיאפשר אלגוריתמים אקראיים שבהם זמן הריצה ותוצאת הריצה יכולות להיות תלויות במטבעות אקראיים. נוסיף למ"ט סרט של מטבעות אקראיים בתחילת הריצה בסרט הזה מופיעים 1 או 0 בכל תא בהסתברות שווה ובלתי-תלויה בין התאים.

קלט ועבודה	w	
------------	-----	--

מטבעות לקריאה בלבד	1	1	0	1	0	
--------------------	---	---	---	---	---	--

נגדיר מחלקות סיבוכיות עבור מודל מ"ט זה:

הגדרה - ZPP: מחלקת כל השפות L כך שיש עבורן מ"ט M שמכריעה את L ורצה בתוחלת זמן פולינומי (התוחלת על פני הגרלות המטבע השונות). כלומר, $M(w) = q_{acc}$ אם $w \in L$ ואחרת $M(w) = q_{rej}$. זמן הריצה בתוחלת (על פני המטבעות האקראיים) חסום על ידי פולינום.

[טענה](#)

$$P \subseteq ZPP$$

הוכחה: אם $L \in P$ אז יש מ"ט דטרמיניסטית פולינומית שמכריעה אותה. אפשר להוסיף לה סרט מטבעות ואז נקבל מכונה שרצה תמיד בזמן פולינומי, ובפרט בתוחלת בזמן פולינומי.

הגדרה - RP: מכילה את כל השפות L כך שקיימת מ"ט M שרצה בזמן פולינומי (תמיד), ומתקיים: אם $w \in L$ אז $\mathbb{P}[M(w) = q_{acc}] \geq \frac{1}{2}$, ואם $w \notin L$ אז $\mathbb{P}[M(w) = q_{rej}] = 1$.

[טענה](#)

$$RP \subseteq NP$$

הוכחה: נסתכל על מוודא פולינומי לשפה ב-NP לעומת מכונת RP. בשני המקרים:

- זמן הריצה פולינומי
- אם $w \notin L$ אז $M(w) = q_{rej}$ תמיד.
- אם $w \in L$ אז יש ריצה שעבורה $M(w) = q_{acc}$. במכונת RP הדרישה היא שחצי מהריצות לפחות מסיימות ב- q_{acc} .

לכן מכונת RP היא מקרה פרטי של מוודא פולינומי.

טענה

$$P \subseteq RP$$

הוכחה: בשני המקרים המכונה רצה בזמן פולינומי. בשני המקרים המכונה עונה q_{rej} תמיד עבור $w \notin L$. מכונה ב-P מחזירה תמיד q_{acc} לקלט $w \in L$ ובפרט מחזירה q_{acc} בסיכוי $\geq \frac{1}{2}$.

הגדרה - coRP: המחלקה coRP מכילה כל שפה L כך שמתקיים $\bar{L} \in RP$. לחלופין: $L \in \text{coRP}$ אם"ם קיימת מ"ט M עבור L כך שמתקיים:

- M רצה בזמן פולינומי (תמיד).
- לכל $w \in L$ מתקיים $M(w) = q_{acc}$ (תמיד).
- לכל $w \notin L$ מתקיים $\mathbb{P}[M(w) = q_{rej}] \geq \frac{1}{2}$.

הסבר:

$$L \in \text{coRP} \Leftrightarrow \bar{L} \in RP \Leftrightarrow \bar{L} \text{ עבור } RP \text{ מ"ט יש} \Leftrightarrow L \text{ עבור } \text{coRP} \text{ מ"ט יש}$$

המעבר הימני ביותר על ידי החלפת מצבים q_{acc}, q_{rej} .

טענה

$$RP \cap \text{coRP} = ZPP$$

הגדרה - BPP: המחלקה BPP מכילה את כל השפות L כך שיש מ"ט M שרצה בזמן פולינומי עונה נכון בסיכוי $\leq \frac{2}{3}$ (אבל מותרת טעות דו-צדדית). כלומר: אם $w \in L$ אז $\mathbb{P}[M(w) = q_{acc}] \geq \frac{2}{3}$ ואם $w \notin L$ אז $\mathbb{P}[M(w) = q_{rej}] \geq \frac{2}{3}$.

טענה

$$P \subseteq BPP$$

הסבר: בשני המקרים המכונה רצה בזמן פולינומי. מכונה דטרמיניסטית פולינומיות שמכריעה את L בפרט מבטיחה שהסיכוי להשיב נכון הוא תמיד $\geq \frac{2}{3}$.

טענה

$$BPP \subseteq EXP$$

הסבר: תהי $L \in BPP$. תהי M מכונת BPP עבור L . נבנה בעזרתה מכונה M' שמכריעה את L בזמן אקספוננציאלי. M' תפעל באופן הבא:

תעבור על כל האפשרויות למטבעות אקראיים. לכל אפשרות (של מחרוזת מעל $\{0,1\}$) M' תריץ את $M(w)$. M' תספור כמה פעמים התקבל q_{acc} ותענה לפי הרוב.

הנכונות ברורה. נותר לנתח את זמן הריצה: כל הריצה של $M(w)$ דורשת זמן ריצה $\text{Poly}(n)$. כמה ריצות יש? 2^ℓ כאשר ℓ מספר התאים בסרט המטבעות. אבל כיוון ש- M רצה בזמן פולינומי היא יכולה להספיק לכל היותר מספר פולינומי של מטבעות, כלומר $\ell \leq \text{Poly}(n)$. לכן סה"כ זמן ריצה של M' הוא $\geq 2^{\text{Poly}(n)}$, כלומר חסום אקספוננציאלי ב- n .

בעיה

נרצה לדון על היחס בין RP, BPP . לפני כן, נראה קודם שגודל הטעות הוא קבוע קטן כרצוננו. נסמן $\text{RP}(p)$ כמו המחלקה RP אבל עם טעות חד צדדית $p \geq \frac{1}{2}$. המחלקה שהגדרנו לעיל היא בעצם $\text{RP} = \text{RP}\left(\frac{1}{2}\right)$. נבחין שעם הגדרה זו מתקיים $\text{RP} = \text{RP}\left(\frac{1}{2}\right) = \text{RP}\left(\frac{1}{4}\right) = \dots$. כלומר $L \in \text{RP}$ אם"ם קיימת מ"ט שרצה בזמן פולינומי ומקיימת שאם $w \in L$ אז $\mathbb{P}[M(w) = q_{acc}] \geq \frac{3}{4}$ ואם $w \notin L$ אז $\mathbb{P}[M(w) = q_{rej}] = 1$. ברור שמתקיים $\text{RP}\left(\frac{1}{2}\right) \supseteq \text{RP}\left(\frac{1}{4}\right)$. נותר להראות את ההכלה בכיוון השני.

תהי $L \in \text{RP}$, כלומר קיימת מ"ט M שרצה בזמן פולינומי ויש לה טעות חד-צדדית בסיכוי $\geq \frac{1}{2}$. נבנה בעזרתה מ"ט M' עבור L שרצה בזמן פולינומי ויש לה טעות חד-צדדית בסיכוי $\geq \frac{1}{4}$.

בנייה: M' תפעל באופן הבא – תריץ את M על w פעמיים, ותחזיר q_{rej} אם"ם בשתי הפעמים $M(w)$ החזירה q_{rej} .

זמן ריצה: זמן הריצה של M פולינומי, ולכן גם של M' (פי 2).

נכונות: אם $w \notin L$ אז $M(w) = q_{rej}$ תמיד, ולכן $M'(w) = q_{rej}$ תמיד.

אם $w \in L$ אז $\mathbb{P}[M(w) = q_{rej}] \leq \frac{1}{2}$, לכן הסיכוי $\mathbb{P}[M'(w) = q_{rej}] \leq \left(\frac{1}{2}\right)^2 = \frac{1}{4}$.

באופן דומה, יכולנו להראות $\text{RP} = \text{RP}(p)$ לכל קבוע p קטן כרצוננו. כעת, נחזור ליחס ביניהן:

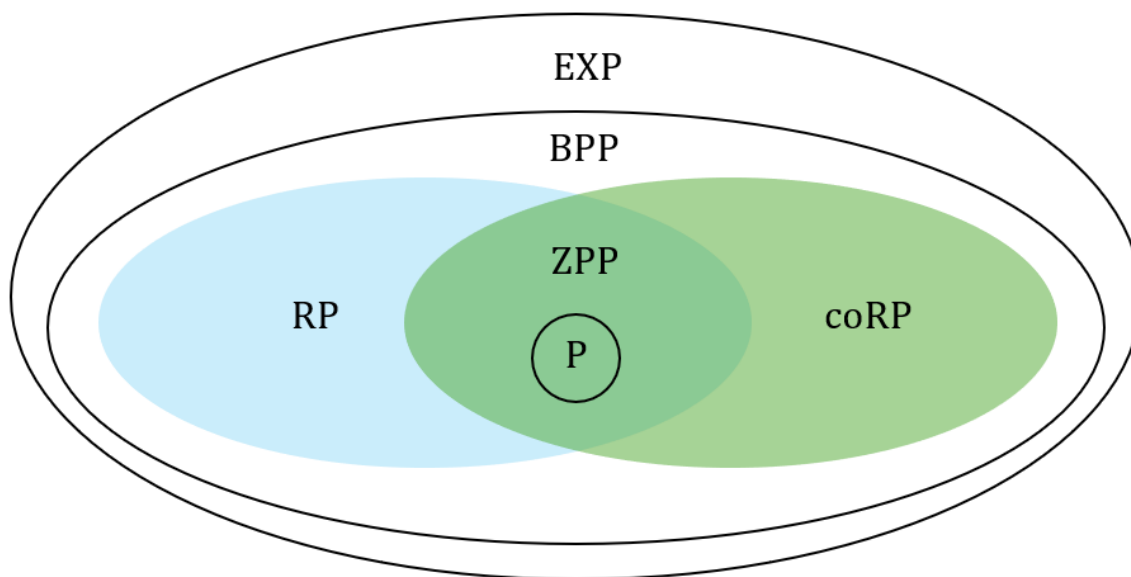
טענה

$$\text{RP} \subseteq \text{BPP}$$

הוכחה: $\text{RP} = \text{RP}\left(\frac{1}{3}\right)$, כלומר $L \in \text{RP}$ אם"ם יש מ"ט פולינומיאלית עם טעות חד-צדדית $\geq \frac{1}{3}$, מקרה פרטי

של מכונת BPP (שם מותר טעות דו-צדדית $\geq \frac{1}{3}$).

נרצה לקבל את התמונה המלאה:



(לא את כל ההכלולות ראינו, את השאר נוכיח מטה).

טענה

$$\text{coRP} \subseteq \text{BPP}$$

טענת עזר: $\text{BPP} = \text{coBPP}$ סגורה למשלים, כלומר $\text{BPP} = \text{coBPP}$.

הוכחת טענת עזר: מתקבלת על ידי החלפת מצבי q_{acc}, q_{rej} של מכונת BPP. נקבל מכונת BPP לשפה המשלימה.

הוכחת הטענה: $L \in \text{coRP}$ אז $\bar{L} \in \text{RP}$, כלומר $\bar{L} \in \text{BPP}$ ולכן $L \in \text{BPP}$ (מסגירות BPP למשלים).

שאלה

איפה NP משתלבת בדיאגרמה לעיל? אנחנו יודעים $P \subseteq ZPP \subseteq RP \subseteq NP \subseteq EXP$.

$$\text{RP} \cap \text{coRP} = \text{ZPP}$$

הוכחה:

כיוון ראשון: נניח $L \in \text{RP} \cap \text{coRP}$ ונראה $L \in \text{ZPP}$. כלומר, יש מכונת RP M_1 עבור L וגם יש מכונת coRP M_2 עבור L . שתי המכונות רצות בזמן פולינומי, כל אחת עם טעות חד צדדית (שונה).

M_1	M_2	
q_{acc} בסיכוי $\frac{1}{2}$	q_{acc} תמיד	$w \in L$
q_{rej} תמיד	q_{rej} בסיכוי $\frac{1}{2}$	$w \notin L$

נרצה בעזרת M_1, M_2 לבנות מ"ט M שהיא מכונת ZPP עבור L . כלומר, תמיד עונה נכון. זמן הריצה של M פולינומי בתוחלת (על פני המטבעות).

בניה: M תפעל באופן הבא:

- תריץ את $M_1(w)$. אם M_1 ענתה q_{acc} אז M תעצור ותחזיר q_{acc} .
- תריץ את $M_2(w)$. אם M_2 ענתה q_{rej} אז M תעצור ותחזיר q_{rej} .
- תחזור לשלב א'.

נכונות: אם בשלב כלשהו $M_1(w) = q_{acc}$ בהכרח $w \in L$ ואז התשובה של M נכונה. אם בשלב כלשהו $M_2(w) = q_{rej}$ אז בהכרח $w \notin L$ ואז התשובה של M נכונה. כלומר, אם M עונה – היא עונה נכון.

זמן ריצה: נניח $w \in L$. זמן הריצה של M הוא זמן הריצה של שתי המכונות כפול מספר הפעמים ש- $M_1(w)$ החזירה q_{rej} . נסמן ב- i את מספר הפעמים שזה קרה.

ולכן $\mathbb{P}[M(w) = q_{rej}] \leq \left(\frac{1}{2}\right)^i$ ו- $\mathbb{P}[\#(M(w) = q_{rej}) = i] \leq \left(\frac{1}{2}\right)^i$, לכן תוחלת זמן הריצה היא לכל היותר $\sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i \cdot i \cdot \text{Poly}(n)$ (הפולינום ב- n הוא בעקבות חסם על זמן ריצת המכונות). הטור שקיבלנו הוא טור הנדסי ומתקיים $\sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i \cdot i \leq \Theta(i)$ ולכן זמן הריצה הוא לכל היותר $\Theta(i) \cdot \text{Poly}(n) = \text{Poly}(n)$ (הוצגה הוכחת האינטואיציה לכך).

כיוון שני: נניח $L \in ZPP$ ונראה $L \in RP \cap coRP$.

נתחיל מלהראות $L \in RP \Rightarrow L \in ZPP$. כלומר קיימת מ"ט שמכריעה את L בזמן פולינומי בתוחלת. נבנה

בעזרתה מ"ט M_1 שרצה בזמן פולינומי (תמיד) ויש לה טעות חד-צדדית $\geq \frac{1}{2}$.

תזכורת: אי-שוויון מרקוב $\mathbb{P}[X > a \cdot \mathbb{E}[X]] \leq \frac{1}{a}$ עבור מ"מ אי-שלילי X .

נבנה את M_1 באופן הבא:

- M_1 תריץ את $M(w)$ עם מונה למספר צעדים ותעצור את הריצה אם עברו יותר מ- $2 \cdot \mathbb{E}[\#steps]$. אם

M עצרה לפני כן, אז M_1 תשיב כמו M , ואם M נעצרה בכוח אז M_1 תשיב q_{rej} .

זמן ריצת M_1 : פי 2 מתוחלת זמן ריצת M ולכן פולינומי ב- n .

נכונות: אם $w \notin L$ אז M_1 תמיד עונה נכון. או שעונה כמו M (שתמיד עונה נכון) או שעצרה את M בכוח

וענתה q_{rej} , שגם אז זו התשובה הנכונה.

אם $w \in L$ אז M_1 עונה נכון אם"ם $M(w)$ סיימה לרוץ בזמן $\geq 2 \cdot$ תוחלת. כלומר, M עונה לא נכון אם"ם זמן

ריצת $M(w) < 2$ פי 2 מהתוחלת, ולפי א"ש מרקוב זה קורה בסיכוי $\geq \frac{1}{2}$. כלומר אם $w \in L$ אז

$$\mathbb{P}[M'(w) = q_{acc}] \geq \frac{1}{2}. \text{ סיימנו להראות } ZPP \supseteq RP.$$

נראה עכשיו $L \in ZPP \Rightarrow L \in coRP$. כדי לעשות זאת, נבנה בעזרת M מכונת $coRP$ M_2 . הבנייה כמעט

בדיוק כמו M_1 , רק שבמקרה שעוצרים את M בכוח, M_2 תחזיר q_{acc} .